

FUJINET ENGINEERING SERIES

The ColecoVision Programmers Handbook

Programming the FujiNet cartridge in Z80 assembly language and C
The mailbox · a complete command reference · four worked programs

About this handbook

This handbook teaches you to program the FujiNet cartridge for the ColecoVision, in both Z80 assembly language and C. It covers the cartridge *mailbox* — the in-memory protocol the console uses to talk to the adapter — from first principles; a complete reference for every command the FujiNet answers, each with a worked example in both languages; and four real programs, explained end to end, with their full listings.

It is typeset in the FujiNet engineering series style, alongside *The FujiNet Network Protocol Handbook* and the *Platform Bring-Up Guide*. Every address, opcode, structure offset and code fragment is transcribed from the live project sources and verified against the firmware and a running adapter.

Canonical sources

<code>fujinet-firmware</code>	<code>pico/coleco</code> — the mailbox spec, RP2040 firmware, MAME device, and CONFIG (<code>d1e89c21e</code>)
<code>fujinet-lib</code>	the ColecoVision mailbox bus, in the portable C library (<code>dc399ec</code>)
<code>fujinet-5cardstud</code>	the 5 Card Stud ColecoVision port (<code>f2b5298</code>)
<code>fujinet-battleship</code>	the Battleship ColecoVision port (<code>c413cba</code>)
<code>fujinet-go-coleco-desktop</code>	the desktop emulator + FujiNet (<code>e6db9ec</code> , v0.2.0)
<code>os7lib</code>	the OS-7 BIOS binding for Z88DK (<code>a6254e8</code>)
<code>z88dk</code>	<code>zcc</code> and <code>z88dk-z80asm</code> (<code>3bd06cad56</code>)

Contents

1	Welcome to FujiNet	1
2	Your FujiNet Cartridge	2
3	Installing the Tools	4
4	The Mailbox	6
5	First Contact	10
6	The Z80 Assembler Library	12
7	C on the ColecoVision	14
8	The Network Device	16
9	The Fuji Device	17
10	Boot, Swap, and the Mappers	19
11	Command Reference: The Fuji Device	21
11.1	WiFi and the adapter	21
11.2	Hosts, slots, and mounting	23
11.3	Directories	26
11.4	Files, prefixes, and paths	27
11.5	App keys	28
11.6	Utilities: entropy, encoding, hashing, QR	28
11.7	Answered with a NAK on this platform	29
12	Command Reference: The Network Device	30
12.1	The lifecycle five	30
12.2	Structured channels, seek, and line discipline	32
12.3	Filesystems, sockets, and credentials	32
12.4	Answered with a NAK on this platform	33
13	Command Reference: The Other Devices	34
13.1	Disk — devices 31H–38H	34
13.2	Printer — device 40H	35
13.3	Clock — device 45H (APETime)	35
13.4	Modem — device 50H	35
14	Command Reference: The Cartridge Itself	37
15	Netcat	38
16	5 Card Stud	40
17	Battleship	42
18	Config	43
19	Always More To Come	45
20	FujiNet Go Coleco Desktop	46
A	Mailbox Quick Reference	48
B	Error and Status Codes	49
C	Program Listings	50
C.1	The shared client library	50
C.2	The Z80 assembler library	60
C.3	NETCAT	64
C.4	The milestone clients	69
C.5	CONFIG	72
C.6	5 CARD STUD (ColecoVision port)	77
C.7	BATTLESHIP (ColecoVision port)	87

D Troubleshooting	100
--------------------------------	------------

CHAPTER 1

Welcome to FujiNet

Your ColecoVision was sold in 1982 as “the Arcade Quality Video Game System.” Four decades on, the FujiNet cartridge gives it something the arcade never had: a network. Plug FujiNet into the cartridge slot and your ColecoVision is online. WiFi. Web servers. Disk hosts over TNFS. Telnet bulletin boards. Multiplayer game lobbies. All of it reachable from 3.58 megahertz of Z80.

FujiNet is a family of network adapters for classic computers and consoles. On every platform it presents the same two personalities: the *Fuji device*, which manages the adapter itself — WiFi, host slots, directories, mounting — and the *network device*, the famous `N:`, which opens connections to the outside world by URL. What differs per platform is only the plumbing between the machine and the adapter. On the ColecoVision that plumbing is a small, strict, rather beautiful protocol called the *mailbox*, and it is the subject of this book.

What this handbook teaches

You will learn, in order: how the FujiNet cartridge works and why the ColecoVision edition is the way it is (a cartridge port that cannot write!); how to install the tools; the mailbox, the in-memory protocol every program uses to talk to the cartridge; how to drive it from Z80 assembly language and from C with the Z88DK compiler; a complete reference for every command the FujiNet answers, each with a worked example in both languages; and then four real programs, explained end to end, with their listings in the appendix: **NETCAT**, **5 CARD STUD**, **BATTLESHIP**, and **CONFIG**.

What you will need

1. A ColecoVision — or MAME’s `colecto` driver with the FujiNet cartridge device grafted in (the firmware tree’s `pico/colecto/emu/apply.sh` does the grafting).
2. A FujiNet cartridge — or, under MAME, a running *fujinet-pc* with its bus-over-IP listener on `127.0.0.1:9995`.
3. The Z88DK compiler (for `zcc` and `z88dk-z80asm`) and the `os7lib` BIOS binding library. The next chapter installs both.
4. This handbook, a cup of coffee, and a colour television. The automatic colour circuitry will forgive long debugging sessions.

NOTE

Everything network-side — the URL schemes the `N:` device speaks, from `TCP:` and `TELNET:` to `HTTPS:`, `JSON:`, `SSH:` and `GMAIL:` — is a book of its own: *The FujiNet Network Protocol Handbook*, from this same series. This handbook teaches you to reach the `N:` device; that one teaches you everything it can say once reached.

CHAPTER 2

Your FujiNet Cartridge

The ColecoVision cartridge port is the most asymmetric in the FujiNet family. Its thirty pins carry fifteen address lines (A0–A14), eight data lines (D0–D7), four pre-decoded *chip selects* (one for each 8K block of the cartridge window), power and ground. That is the whole interface.

Count what is missing: no */RD*. No */WR*. No */MREQ*, no clock, no reset line. The four chip selects come out of the console’s address decoder — a 74LS138 with A13/A14/A15 on its select inputs and */MREQ* and */RFSH* as its enables — and they assert for **reads and writes alike**. The cartridge cannot tell the two apart. This is not a FujiNet limitation; it is why every real ColecoVision mapper — MegaCart, Activision, X-in-1 — takes its bank number from the *address*. The hardware left them no choice, and it leaves us none either.

So how can a read-only cartridge be a *network adapter*? With the same trick the mappers use, in both directions:

1. The console talks by reading. Certain pages of the cartridge window are *hotspots*. When the console reads an address inside one, the cartridge does not care what byte it returns — it cares **which address was read**. The low eight address bits are the payload. Reading `$FF41` “sends” the byte `$41`.

2. The cartridge talks by repainting. The cartridge serves its 32K window out of RAM on its own processor. To answer, it simply changes the bytes it serves; by the time the console reads them, they are just ROM.

A happy consequence of that decoder: the chip selects are inhibited during Z80 refresh, so the R register’s traffic on the low address lines never reaches the cartridge. The Astrocade port had to defend against exactly that hazard; here the console filters it out for us.

The tandem inside the cartridge

Two processors share the cartridge shell. An **RP2040** sits on the cartridge edge, serves the 32K window at Z80 bus speed, decodes the hotspot reads, and repaints the reply bytes. It speaks to an **ESP32-S3** — a stock FujiNet — over a USB cable, using the same SLIP-framed FujiBus packets FujiNet speaks over a serial port elsewhere. The ESP32-S3 is the actual network adapter: WiFi, TNFS, HTTP, TLS, the lot. The RP2040 is its translator into read-only cartridge language.

Under MAME there is no silicon, but the shape survives: the MAME cartridge device compiles the very same mailbox sources and forwards the FujiBus packets over TCP — *bus-over-IP* — to a *fujinet-pc* process on your desk.



Figure 1. The three-tier stack. The mailbox is the only part your program ever sees.

The 32K window

The cartridge window is 32,768 bytes at console addresses `$8000` – `$FFFF`. Your program gets nearly all of it. The mailbox owns only the top 2K; the contract, from `fuji_mailbox.h` (the single source of truth, compiled into the RP2040 firmware and the MAME model alike):

1. Program code and data stay **below** `$F800` — 30K, cart offsets `$0000` – `$77FF`.
2. `$F800` – `$FBFF` is the **reply window**, one kilobyte the cartridge repaints.
3. `$FC00` – `$FC0C` are the **status bytes**, and `$FCFC` carries the *claim signature*.
4. `$FD00`, `$FE00` and `$FF00` are the three **hotspot pages** the console reads to talk.

Why the ColecoVision is scarce, not the window

Every other cartridge port in the FujiNet family pages its reply through a 256-byte slice, because its window was too small to do otherwise. Here the window is 32K and the *console* is what is scarce: the ColecoVision has one kilobyte of RAM, of which the BIOS keeps `$73B9 – $73FF` and a C program's variables and stack take most of the rest — call it 700 usable bytes. So this port spends its big window on the console's behalf: the whole one-kilobyte reply is handed over as directly addressable ROM, and a program reads a directory entry or a game state *in place*, out of the cartridge, without a copy in RAM. Keep that in mind; it shapes every program in this book.

CAUTION

Programs that use the mailbox run with maskable interrupts **disabled** — `DI` at entry. But the ColecoVision's vblank interrupt is wired to the Z80's **/NMI**, which `DI` cannot stop, and it *will* land in the middle of a mailbox transaction. That is harmless, provided your NMI handler never reads `$F800` or above. This is the single most important rule on the platform, and the next chapters earn it.

CHAPTER 3

Installing the Tools

Everything in this book is built with **Z88DK**, a mature cross-compiler and assembler for the Z80. From it you get `zcc` (the C compiler front end and the C clients' toolchain) and `z88dk-z80asm` (the assembler used for the pure-assembly examples). The ColecoVision C clients also link against **os7lib**, a small library that binds the ColecoVision's own BIOS (OS-7) routines — `mode_1`, `load_ascii`, `put_vram`, the controller poller — so a C program gets a 32×24 text screen and debounced input for nothing.

Installing Z88DK

The quickest route is a nightly build; the surest, for a machine that must match this book, is a source build. Both give you the same `zcc` and `z88dk-z80asm`.

A. The nightly build (fastest)

1. Download the current nightly for your platform from `nightly.z88dk.org` (Windows and macOS get complete binary packages; Linux gets a source package). Unpack it somewhere permanent, say `~/z88dk`.
2. Point two environment variables at it, and put its `bin` on your path, in your shell profile (below).
3. Check it: `zcc +coleco` should report the ColecoVision target, and `z88dk-z80asm` should print its version.

SHELL PROFILE

```
export Z88DK=$HOME/z88dk
export PATH="$Z88DK/bin:$PATH"
export ZCCCFG="$Z88DK/lib/config"
```

B. From source (surest)

Z88DK uses git submodules, so clone recursively and run its build script. This is the path that matches the sources in this book.

BUILDING Z88DK FROM SOURCE

```
git clone --recursive https://github.com/z88dk/z88dk.git
cd z88dk
export PATH="$PWD/bin:$PATH"
export ZCCCFG="$PWD/lib/config"
./build.sh # Linux/Unix; see the wiki for Windows
# then, from anywhere:
zcc +coleco -v
```

Installing os7lib

The C clients build against **os7lib**, a thin C binding of the ColecoVision BIOS. Clone and build it, then remember where it landed — the C programs' `build.sh` reads its path from the `OS7LIB` variable (default `~/Workspace/os7lib`).

OS7LIB

```
git clone https://github.com/tschak909/os7lib.git
cd os7lib
make # produces os7.lib
export OS7LIB=$PWD
```

The test bench: MAME and fujinet-pc

You do not need cartridge hardware to develop for FujiNet — in fact, at the time of writing, ColecoVision FujiNet hardware does not yet exist. The whole stack runs on your desk:

1. **fujinet-pc** is the ESP32 firmware built to run as a desktop process. Start it; its bus-over-IP listener waits on `127.0.0.1:9995`.
2. **MAME** emulates the ColecoVision. The firmware tree carries a MAME cartridge device that compiles the cartridge's own mailbox sources and forwards FujiBus packets to fujinet-pc over TCP. Graft it in once:

GRAFTING THE FUJINET CARTRIDGE INTO MAME

```
cd ~/Workspace/fujinet-firmware/pico/coleco
./emu/apply.sh ~/Workspace/mame
make -C ~/Workspace/mame -j$(nproc) NOWERROR=1 REGENIE=1
# REGENIE=1 is needed the FIRST time (the build files changed);
# re-run apply.sh after every edit to the cartridge sources.
```

With both in place, `pico/coleco/run.sh` boots a built client in MAME against the live fujinet-pc — windowed, or headless with a one-line PASS/FAIL verdict. Every program in this book was built and verified exactly this way.

Building a client

Each C client in this book carries a `build.sh` that is the firmware testrom's recipe narrowed to one program. It compiles with `zcc +coleco`, forces the image to exactly 32,768 bytes, warns if any byte reaches the mailbox pages, and stamps the claim signature. The next chapters explain every part of that recipe; for now, the shape:

THE SHAPE OF A BUILD

```
zcc +coleco -O2 -I$OS7LIB/src -I../common \
  program.c $SHARED -o build/program -create-app \
  -Cz--romsize=32768 -Cz--rombase=32768 \
  -Cz--code-fence=0xF800 -Cz--data-fence=0xF800 \
  -pragma-define:CRT_ORG_BSS=0x702C \
  -pragma-define:REGISTER_SP=0x73B8 \
  -L$OS7LIB -los7
python3 checkrom.py --stamp build/program.bin
```

IMPORTANT

The C clients build against **fujinet-lib-experimental**, the branch of the portable FujiNet C library that carries the ColecoVision mailbox bus. The two games in this book fetch it automatically through their `make-exp` script; the standalone examples call the mailbox directly through the small library this book develops. Both talk to the identical mailbox.

CHAPTER 4

The Mailbox

This chapter is the heart of the handbook. The mailbox is a small, strict protocol; learn its five moves and every FujiNet feature on this console is yours. All addresses below are **console addresses** — cart offset plus `$8000` — exactly as your program uses them.

The two directions

Console to cartridge: hotspot reads. Three pages carry everything you will ever say. A read of `$FD00+r` arms mailbox register `r`. A read of `$FE00+v` delivers the value `v` to the armed register — one register write is therefore a *pair* of reads, back to back. A read of `$FF00+v` appends the byte `v` to the *TX stream*, the outgoing parameter-and-payload buffer.

Cartridge to console: repainted ROM. The reply window at `$F800` — `$FBFF` is plain memory you read whenever you like — the whole one-kilobyte reply, in one piece. Above it, `$FC00` — `$FC0C` are thirteen status bytes the cartridge paints.

Instruction fetches are harmless: your code executes from `$8000` upward, far below the hotspot pages, so the thousands of reads the Z80 performs just running your program never trip anything.

Console	Name	What lives there
<code>\$8000-\$F7FF</code>	—	your program: 30K of code and data
<code>\$F800-\$FBFF</code>	<code>FN_REPLY</code>	the whole 1K reply, in one piece
<code>\$FC00-\$FC0C</code>	status	thirteen painted status bytes
<code>\$FCFC-\$FCFF</code>	claim	<code>FUJI</code> signature (cart offset <code>\$7CFC</code>)
<code>\$FD00-\$FD7F</code>	<code>FN_REGSEL</code>	hotspot: read <code>+r</code> arms register <code>r</code>
<code>\$FD7F-\$FDFF</code>	<code>FN_SWAP</code>	hotspot: serve the staged boot image (armed only)
<code>\$FE00-\$FEFF</code>	<code>FN_REGDAT</code>	hotspot: read <code>+v</code> delivers <code>v</code> to the armed register
<code>\$FF00-\$FFFF</code>	<code>FN_TXPAGE</code>	hotspot: read <code>+v</code> appends <code>v</code> to the TX stream

Figure 2. The cartridge window, as the console sees it.

Address	Name	Meaning
<code>\$FC00</code>	<code>FN_ACKSEQ</code>	echoes your SEQ when the reply is ready
<code>\$FC01</code>	<code>FN_STATUS</code>	bit 0 link up (bit 1 defined but never set)
<code>\$FC02</code>	<code>FN_ERR</code>	transport verdict of the last transaction
<code>\$FC03</code>	<code>FN_REPLYCMD</code>	<code>\$06</code> ACK or <code>\$15</code> NAK
<code>\$FC04</code>	<code>FN_RXLEN_LO</code>	reply length, low byte
<code>\$FC05</code>	<code>FN_RXLEN_HI</code>	reply length, high byte
<code>\$FC06</code>	<code>FN_BOOTSTAT</code>	boot state: idle 0, xfer 1, ready 2, failed <code>\$80</code>
<code>\$FC07</code>	<code>FN_BOOTPCT</code>	boot progress, 0–100
<code>\$FC08</code>	<code>FN_BOOTERR</code>	boot error code
<code>\$FC09</code>	<code>FN_MAGIC0</code>	the letter <code>F</code> — cartridge presence
<code>\$FC0A</code>	<code>FN_MAGIC1</code>	the letter <code>N</code>
<code>\$FC0B</code>	<code>FN_PROTOVER</code>	mailbox protocol version (1)
<code>\$FC0C</code>	<code>FN_SLICE_ECHO</code>	slice echo — vestigial here, always 0

Figure 3. The thirteen painted status bytes at `$FC00`.

The registers

Register numbers `$00` — `$0F` mirror the FujiNet mailbox on its sibling cartridge ports where the meanings match; `$10` and up are shared with the Astrocade numbering.

<code>\$00</code>	DEVICE	FujiBus device id: <code>\$70</code> Fuji, <code>\$71</code> — <code>\$78</code> network
<code>\$01</code>	COMMAND	FujiBus command id
<code>\$02</code>	NPARAM	how many parameters lead the TX stream
<code>\$05</code>	DATA RST	any value: rewind the TX write pointer
<code>\$06</code>	RXSLICE	reply slice — vestigial here, one slice only
<code>\$10</code>	SEQ	nonzero and $\neq$ last ACK: launch the transaction
<code>\$11</code>	BOOTLOCK	magic <code>\$B5</code> arms the ROM swap
<code>\$12</code> / <code>\$13</code>	BOOTSEL	<code>\$B5</code> then <code>\$4A</code> : reboot cart to firmware update mode

Registers `$03` , `$04` , `$07` — `$0F` and `$14` — `$7F` are undefined and do nothing — reading their REGDATA is a silent no-op.

The TX stream

Every transaction's outgoing bytes form one stream, at most **320 bytes**, with a fixed grammar: first the parameters — NPARAM of them, each a *size byte* (1, 2 or 4) followed by that many value bytes, little-endian — then the raw payload, whose length is simply whatever remains. At most eight parameters are unpacked.



Figure 4. The TX stream for `NET_OPEN` with two one-byte parameters and a devicespec payload.

One whole transaction

Every exchange with the FujiNet — from a WiFi scan to an HTTP GET — is the same five moves:

- 1. Begin.** Rewind the TX stream (register `$05`), then set DEVICE, COMMAND, and NPARAM.
- 2. Stream.** Append the parameters and payload with `$FF00+v` reads.
- 3. Commit.** Read `FN_ACKSEQ` , add one (wrapping 255 to 1 — zero is reserved), and write the result to the SEQ register. The cartridge sees a fresh sequence number and fires the whole stream at the ESP32-S3 as one FujiBus packet.
- 4. Wait.** Poll `FN_ACKSEQ` until it equals the number you wrote. When it does, the reply is painted: check `FN_ERR` (the transport verdict), `FN_REPLYCMD` (ACK or NAK), and capture the reply length from `FN_RXLEN_LO / HI` *immediately*.
- 5. Read.** The whole reply, up to 1,024 bytes, is already in the window at `$F800` . Read it in place.

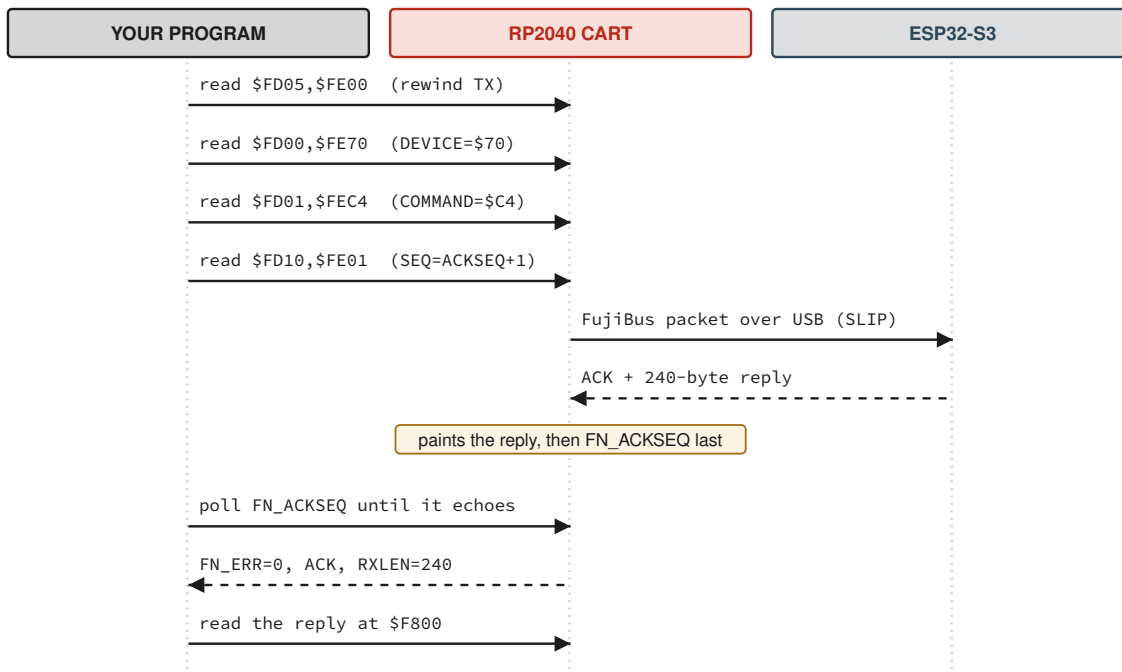


Figure 5. A `GET_ADAPTERCONFIG_EXTENDED` transaction, end to end.

The sequence rule

Why derive SEQ from the cartridge's own `FN_ACKSEQ` instead of counting locally? Because the RESET button restarts *your program* — but not the cartridge. The cart edge carries no reset line. A program that kept its own counter would, after a RESET, re-send sequence number 1; the cartridge, having already acknowledged a 1 this session, would ignore it forever, while the stale reply sat there looking like success. Read the cart's persisted `FN_ACKSEQ`, add one, wrap 255 to 1, and RESET cannot desynchronise you. This rule is inherited from hard experience on the Intellivision and Astrocade bring-ups; honour it.

The interlock

The cartridge paints `FN_ACKSEQ last`, after every other byte of the reply is already in place. That single ordered store is the whole handshake: the instant your poll sees the sequence match, everything else you are about to read is guaranteed to be there. There is nothing else to wait on — the slice-echo byte that the sibling ports poll is vestigial here, always zero, because the reply is one piece.

Stray reads, and the defences

On a port where reading is writing, the hazard is the read you did not mean to make — above all the vblank NMI, which cannot be masked and lands mid-transaction. The defences, in depth:

1. A REGDATA read with no immediately-preceding REGSEL read is a **no-op** — arming disarms after one use, so an isolated stray mutates nothing.
2. A transaction launches only on a SEQ value that is nonzero *and* differs from the last acknowledged one.
3. The ROM-swap trigger at `$FD0E` fires only after BOOTLOCK armed it; rebooting the cart to update mode takes two full register writes with two different magics.
4. Refresh cycles never assert a chip select, so the R register's traffic never reaches the cartridge.

The one rule these leave to you: **your NMI handler must never read `$F800` or above**. A handler that bumps a frame counter and reads the VDP status port is safe; one that peeks at the reply window can corrupt the transaction if interrupted. The firmware's `checkrom.py` enforces the image half of this — no code or data may live in the mailbox pages — so a program that drifts up there fails its build, not a debugging session.

Timeouts

The cartridge's own transaction budget is **five seconds**, except `MOUNT_IMAGE` (command `$F8`), which gets **sixty** because a network ROM push rides inside it. Your client should wait a little longer than that, so a real timeout is reported as the cart's error code rather than as your own. A commit that never sees the sequence echo returns the client-timeout sentinel.

CAUTION

The reply window is **never cleared** between transactions. A short reply leaves the stale tail of a longer, older one behind it, and bytes past the reported length read as zero. Capture `FN_RXLEN` the moment a transaction acknowledges, believe only that many bytes, and validate what they claim to be before acting on them.

CHAPTER 5

First Contact

Enough theory — let us shake hands with the cartridge. The smallest useful FujiNet program does three things: proves the cartridge is there, runs one transaction, and shows what came back. This chapter builds it twice — once in Z80 assembly, once in C — and every program in this book, games included, is this program with a bigger middle. The assembly version is `acfg.asm`, driving the mailbox directly; the C version uses the portable **fujinet-lib** the games link. (The appendix's `fujitest.c` is the same round trip written against the low-level library, for comparison.)

Is anybody home?

The cartridge paints the letters **F** and **N** at `$FC09` and `$FC0A`. If they are not both there, you are running on a bare console — or a different cartridge — and must not touch the hotspot pages. Every client checks this first, and so should you.

One transaction, no parameters

`GET_ADAPTERCONFIG_EXTENDED` (command `$C4` to device `$70`) is the perfect first transaction: it takes no parameters and no payload, works even before WiFi is configured, and returns a 240-byte structure whose best fields sit at convenient offsets — the SSID at 0, the firmware version at 125, and the IP address, already in dotted-decimal text, at 140.

Z80 ASSEMBLER

```
call    FNPRES      ; Z if 'F','N'
jr      nz,NO_CART
ld      d,$70       ; device
ld      e,$C4       ; ACFG_EXT
call    FNSTART
call    FNCOMMIT
jr      c,TIMEOUT   ; never answered
or      a
jr      nz,LERR     ; FN_ERR != 0
call    FNACKED
jr      nz,GOTNAK
; reply is at $F800: ssid +0,
; fw +125, ip +140.
```

Z88DK C

```
AdapterConfigExtended ac;

if (!fuji_coleco_present()) {
    show("NO FUJINET CART");
    for (;;) ;
}
if (!fuji_get_adapter_config_extended(&ac))
    fail();

show(ac.ssid);
show(ac.sLocalIP); /* dotted text */
show(ac.fn_version);
```

What success looks like

Build either version and run it. Against a live FujiNet — or `fujinet-pc` under MAME — the screen answers in about a second:



Figure 6. The assembly demo's first contact, captured from MAME against a live fujinet-pc.

Read the error paths as carefully as the happy one. At the mailbox there are three layers. **Carry set** from `FNCOMMIT` in assembly (the low-level C library returns the sentinel `FN_EWAIT`) means the cartridge never acknowledged — almost always “no cartridge” or “the RP2040 never enumerated its ESP32.” A **nonzero** `FN_ERR` means the cartridge answered but the transport beneath it failed: `1` no link, `2` timeout, `3` bad frame, `4` too big. A **NAK** in `FN_REPLYCMD` means everything carried perfectly and the FujiNet itself refused the command — wrong parameters, an unmounted host, a file that is not there.

Three different layers, three different bytes, three different remedies. fujinet-lib folds them together for you: a `fuji_*` call returns `false` on any failure, and a `network_*` call returns a non-`FN_ERR_OK` code — but the three bytes are still there underneath when you need to tell the cases apart.

CHAPTER 6

The Z80 Assembler Library

The mailbox is nothing but memory reads, and reading memory is what a Z80 does best. This chapter walks the assembly library `fujimail.inc` — printed in full in the appendix and used by the assembly examples throughout — routine by routine. Know these nine and you know the mailbox by heart.

Calling conventions are register-based and stated at each routine; two scratch bytes, `NPARAM` and `WANT`, live in low RAM. In assembly there is no volatile hazard: `ld a,(hl)` always performs the read, so the “read whose value we discard” needs no ceremony — it is simply a load.

FNREGWR — one register write

The atom everything is built from. `C` names the register, `A` the value; the routine issues the REGSEL/REGDATA read pair, back to back. The read whose result is thrown away *is* the message.

FNREGWR, FUJIMAIL.INC

```
; C = register number, A = value.
FNREGWR:
    push    af          ; hold the value
    ld      h,$FD       ; REGSEL page
    ld      l,c
    ld      a,(hl)       ; arm the register
    pop     af
    ld      h,$FE       ; REGDATA page
    ld      l,a
    ld      a,(hl)       ; deliver the value
    ret
```

Nothing requires the two reads to be adjacent — ordinary instruction fetches between them are invisible to the hotspot decode, and the vblank NMI can land there harmlessly because the armed register simply waits. The library keeps them adjacent as hygiene.

FNTXB, FNP8, FNP16 — feeding the stream

`FNTXB` appends the byte in `A` with a single `$FF00+v` read. `FNP8` wraps it for the parameter grammar — size byte 1, then the value, then bump `NPARAM`. `FNP16` sends a two-byte parameter: size byte 2, low, high. The raw payload after the parameters is just `FNTXB` in a loop, or `FNTXPAD`, which streams a string and pads it with NULs to a fixed width — the shape `SET_DEVICE_FULLPATH` and `OPEN_DIRECTORY` demand.

FNSTART — opening the envelope

`D` = device, `E` = command. Rewinds the TX pointer (register `$05`) and writes `DEVICE`, `COMMAND` and `NPARAM=0`. Because `FNREGWR` touches only `A`, `H` and `L`, the `D` and `E` you loaded survive the call. Afterwards the stream is empty and waiting.

FNCOMMIT — the moment of truth

Computes the sequence rule from the last chapter — `FN_ACKSEQ` + 1, wrap 255 to 1 — writes `SEQ`, and polls. On success it returns with `A = FN_ERR` and the flags set from `or a`, so `jr nz,error` reads naturally; on a client-side timeout it returns with carry set.

FNCOMMIT, FUJIMAIL.INC -- THE SEQUENCE RULE IN THE FLESH

```

FNCOMMIT:
    ld    a,(FN_ACKSEQ)
    inc   a
    jr    nz,FNCM1
    inc   a           ; 255 wrapped; step past reserved 0
FNCM1:   ld    (WANT),a
    ld    c,FNR_SEQ
    call  FNREGWR     ; A = want
    ld    b,40        ; outer quanta; inner is a full spin
FNCM0:   ld    de,0
FNCMI:   ld    a,(FN_ACKSEQ)
    ld    hl,WANT
    cp    (hl)
    jr    z,FNCMOK
    dec   de
    ld    a,d
    or    e
    jr    nz,FNCMI
    djnz  FNCM0
    scf                     ; timed out
    ret
FNCMOK:  ld    a,(FN_ERR)
    or    a               ; clears carry
    ret

```

FNACKED, FNRLN, FNPRES — reading the answer

FNACKED returns Z set when **FN_REPLYCMD** is **\$06** (ACK). **FNRLN** loads **HL** from **FN_RXLEN** in one `ld hl,(nn)` — the length is little-endian, so the low byte at **\$FC04** becomes **L** and the high at **\$FC05** becomes **H**, for free. **FNPRES** compares **\$FC09 / \$FC0A** against **F, N** and returns Z set when the cartridge is present; call it once at boot, before any hotspot is touched.

That is the whole library. Everything else in this book — the C client, the games, CONFIG — is these routines, or their C twins, with a purpose.

CHAPTER 7

C on the ColecoVision

The mailbox is memory reads, and C reads memory too. This book’s C clients build with Z88DK’s `+colemco` target against `os7lib`, and that is not a shortcut — it is the design. The ColecoVision window is 32K, so C’s overhead is affordable, and “use OS7 for everything” gives every client a 32×24 text screen, a debounced controller, and a font, all from the console’s own BIOS. The C mailbox library is `fujilib.c`; it mirrors `fujimail.inc` function for function.

The build, part by part

The link line looks alarming but each pragma earns its place. The ColecoVision’s whole RAM is one kilobyte; after OS7’s tables and the C stack, a client has about 700 usable bytes. The layout is pinned by hand:

```
+colemco ..... the ColecoVision target: crt0, header, VDP glue
romsize/rombase 32768 ..... force a full 32K image based at $8000
code-fence/data-fence $F800 warn if anything reaches the mailbox pages
CRT_ORG_BSS=0x702C ..... variables start just above the header’s tables
REGISTER_SP=0x73B8 ..... stack below the BIOS scratch at $73B9
CLIB_DEFAULT_SCREEN_MODE=-1 keep the crt0 out of the VDP; os7lib owns it
```

The guard macro for ColecoVision-only code is `BUILD_COLECO`, not `__COLECO__`: Z88DK defines the latter for the ADAM sub-target too (`+colemco -subtype=adam`), so the games use their own define to tell the two apart.

Reading as writing, in C

Here C needs one idiom, and one warning. The hotspot “write” is a read whose *address* matters and whose value does not. Written the obvious way — `(void)FN_REGSEL[reg];` — `sccz80` computes the address and then throws the load away, so the register write never happens and no transaction ever launches. Storing the read into a `volatile` sink is what forces the compiler to keep it.

FUJILIB.C -- THE TRANSPORT ATOM, AND THE TRAP IT DODGES

```
/* sccz80 discards (void)FN_REGSEL[reg]; the store into a
   volatile sink is what makes the read a real side effect.
   Check: zcc +colemco -O2 -a fujilib.c -- look for TWO loads. */
volatile unsigned char fn_sink;
#define FN_TOUCH(a) (fn_sink = *(volatile unsigned char *) (a))

static void fn_regwr(unsigned char reg, unsigned char val)
{
    FN_TOUCH(FN_REGSEL + reg); /* arm the register */
    FN_TOUCH(FN_REGDAT + val); /* deliver the value */
}
```

CAUTION

A second helping of the same compiler, and the subtler of the two: never index the reply window as `FN_REPLY[i]`. `sccz80` indexes the macro’s cast-constant differently from a `volatile unsigned char * local` and gets it wrong — the identical expression that draws a filename correctly through a local reads back zero through the macro, which shows up as a directory listing that looks perfect but whose end-of-directory test never fires. Always read the window through a pointer variable.

The rest of the library

`fujilib.c` continues exactly along `fujimail.inc`’s lines: `fn_start`, `fn_param8 / fn_param16`, `fn_tx / fn_tx_padded`, `fn_commit`, `fn_acked`, `fn_reply_len`, plus two streaming helpers the games and CONFIG lean on — `fn_tx_path`, which streams a filename *out of the reply window* and into the next transaction’s payload without

a RAM copy, and `fn_tx_from_reply`, which does the same for a run of bytes. On a machine with 700 bytes of RAM those are not optimisations; they are what makes a directory browser possible.

fujinet-lib, the portable road

The game clients — and every C example in this handbook’s command reference — use **fujinet-lib-experimental**, the portable FujiNet C library, rather than calling `fujilib.c` directly. Its ColecoVision bus is the same mailbox by another name: `network_open`, `network_read_nb`, `network_close`, `network_status`, and the `fuji_*` calls for the adapter — WiFi, host and device slots, directories, mounting, app keys, hashing, and the `fuji_coleco_*` boot helpers. The same source compiles for the Apple II, the C64 and the CoCo; only the bus underneath changes. `fujilib.c` is shown in this book because it *is* that bus, laid bare, so the portable library holds no mysteries — and the two `sccz80` traps above apply to it just the same: its coleco bus stores every hotspot read through a volatile sink, and reads the reply window through a pointer, for exactly the reasons given here. A handful of commands have no wrapper (disk and printer I/O, `RANDOM_NUMBER`); for those the reference drops to the low-level calls, and says so.

CHAPTER 8

The Network Device

Device `$71` is the first of eight network units — N1: through N8:, `$71` — `$78` — each an independent connection to somewhere in the world. You give one a *devicespec*, a URL with an `N:` prefix, and from then on it is a byte pipe with a status word.

```
N:TELNET://bbs.fozztexx.com/
```

```
N:TCP://192.168.1.10:6502/
```

The scheme decides everything: `TCP:`, `TELNET:`, `HTTP:`, `HTTPS:`, `TNFS:`, `JSON:`, `SSH:`, `UDP:` and more, each with its own grammar and behaviour. This handbook does not repeat that material — it fills a book of its own, *The FujiNet Network Protocol Handbook* — but everything there applies to the ColecoVision verbatim once you can drive the five commands below.

The lifecycle five

OPEN `$4F` mode and translation params, devicespec payload
STATUS `$53` four reply bytes: bytes waiting, connected, device status
READ `$52` up to 1,024 bytes into the reply window
WRITE `$57` bytes out — NETCAT is this book's user
CLOSE `$43` hang up

OPEN

Two one-byte parameters, then the devicespec as payload, padded to a fixed 256 bytes. The *mode* is a `fileAccessMode_t`: 4 read, 8 write, 12 read/write — and 12 also performs a GET on an HTTP adapter. The *translation* is 0 (none) for a binary stream. NETCAT opens a socket read/write.

STATUS

The heartbeat. On an open channel the reply is four bytes:

avail lo	avail hi	connected	devstatus
----------	----------	-----------	-----------

Figure 7. The `NET_STATUS` reply on an open channel.

`avail` is how many bytes wait to be read, little-endian. `connected` is the socket truth. `devstatus` is the protocol adapter's verdict: `1` SUCCESS, `136` end-of-file, and a table of per-protocol error codes the Protocol Handbook documents. A terminal like NETCAT watches `connected` — on a live socket, end-of-file is a hangup, not an error — while a game that fetches an HTTP body watches `devstatus`, because a GET is performed at *read* time and an error page still has a readable body that `connected` alone would happily render as garbage.

READ, WRITE, CLOSE

READ takes a one word count, clamped to what STATUS reported and to the 1,024-byte window; the reply lands at `$F800` and you capture its length that instant. **WRITE** carries the count as a parameter *and* the bytes as payload — so keep a write inside the 320-byte TX stream; NETCAT caps at 120. **CLOSE** takes nothing.

The one rule of discipline: nothing may run between a **READ** and the code that renders its bytes, because every transaction — **STATUS**, **CLOSE**, anything — repaints the reply window. NETCAT's loop is written around that: status, read, draw, in that order.

CHAPTER 9

The Fuji Device

Device `$70` is the adapter itself: WiFi, host slots, directories, disk images, and a drawer of utilities. CONFIG is its natural habitat — the one client in this book that lives on device `$70` — and this chapter follows CONFIG’s road through it. The complete command cards are in the reference chapter; here is how the pieces fit.

Getting on the air

CONFIG’s WiFi checkout: `SCAN_NETWORKS` (`$FD`) starts a scan and returns a count; `GET_SCAN_RESULT` (`$FC`) takes an index and returns 34 bytes — `ssid[33]` and an RSSI byte, a *negative dBm* you must read as signed or every network looks impossibly strong. Joining is `SET_SSID` (`$FB`), and it has two famous edges: it demands **at least one parameter whose value it ignores**, and the payload must be **exactly 97 bytes** — `ssid[33]` + `password[64]`, NUL-padded to their full widths.

The payload-shape rule

Behind that edge is one firmware fact worth internalising: the transaction layer **fails a short read**. A command that expects a struct wants *all* of it. `OPEN_DIRECTORY` and `SET_DEVICE_FULLPATH` always send a full **256-byte, NUL-padded** payload; `SET_SSID` sends exactly 97.

Hosts and directories

Eight host slots of 32 bytes each. `READ_HOST_SLOTS` (`$F4`) returns all 256 bytes in one reply, which CONFIG renders straight from the window. `WRITE_HOST_SLOTS` (`$F3`) is **all-or-nothing** — there is no write-one-slot form — so CONFIG re-reads the slots and streams them back out of the reply window with the edited one substituted, never mirroring 256 bytes in a machine that has under a kilobyte.

FUJICFG.C -- THE REPLY WINDOW IS THE BUFFER (RENAME A HOST SLOT)

```
/* WRITE_HOST_SLOTS takes all eight 32-byte slots, every time.
   Re-read, then stream them straight back with the edited one
   swapped in -- no 256-byte RAM mirror, because there is nowhere
   to put one. The editor runs no transactions, so nothing
   repaints the window between the read and the streamed write. */
fn_start(FN_DEV_FUJINET, CMD_READ_HOST_SLOTS);
...
fn_start(FN_DEV_FUJINET, CMD_WRITE_HOST_SLOTS);
for (i = 0; i < HOST_SLOTS; i++)
    if (i == cur) fn_tx_padded(fn_entry, HOST_STRIDE);
    else         fn_tx_from_reply(i * HOST_STRIDE, HOST_STRIDE);
```

Then `MOUNT_HOST` (`$F9`) takes the slot number; `OPEN_DIRECTORY` (`$F7`, host parameter, 256-byte path payload), `SET_DIRECTORY_POSITION` (`$E4`, a word), `READ_DIR_ENTRY` (`$F6`, maxlen and flags), and `CLOSE_DIRECTORY` (`$F5`). Entries arrive as names with a trailing `/` on subdirectories; end of directory is **exactly two `$7F` bytes**.

CAUTION

Do not read past the end of a directory. `fujidevice` returns two `$7F` bytes when the listing runs out, but a `fujinet-pc` SD host hands back `..` over and over instead — and once you have done that, every later `SET_DIRECTORY_POSITION` NAKs for the rest of the session. CONFIG stops on `$7F $7F`, `.` and `..` alike.

The utility drawer

The Fuji device also answers `GET_ADAPTERCONFIG_EXTENDED` (`$C4`, the first-contact command), `RANDOM_NUMBER` (`$D3`, four bytes of entropy), `GENERATE_GUID` (`$BB`), app keys (`$DC` – `$DB`, per-app key/value storage on the adapter’s SD card), base64 and hash engines (`$D0` – `$C2` — an SHA-256 on call from a Z80), and QR code

rendering (`$BC` – `$BF`). Command cards for all of them, with paired assembly and C examples, follow in the reference.

CHAPTER 10

Boot, Swap, and the Mappers

The finale of CONFIG’s road: pick a program from a network host and *become* it. On other FujiNets that is “mount the disk and reboot.” On a console whose cartridge port cannot write, it is a small magic trick in three acts.

Act one: the push

`SET_DEVICE_FULLPATH` (`$E2` : device slot 0, host, mode 1-read, path padded to 256) names the file. Then `MOUNT_IMAGE` (`$F8`) — and here the ColecoVision departs from every sibling. While your `MOUNT_IMAGE` transaction is *still outstanding*, the ESP32-S3 fetches the file and streams it to the RP2040 over a side channel: FujiBus frames addressed to device `$FF` (DBC) — an OPEN carrying a stream id and a 32-bit size, WRITE after WRITE of 512-byte chunks, then a CLOSE that commits. Stream 0 is the ROM; stream 1, pushed first when it exists, is the image’s `.cfg` sidcar.

Your program sees all this as theatre on the status bytes: `FN_BOOTSTAT` walks idle → transfer → ready (or `$80` , failed, with the reason in `FN_BOOTERR`), and `FN_BOOTPCT` climbs 0 to 100. Watch the counter, paint a progress bar, and touch no hotspot while the push is in flight.

Act two: the swap

When `FN_BOOTSTAT` reads ready: write `BOOTLOCK` (`$11`) = `$B5` to arm the trigger, copy a small stub into console RAM, and jump to it. The stub must run from RAM for the best reason imaginable — the swap replaces **every byte of the cartridge window, including the code that triggers it**.

FUJILIB.C -- THE SWAP STUB, HAND-ASSEMBLED

```
static const unsigned char stub_src[] = {
    0x3E,0x00, 0xD3,0xBF, 0x3E,0x81, 0xD3,0xBF, /* VDP reg1 = 0: */
                                           /* screen+INT off */
    0xDB,0xBF,                               /* clear latched INT */
    0x00,0x00,0x00,0x00, 0x00,0x00,0x00,0x00, /* let an NMI drain */
    0x3E,0x9F,0xD3,0xFF, 0x3E,0xBF,0xD3,0xFF, /* silence the PSG */
    0x3E,0xDF,0xD3,0xFF, 0x3E,0xFF,0xD3,0xFF,
    0x3A,0xFE,0xFD,                               /* THE SWAP: read $FDFF */
    0x21,0x00,0x80, 0x06,0x08,                   /* VDP regs 0..7 -> 0 */
    0x7D, 0xD3,0xBF, 0x7C, 0xD3,0xBF, 0x24, 0x10,0xF8,
    0xC3,0x00,0x00                                /* jp $0000: cold start */
};
```

The stub blanks the screen and clears VDP register 1 *before* the swap, because the BIOS vectors the vblank NMI to `$8021` — a byte that belongs to the *next* image the instant the swap happens. Then one read of `$FDFF` flips the cartridge, the VDP registers are reset to a power-on-like state so a `$55AA` game that skips the BIOS title does not inherit CONFIG’s video mode, the sound chip is silenced, and `JP $0000` cold-starts the BIOS, which reads the new image’s own `$8000` header exactly as it would a real cartridge.

Act three: after the swap

What the mailbox does next depends on what was pushed. An image carrying the `FUJI` claim at cart offset `$7CFC` — every client in this book — keeps the mailbox alive: the program can go on talking to the network, and CONFIG can be re-entered. An ordinary game makes no such promise, so booting one shuts the mailbox down for the session; RESET all you like, the game works exactly as a real cartridge would.

The mappers

A pushed image up to 128K is served through whichever real ColecoVision mapper its size selects: flat (up to 32K), MegaCart, Activision, X-in-1, or the Opcode Super Game Module. Every one of those puts its bank hotspots inside the `$FF00` page — the same page as the TX stream — which is not a collision but the same mutual exclusion: a

game image carries no claim, so the mailbox is already dead before any mapper decode goes live, and a claimed client image is flat 32K by construction. The size ceiling is 128K because a banked image must live in the RP2040's SRAM; there is no execute-in-place flash tier, since this port has only about 373 nanoseconds from address-valid to data-required.

CHAPTER 11

Command Reference: The Fuji Device

Every command device `$70` answers on this platform, verified against the firmware dispatch tables (`fujiDevice.cpp` , its five mixins, and the RS232 fall-through switch in `rs232Fuji.cpp`). Conventions: *nparam* is the value written to the NPARAM register; a *byte* parameter is size 1, a *word* is size 2, little-endian. Replies land in the reply window at `$F800` . Each card carries a compact example in Z80 assembly (using `fujimail.inc`) and C (using the portable **fujinet-lib**, dropping to the low-level library only for the few commands it does not wrap). Any command not listed here is answered with a NAK on this platform — the closing section names them.

11.1 WiFi and the adapter

GET_ADAPTERCONFIG_EXTENDED

\$C4

DEVICE	\$70	NPARAM	0
PARAMS	none		
PAYLOAD	none		
REPLY	240 bytes: ssid[33] hostname[64] binary localIP/gateway/ netmask/dnsIP[4] mac[6] bssid[6] fn_version[15] , then the text fields sLocalIP[16] (offset 140), sGateway , sNetmask , sDnsIP , sMacAddress[18] (204), sBssid — all NUL-terminated		

The first-contact command, and the one every status screen wants: no parameters, works before WiFi is up, and its text fields save you a print-an-IP routine. The compact form `GET_ADAPTERCONFIG` (`$E8`) returns the first 140 bytes only, addresses in binary.

Z80 ASSEMBLER

```
ld d,$70
ld e,$C4
call FNSTART
call FNCOMMIT
; ssid $F800+0
; fw $F800+125
; ip $F800+140
```

Z88DK C

```
AdapterConfigExtended ac;
if (fuji_get_adapter_config_extended(&ac)) {
    show(ac.ssid);
    show(ac.sLocalIP); /* dotted text */
    show(ac.fn_version);
}
```

SCAN_NETWORKS

\$FD

DEVICE	\$70	NPARAM	0
PARAMS	none		
PAYLOAD	none		
REPLY	1 byte: number of networks found		

Starts a fresh scan and blocks until it finishes. Follow with `GET_SCAN_RESULT` once per index. Reachable but slow; CONFIG shows a “scanning” line first.

GET_SCAN_RESULT**\$FC**

DEVICE	\$70	NPARAM	1
PARAMS	byte: result index, 0-based		
PAYLOAD	none		
REPLY	34 bytes: <code>ssid[33]</code> + <code>rsssi</code> (signed dBm)		

One scanned network. The RSSI byte at offset 33 is a negative dBm — closer to zero is stronger — so bar-graph its magnitude.

Z80 ASSEMBLER

```
ld d,$70
ld e,$FC
call FNSTART
ld a,(IDX)      ; index
call FNP8
call FNCOMMIT
; rssi at $F800+33, signed
```

Z88DK C

```
uint8_t count;
SSIDInfo net;
fuji_scan_for_networks(&count);
fuji_get_scan_result(idx, &net);
/* net.rssi is already signed dBm */
bar_graph(net.ssid, net.rssi);
```

SET_SSID**\$FB**

DEVICE	\$70	NPARAM	1
PARAMS	byte: any value — required but ignored		
PAYLOAD	exactly 97 bytes: <code>ssid[33]</code> + <code>password[64]</code> , NUL-padded		
REPLY	none		

Joins a network and saves it as the default. The two sharp edges are in bold: a short payload fails the transaction outright.

Z80 ASSEMBLER

```
ld d,$70
ld e,$FB
call FNSTART
xor a
call FNP8      ; dummy param
ld hl,SSID
ld de,33
call FNTXPAD
ld hl,PASS
ld de,64
call FNTXPAD
call FNCOMMIT
```

Z88DK C

```
NetConfig nc;
memset(&nc, 0, sizeof nc); /* NUL-pad */
strcpy(nc.ssid, ssid);
strcpy(nc.password, pass);
if (fuji_set_ssid(&nc))
    /* joined and saved as default */;
```

GET_SSID**\$FE**

DEVICE	\$70	NPARAM	0
PARAMS	none		
PAYLOAD	none		
REPLY	97 bytes: <code>ssid[33]</code> + <code>password[64]</code> , NUL-padded		

The current network configuration — password in clear text; your adapter is a trusted friend.

GET_WIFISTATUS**\$FA**

DEVICE	\$70	NPARAM	0
PARAMS	none		
PAYLOAD	none		
REPLY	1 byte: 3 = connected; other values not (yet)		

The poll target while a join is in flight. `GET_WIFI_ENABLED` (`$EA`) answers 1 byte: is the radio administratively on at all.

RESET / GET_WIFI_ENABLED**\$FF / \$EA**

DEVICE	\$70	NPARAM	0
PARAMS	none		
PAYLOAD	none		
REPLY	\$FF : ACK, adapter reboots · \$EA : 1 byte		

`RESET` reboots the ESP32-S3; the RP2040 and mailbox stay up while the link bit in `FN_STATUS` drops and returns. `GET_WIFI_ENABLED` reports whether the radio is enabled.

STATUS**\$53**

DEVICE	\$70	NPARAM	1
PARAMS	byte: request type — 1 mount times, else diagnostic		
PAYLOAD	none		
REPLY	type 1: a <code>time_t</code> per device slot (0 = unmounted); else 4 bytes		

Requires the parameter on this build; mostly a diagnostic.

DEVICE_READY**\$00**

DEVICE	\$70	NPARAM	0
PARAMS	none		
PAYLOAD	none		
REPLY	the bus self-test response		

The “FUJICMD DEVICE TEST” round trip — proof the mailbox, the RP2040, the USB link and the ESP32-S3 are all telling the truth.

11.2 Hosts, slots, and mounting

READ_HOST_SLOTS**\$F4**

DEVICE	\$70	NPARAM	0
PARAMS	none		
PAYLOAD	none		
REPLY	256 bytes: 8 slots × <code>hostname[32]</code>		

The host list, one transaction. CONFIG renders its HOSTS screen straight from the reply window — no copy in RAM.

Z80 ASSEMBLER

```
ld d,$70
ld e,$F4
call FNSTART
call FNCOMMIT
; slot i name at $F800 + i*32
```

Z88DK C

```
/* read in place -- 256 bytes is a
   quarter of RAM, so point at the
   reply window as the games do */
HostSlot *hosts = (HostSlot *) 0xF800;
fuji_get_host_slots(hosts, 8);
```

WRITE_HOST_SLOTS**\$F3**

DEVICE	\$70	NPARAM	0
PARAMS	none		
PAYLOAD	256 bytes: all 8 slots, full width		
REPLY	ACK		

All-or-nothing — there is no write-one-slot command. See the “reply window IS the buffer” idiom in the Fuji Device chapter: stream the payload straight back out of a fresh `READ_HOST_SLOTS`, substituting the edited slot with `fn_tx_from_reply`.

MOUNT_HOST / UNMOUNT_HOST**\$F9 / \$E6**

DEVICE	\$70	NPARAM	1
PARAMS	byte: host slot, 0-based		
PAYLOAD	none		
REPLY	ACK, or NAK if unreachable		

Connects (or releases) the named host — TNFS, SMB and the rest. Required before `OPEN_DIRECTORY` or a boot.

Z80 ASSEMBLER

```
ld d,$70
ld e,$F9
call FNSTART
ld a,(HOST)
call FNP8
call FNCOMMIT
```

Z88DK C

```
if (fuji_mount_host_slot(host))
    /* host mounted */;
/* fuji_unmount_host_slot(host)
   releases it */
```

READ_DEVICE_SLOTS / WRITE_DEVICE_SLOTS**\$F2 / \$F1**

DEVICE	\$70	NPARAM	0
PARAMS	none		
PAYLOAD	none		
REPLY	\$F2 : the device-slot table (<code>hostSlot</code> , <code>mode</code> , <code>filename[]</code> per slot); \$F1 takes it back as payload		

What is configured in each virtual drive. The ColecoVision boots images rather than serving drives, so CONFIG leaves these screenless — but they answer.

SET_DEVICE_FULLPATH**\$E2**

DEVICE	\$70	NPARAM	3
PARAMS	bytes: device slot, host slot, mode (1 = read)		
PAYLOAD	256 bytes: the path, NUL-padded		
REPLY	ACK		

Names the file for a device slot — the first half of booting. The argument order is (mode, host, device slot, path) ; the library pads the path to the 256-byte buffer the adapter expects.

Z80 ASSEMBLER

```
ld d,$70
ld e,$E2
call FNSTART
xor a
call FNP8      ; devslot 0
ld a,(HOST)
call FNP8
ld a,1
call FNP8      ; mode read
ld hl,PATH
ld de,256
call FNTXPAD
call FNCOMMIT
```

Z88DK C

```
fuji_set_device_filename(
    1 /*read*/, host,
    0 /*dev slot*/, path);
```

MOUNT_IMAGE / UNMOUNT_IMAGE**\$F8 / \$E9**

DEVICE	\$70	NPARAM	2 / 1
PARAMS	\$F8 : device slot, access mode (1 read) · \$E9 : device slot		
PAYLOAD	none		
REPLY	ACK when the mount — and, on this platform, the push — completes		

The second half of booting, and the platform's most theatrical command: while it is outstanding the adapter streams the image to the cartridge over the DBC side channel, and **FN_BOOTSTAT** / **FN_BOOTPCT** narrate the progress. Allow the 60-second budget and paint a progress bar. See the Boot chapter.

Z80 ASSEMBLER

```
ld d,$70
ld e,$F8
call FNSTART
xor a
call FNP8      ; devslot 0
ld a,1
call FNP8      ; mode read
call FNCOMMIT
; then watch FN_BOOTSTAT / FN_BOOTPCT
```

Z88DK C

```
fuji_mount_disk_image(0 /*slot*/, 1);
while (fuji_coleco_boot_state()
    != FUJI_COLECO_BOOT_READY) {
    if (fuji_coleco_boot_state()
        == FUJI_COLECO_BOOT_FAILED) break;
    show_pct(fuji_coleco_boot_percent());
}
```

MOUNT_ALL / SET_BOOT_MODE / CONFIG_BOOT**\$D7 / \$D6 / \$D9**

DEVICE	\$70	NPARAM	0 / 1 / 1
PARAMS	none		
PAYLOAD	none		
REPLY	ACK		

MOUNT_ALL mounts every configured slot; **SET_BOOT_MODE** and **CONFIG_BOOT** select what the adapter offers at power-on on disk-serving platforms. On this console the mailbox claim, not a config flag, decides who boots — **CONFIG** deliberately never sends **CONFIG_BOOT** .

NEW_DISK**\$E7**

DEVICE	\$70	NPARAM	0
PARAMS	none		
PAYLOAD	numSectors (word) sectorSize (word) hostSlot deviceSlot filename[256]		
REPLY	ACK on creation		

Creates a blank image on a host — full-struct payload, NUL-padded filename, the 256-pad rule again.

11.3 Directories**OPEN_DIRECTORY****\$F7**

DEVICE	\$70	NPARAM	1
PARAMS	byte: host slot		
PAYLOAD	256 bytes: path, NUL, then an optional *.COL -style filter, the rest NUL padding		
REPLY	ACK		

Opens a directory on a mounted host; the filter is applied server-side.

Z80 ASSEMBLER

```
ld d,$70
ld e,$F7
call FNSTART
ld a,(HOST)
call FNP8
ld hl,ROOTP      ; "/",0
ld de,256
call FNTXPAD
call FNCOMMIT
```

Z88DK C

```
fuji_open_directory(host, "/");
/* with a filter:
   fuji_open_directory_filter(
       host, "/", "*.COL"); */
```

READ_DIR_ENTRY**\$F6**

DEVICE	\$70	NPARAM	2
PARAMS	bytes: maxlen (crunch width), flags (0)		
PAYLOAD	none		
REPLY	one entry name, NUL-terminated; a trailing / marks a subdirectory; end of directory is two \$7F bytes		

One entry per transaction; the entry advances the cursor itself. `maxlen` truncates the name — ask for the display width when drawing, the full width when building a boot path. See the caution in the Fuji Device chapter about reading past the end.

Z80 ASSEMBLER

```
ld d,$70
ld e,$F6
call FNSTART
ld a,29      ; maxlen
call FNP8
xor a
call FNP8    ; flags
call FNCOMMIT
; $F800: $7F,$7F => end
```

Z88DK C

```
uint8_t entry[32];
fuji_read_directory(maxlen, 0, entry);
if (entry[0]==0x7F &&
    entry[1]==0x7F) /* end */ ;
```

SET_DIRECTORY_POSITION / GET_DIRECTORY_POSITION**\$E4 / \$E5**

DEVICE	\$70	NPARAM	1 / 0
PARAMS	\$E4 : word, absolute entry index		
PAYLOAD	none		
REPLY	\$E5 : word, the current index; \$E4 : ACK		

Seek and tell for the open directory — CONFIG's paging in one command. Note the two-byte parameter for the seek.

CLOSE_DIRECTORY**\$F5**

DEVICE	\$70	NPARAM	0
PARAMS	none		
PAYLOAD	none		
REPLY	ACK		

Closes the walk. Directories are a scarce resource on the adapter; close what you open.

11.4 Files, prefixes, and paths**COPY_FILE****\$D8**

DEVICE	\$70	NPARAM	2
PARAMS	bytes: source host, destination host — both 1-based		
PAYLOAD	exact length , no padding: <code>sourcepath destdir/destname</code>		
REPLY	ACK when the copy completes		

Server-to-server copy without a byte passing through the console. The payload is read as a string, so it goes at exact length — one padding NUL would land inside the destination name — and its host parameters are 1-based, alone among slot parameters.

SET_HOST_PREFIX / GET_HOST_PREFIX**\$E1 / \$E0**

DEVICE	\$70	NPARAM	1
PARAMS	byte: host slot; SET carries the prefix as payload		
PAYLOAD	none		
REPLY	GET: the prefix string		

A per-host working directory prepended to relative paths.

GET_DEVICE_FULLPATH**\$DA**

DEVICE	\$70	NPARAM	1
PARAMS	byte: device slot		
PAYLOAD	none		
REPLY	the stored path		

Reads back what `SET_DEVICE_FULLPATH` stored.

11.5 App keys

Small key/value files on the adapter's SD card, namespaced by a 16-bit creator id, an app id and a key id — how a FujiNet program remembers a player name between sessions. `OPEN_APPKEY` selects the key and the direction; then read or write; then close. The 5 Card Stud and Battleship clients use these for the Lobby username.

OPEN_APPKEY

\$DC

DEVICE	\$70	NPARAM	0
PARAMS	none		
PAYLOAD	6 bytes: creator (word) app key mode (0 read, 1 write) reserved		
REPLY	ACK — NAK if no SD card, creator =0, or a bad mode		

Selects which key the next read or write touches. The portable library has no standalone `OPEN_APPKEY`: `fuji_set_appkey_details(creator, app, size)` records the ids, and `fuji_read_appkey` / `fuji_write_appkey` issue the OPEN implicitly. The file is `/FujiNet/CCCCAAKK.key` on the SD card; the value cap is 64 bytes.

Z80 ASSEMBLER

```
ld d,$70
ld e,$DC
call FNSTART
ld hl,AKREC ; 6-byte struct
ld de,6
call FNTXPAD
call FNCOMMIT
```

Z88DK C

```
/* the portable library folds OPEN
into the read/write: set the ids... */
fuji_set_appkey_details(0xE41C, 1, DEFAULT);
uint8_t val[MAX_APPKEY_LEN + 2];
uint16_t len;
fuji_read_appkey(0 /*key*/, &len, val);
```

READ_APPKEY / WRITE_APPKEY / CLOSE_APPKEY

\$DD / \$DE / \$DB

DEVICE	\$70	NPARAM	0
PARAMS	none		
PAYLOAD	WRITE: the value, up to 64 bytes		
REPLY	READ: a u16 length then up to 64 data bytes; others ACK		

`READ_APPKEY`'s reply is length-prefixed on this bus. A successful WRITE resets the selection, so each write needs a fresh OPEN in write mode.

11.6 Utilities: entropy, encoding, hashing, QR

RANDOM_NUMBER / GENERATE_GUID

\$D3 / \$BB

DEVICE	\$70	NPARAM	0
PARAMS	none		
PAYLOAD	none		
REPLY	\$D3 : 4 random bytes · \$BB : a 37-byte UUID text		

Hardware entropy from the ESP32, and a fresh printable unique id. `GENERATE_GUID` has a wrapper; `RANDOM_NUMBER` does not.

Z80 ASSEMBLER

```
ld d,$70
ld e,$D3
call FNSTART
call FNCOMMIT
; 4 bytes at $F800
```

Z88DK C

```
char guid[MAX_GUID_LEN];
fuji_generate_guid(guid);
/* RANDOM_NUMBER ($D3) has no
fujinet-lib wrapper -- use the
low-level fuji_bus_call for it */
```

BASE64 / HASH: INPUT, COMPUTE, LENGTH, OUTPUT**\$D0-\$C9 / \$C8-\$C2**

DEVICE	\$70	NPARAM	varies
PARAMS	INPUT: word byte count; HASH COMPUTE: byte algorithm (0 MD5, 1 SHA1, 2 SHA256, 3 SHA512); LENGTH/OUTPUT: byte (1 = hex)		
PAYLOAD	INPUT: the bytes to feed		
REPLY	LENGTH: the size; OUTPUT: the result		

Four-step pipelines on the adapter: feed bytes in with INPUT (call it repeatedly for long data), COMPUTE, ask LENGTH, then drain with OUTPUT. An SHA-256 on call from a Z80. fujinet-lib wraps the whole hash pipeline in one `fuji_hash_data(...)` call (`hash_alg_t = MD5 / SHA1 / SHA256 / SHA512`); the base64 pair (`$D0 - $C9`) is driven with the four step macros.

Z80 ASSEMBLER

```
; SHA-256 of the INPUT buffer:
; $C8 INPUT (append), then:
ld d,$70
ld e,$C7      ; COMPUTE
call FNSTART
ld a,2        ; SHA256
call FNP8
call FNCOMMIT
```

Z88DK C

```
/* one-shot SHA-256, hex output */
uint8_t digest[64];
fuji_hash_data(SHA256, data, n,
               true /*hex*/, digest);
```

QR CODE: INPUT, ENCODE, LENGTH, OUTPUT**\$BC / \$BD / \$BE / \$BF**

DEVICE	\$70	NPARAM	varies
PARAMS	INPUT: word count · ENCODE: version (0=auto), ECC (0–3), shorten · LENGTH: output mode (0 binary, 2 bitmap) · OUTPUT: word count		
PAYLOAD	INPUT: the text to encode		
REPLY	LENGTH: 4-byte size; OUTPUT: the module data		

The adapter renders a QR matrix you can blit — feed the text, ENCODE, ask LENGTH in the mode you want, drain OUTPUT. A version-1 code is 21×21 modules. There is no `QRCODE_CLEAR` on this build.

11.7 Answered with a NAK on this platform

These `fujiCommandID.h` opcodes fall through every dispatch table in this build and earn a NAK; none needs an example. `ENABLE_UDPSTREAM` (`$F0`), `SET_BAUDRATE` (`$EB`), `SET_HSIO_INDEX` (`$E3`), `SET_SIO_EXTERNAL_CLOCK` (`$DF`), `ENABLE / DISABLE_DEVICE` (`$D5 / $D4`), `GET_TIME` (`$D2` — use device `$45`), `DEVICE_ENABLE_STATUS` (`$D1`), `GET_HEAP` (`$C1`), `GET_DEVICE1..10_FULLPATH` (`$A0 - $A9`), `UPDATE_FIRMWARE` (`$90`), and `HSIO_INDEX` (`$3F`). Most are other platforms' bus tuning.

CHAPTER 12

Command Reference: The Network Device

Every command devices `$71 – $78` answer, verified against the RS232 network dispatcher (`network.cpp`). Most opcodes are the ASCII letter of their name — 'O' pen, 'R' ead, 'W' rite — a habit from the Atari. The connection, the devicespec grammar, and per-protocol behaviour belong to *The FujiNet Network Protocol Handbook* these are the cards. Cap a read at 1,024 bytes (the reply window) and a write's payload at what fits the 320-byte TX stream.

12.1 The lifecycle five

NET_OPEN

\$4F 'O'

DEVICE	\$71	NPARAM	2
PARAMS	bytes: access mode (4 read, 8 write, 12 read/write), translation (0 none, 1 CR, 2 LF, 3 CR/LF)		
PAYLOAD	256 bytes: the devicespec, NUL-padded		
REPLY	ACK when the connection is established		

Mode 12 doubles as GET on an HTTP adapter, whose fetch is deferred to the first READ. Modes 13 POST, 14 PUT, 5 DELETE, 6 HEAD are the HTTP family. NETCAT opens a socket read/write with no translation.

Z80 ASSEMBLER

```
ld d,$71
ld e,$4F
call FNSTART
ld a,12      ; read/write
call FNP8
xor a
call FNP8    ; no translation
ld hl,SPEC
ld de,256
call FNTXPAD
call FNCOMMIT
```

Z88DK C

```
if (network_open(spec, OPEN_MODE_RW,
    OPEN_TRANS_NONE) == FN_ERR_OK)
    /* connected */;
/* HTTP GET: OPEN_MODE_HTTP_GET_H */
```

NET_STATUS

\$53 'S'

DEVICE	\$71	NPARAM	2
PARAMS	two zero bytes (the build reads the request type from parameter 1)		
PAYLOAD	none		
REPLY	4 bytes: <code>avail</code> (word), <code>connected</code> , <code>devstatus</code> (1 SUCCESS, 136 end-of-file)		

The heartbeat. A terminal gates on `conn` (connected); a fetch-and-render client gates on `ds` (device status — 1 SUCCESS, 136 end-of-file); everyone reads `avail`. `network_status` fills all three.

Z80 ASSEMBLER

```
ld d,$71
ld e,$53
call FNSTART
xor a
call FNP8
xor a
call FNP8
call FNP8
call FNCOMMIT
; avail=$F800/1, conn=2, ds=3
```

Z88DK C

```
uint16_t avail;
uint8_t conn, ds;
network_status(spec, &avail, &conn, &ds);
```

NET_READ**\$52 'R'**

DEVICE	\$71	NPARAM	1
PARAMS	word: byte count — clamp to <code>avail</code> and to 1,024		
PAYLOAD	none		
REPLY	exactly that many bytes, in the reply window		

The firmware sends exactly `want` bytes on ACK; a short/failed read comes back as a NAK. On deferred HTTP the first READ performs the fetch — give it the OPEN-class wait.

Z80 ASSEMBLER

```
ld d,$71
ld e,$52
call FNSTART
ld hl,(WANT) ; count
call FNP16
call FNCOMMIT
call FNRLN ; HL = length
```

Z88DK C

```
int16_t n = network_read_nb(spec,
                             buf, want);
if (n > 0) render(buf, n);
/* render BEFORE any other
   transaction repaints the window */
```

NET_WRITE**\$57 'W'**

DEVICE	\$71	NPARAM	1
PARAMS	word: byte count		
PAYLOAD	exactly that many bytes		
REPLY	ACK		

The count rides twice — as the parameter and as the payload the cartridge measures. `network_write` handles both. Keep it well inside the 320-byte TX stream; NETCAT caps a line at 120 bytes.

Z80 ASSEMBLER

```
ld d,$71
ld e,$57
call FNSTART
ld hl,(LEN)
call FNP16
ld hl,BUF
ld b,(LEN)
WLP: ld a,(hl)
call FNTXB
inc hl
djnz WLP
call FNCOMMIT
```

Z88DK C

```
network_write(spec, buf, len);
```

NET_CLOSE**\$43 'C'**

DEVICE	\$71	NPARAM	0
PARAMS	none		
PAYLOAD	none		
REPLY	ACK — and an empty reply window		

Hang up. Send it when you mean to end the connection; a game that reopens per request closes at the *start* of the next request, never between a READ and the rendering that follows it.

12.2 Structured channels, seek, and line discipline

NET_SET_CHANNEL_MODE / NET_PARSE / NET_QUERY

\$4D / \$50 / \$51

DEVICE	\$71	NPARAM	2 / 0 / 0
PARAMS	SET_CHANNEL_MODE: byte (ignored), then mode (0 raw, 1 JSON)		
PAYLOAD	QUERY: the path, e.g. /players/0/name		
REPLY	QUERY: the value as text, then READ it		

Switch an open HTTP-family connection to the on-adaptor JSON parser (`'M'` reads the mode from the *second* parameter), `PARSE` the body, then `QUERY` one element — the difference between parsing JSON on a Z80 and not. The games skip this and ask their servers for binary with `?bin=1`.

NET_SEEK / NET_TELL

\$25 '%' / \$26 '&'

DEVICE	\$71	NPARAM	1 / 0
PARAMS	SEEK: 4-byte offset		
PAYLOAD	none		
REPLY	TELL: 4-byte position		

Random access where the protocol supports it (TNFS, HTTP ranges).

NET_TRANSLATION / NET_SET_EOL / NET_SET_INT_RATE

\$54 / \$4C / \$5A

DEVICE	\$71	NPARAM	2
PARAMS	TRANSLATION: byte (ignored) then code; SET_EOL: the EOL pair, a first byte of 0 clears; SET_INT_RATE: byte (ignored) then rate		
PAYLOAD	none		
REPLY	ACK		

Line-ending translation and custom terminators after OPEN. `SET_INT_RATE` paces a bus interrupt on platforms that have one; the ColecoVision has no interrupt line back from the cartridge, so it is accepted and irrelevant — polling `FN_ACKSEQ` is the only completion mechanism here.

12.3 Filesystems, sockets, and credentials

NET_RENAME / DELETE / LOCK / UNLOCK / MKDIR / RMDIR / CHDIR / GETCWD

\$20 \$21 \$23 \$24 \$2A \$2B \$2C \$30

DEVICE	\$71	NPARAM	0
PARAMS	none		
PAYLOAD	the target path (RENAME: <code>old,new</code>)		
REPLY	ACK; GETCWD returns the path		

Filesystem verbs on protocols that have files — TNFS, FTP, SMB. `CHDIR` and `GETCWD` move and report the working directory.

NET_CONTROL / NET_CLOSE_CLIENT**\$41 'A' / \$63 'c'**

DEVICE	\$71	NPARAM	0
PARAMS	none		
PAYLOAD	none		
REPLY	ACK		

TCP server duty: a listening `N:TCP://:port/` accepts a waiting client with `'A'` and hangs up on that client with `'c'` while keeping the listener — your ColecoVision can be the BBS.

NET_GET_REMOTE / NET_SET_DESTINATION /
NET_USERNAME / NET_PASSWORD
\$72 / \$44 / \$FD / \$FE

DEVICE	\$71	NPARAM	0
PARAMS	none		
PAYLOAD	SET_DESTINATION: <code>host:port</code> ; USERNAME/PASSWORD: the credential		
REPLY	GET_REMOTE: the last datagram's sender		

UDP's who-said-that and talk-to-them-instead, plus stored credentials for the next OPEN on protocols that log in (FTP, SSH, SMB) — send them before OPEN.

12.4 Answered with a NAK on this platform

Enum'd but not dispatched in this build, so a NAK, no example: `NET_GET_DSTATS_VALUE` (`$FF`), `NET_CHANNEL_MODE` getter (`$FC` — the setter `$4D` works), `NET_SET_PARAMETERS` (`$FB`), `NET_SET_CHANNEL` (`$FA`), `NET_SET_HSIO_INDEX` (`$E3`), `NET_QUERY_ALT / NET_PARSE_ALT` (`$81 / $80`), `NET_GET_ERROR` (`$45` — read the STATUS device byte instead), and `NET_HSIO_INDEX` (`$3F`).

CHAPTER 13

Command Reference: The Other Devices

The mailbox is a transport, not a translator: whatever byte you put in the DEVICE register goes on the wire. Beyond the Fuji and network devices, the fujinet-pc build instantiates four more — disks, a printer, a clock, and a Hayes-style modem. **Nothing in the ColecoVision tree drives them yet** they are reachable capability, documented here honestly as such. The examples show the shape; the behaviour is transcribed from the RS232 handlers.

13.1 Disk — devices 31H–38H

Eight disk devices, `$31 + n`. On this build every sector is 512 bytes and the offset is `sector × 512`. Read and write name the sector in `param(0)`; a missing parameter throws on the adapter, so always supply it.

DISK_READ / DISK_WRITE / DISK_PUT

\$52 / \$57 / \$50

DEVICE	\$31	NPARAM	1
PARAMS	<code>param(0)</code> : sector number		
PAYLOAD	WRITE/PUT: 512 bytes		
REPLY	READ: 512 bytes; WRITE/PUT: ACK		

`$53 (DISK_STATUS)` is **not** a status call on this build — it falls through into the write path, so never send it to a disk expecting a status reply. With no image mounted, READ returns NAK.

Z80 ASSEMBLER

```
ld d,$31      ; disk 0
ld e,$52      ; READ
call FNSTART
ld hl,(SEC)
call FNP16
call FNCOMMIT
; 512 bytes at $F800
```

Z88DK C

```
/* no fujinet-lib wrapper for disks:
   drop to the low-level mailbox */
fn_start(0x31, 0x52); /* READ */
fn_param16(sector);
fn_commit();          /* 512B @ $F800 */
```

DISK_FORMAT / PERCOM_READ / PERCOM_WRITE

\$21 / \$4E / \$4F

DEVICE	\$31	NPARAM	0
PARAMS	none		
PAYLOAD	none		
REPLY	FORMAT: a bad-sector map; PERCOM_READ: a 12-byte geometry block		

Vestigial on this build: FORMAT writes nothing to the image, and the PERCOM block is a 12-byte scratch area with no effect on geometry. Documented for completeness.

13.2 Printer — device 40H

PRINTER_WRITE / PRINTER_PUT / PRINTER_STATUS

\$57 / \$50 / \$53

DEVICE	\$40	NPARAM	0 / 0 / 1
PARAMS	STATUS: byte request type, required but ignored		
PAYLOAD	WRITE/PUT: the bytes to print (keep under 255)		
REPLY	WRITE/PUT: ACK; STATUS: 4 bytes		

Output goes to a `/paper` file on the adapter, retrieved through its web UI — not to a PDF. The default emulation is trimmed plain text. The printer can be disabled in config, in which case it does not reply at all and the host must time out.

Z80 ASSEMBLER

```

        ld d,$40
        ld e,$57      ; WRITE
        call FNSTART
        ld hl,LINE
        ld b,(LLEN)
PLP:    ld a,(hl)
        call FNTXB     ; one byte
        inc hl
        djnz PLP
        call FNCOMMIT

```

Z88DK C

```

/* no fujinet-lib wrapper for the
   printer: low-level mailbox */
fn_start(0x40, 0x57);
fn_tx_bytes(line, len); /* <255 */
fn_commit();

```

13.3 Clock — device 45H (APETime)

Present only when APETime is enabled in the adapter's config (it is, by default). Reads come in many formats; the parameter, when 1, selects an alternate time zone set by `SETTZ`.

APETIME_GETTIME / GET_ISO_LOCAL / SETTZ

\$93 / \$49 / \$99

DEVICE	\$45	NPARAM	0 / 0 / 0
PARAMS	byte 1 = alternate TZ (on the read commands)		
PAYLOAD	SETTZ: a POSIX TZ string		
REPLY	GETTIME: 6 bytes <code>DD MM YY HH MM SS</code> ; GET_ISO_LOCAL: a 25-byte ISO 8601 string		

Other formats answer too: PRODOS (`$50` , 4 bytes), SOS (`$53` , ASCII), simple (`$54` , 7 bytes), simple+hundredths (`$4D` , 8 bytes), ISO UTC (`$5A`), the raw TZ string (`$47`) and its length (`$4C`). `SETTZ_ALT` (`$74`) stores the system TZ persistently.

Z80 ASSEMBLER

```

        ld d,$45
        ld e,$93      ; GETTIME
        call FNSTART
        call FNCOMMIT
; DD MM YY HH MM SS at $F800

```

Z88DK C

```

uint8_t t[8];
clock_get_time(t, APETIME_BINARY);
/* t = DD MM YY HH MM SS */
/* clock_set_tz("EST5EDT"); then
   clock_get_time(t, TZ_ISO_STRING); */

```

13.4 Modem — device 50H

A Hayes AT-command virtual modem that dials TCP instead of phone lines: `ATDT host:port` opens a socket, `ATNET1` runs the session through telnet negotiation. Ten packet commands configure and poll it; the character stream itself is unframed. Reachable but unexercised on the ColecoVision.

MODEM_STATUS / MODEM_WRITE / MODEM_STREAM**\$53 / \$57 / \$58**

DEVICE	\$50	NPARAM	1 / 0 / 0
PARAMS	STATUS: byte (ignored, required)		
PAYLOAD	WRITE: an AT command line, or data when connected		
REPLY	STATUS: 2 bytes (byte 1 carries the handshake bits: \$CD connected with data, \$CC connected idle, \$00 idle); WRITE: ACK; STREAM: a 9-byte baud table		

The other seven — CONTROL \$41 , CONFIGURE \$42 , SET_DUMP \$44 , LISTEN \$4C , UNLISTEN \$4D , BAUDRATELOCK \$4E ,
 AUTOANSWER \$4F — configure the virtual modem. Disabled in config, it does not reply.

CHAPTER 14

Command Reference: The Cartridge Itself

Below the FujiBus devices sits the cartridge's own protocol — the hotspot decode and the register file the earlier chapters used. This is the whole of it, in one place, plus the one message the cartridge *receives* rather than sends.

Layer 1: the hotspot pages

Every console-to-cartridge byte is a read whose address is the message.

Console read	Effect
<code>\$FD00 + r</code> , <code>r < \$80</code>	arm register <code>r</code>
<code>\$FD00</code>	swap to the staged boot image (BOOTLOCK-armed only)
<code>\$FE00 + v</code>	armed register $\leftarrow v$, then disarm
<code>\$FF00 + v</code>	append <code>v</code> to the TX stream (max 320)
anything below <code>\$FD00</code>	an ordinary ROM read; no side effect

Figure 8. The hotspot decode: what each read does.

Layer 2: the register file

Written through a REGSEL/REGDATA read pair. The complete set:

Reg	Name	Use
<code>\$00</code>	DEVICE	FujiBus device id
<code>\$01</code>	COMMAND	FujiBus command id
<code>\$02</code>	NPARAM	parameters leading the TX stream (max 8)
<code>\$05</code>	DATA_RST	any value rewinds the TX write pointer
<code>\$06</code>	RXSLICE	reply slice — vestigial, one slice only
<code>\$10</code>	SEQ	nonzero, $\neq$ ACKSEQ: launch the transaction
<code>\$11</code>	BOOTLOCK	<code>\$B5</code> arms the ROM swap
<code>\$12 / \$13</code>	BOOTSEL	<code>\$B5</code> then <code>\$4A</code> : reboot cart to update mode

Figure 9. The mailbox registers.

Layer 3: the DBC push (device FFH)

One device the cartridge *receives* on, unsolicited, while a `MOUNT_IMAGE` is still outstanding: the ESP32 pushes the image to the RP2040 as FujiBus frames addressed to device `$FF` (DBC) — `NET_OPEN` with a stream id and 32-bit size, `NET_WRITE` of 512-byte chunks, `NET_CLOSE` to commit. Stream 0 is the ROM, stream 1 its `.cfg` sidecar. Your program never sends these; it watches them arrive on `FN_BOOTSTAT` and `FN_BOOTPCT`. Boot errors land in `FN_BOOTERR`: 1 too big, 2 truncated, 3 no mapping fits, 4 store busy.

CHAPTER 15

Netcat

“Dial, you tinhorn!” NETCAT is the family’s terminal: give it any `N:` devicespec and it pumps bytes between the connection and a scrolling 32×20 pane, forever. Call a telnet BBS, watch a TCP echo server, talk to anything that talks back. It is also the network device’s teaching program — the first client in this book to hold a single connection open for its whole life, the first to send `NET_WRITE`, and the first with a pane that scrolls. Its full source is in Appendix C.

How to get NETCAT on your screen

Build with `./build.sh` (the default devicespec, a telnet BBS, bakes in) and run with `pico/coleco/run.sh netcat` under MAME, or burn `build/netcat.bin` to a cartridge. The dial screen offers the devicespec for editing on the on-screen keyboard; `#` dials.

Hand controls

Control Stick steer the on-screen keyboard cursor
Fire / `#` in the editor: type / accept · in session: compose a line
Keypad `*` in the editor: backspace · in session: hang up

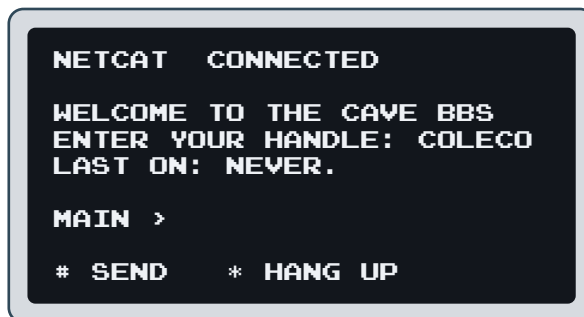


Figure 10. NETCAT in session against a telnet BBS, captured from MAME.

Inside the program

The session loop is the three rules of the network device in the flesh: **status**, **read**, **draw**, in that order, with nothing between the read and the draw that could repaint the reply window. A terminal watches `connected` rather than a device-status byte, because on a live socket end-of-file is a hangup, not an error.

NETCAT.C -- THE WHOLE TERMINAL, ONE LOOP

```
for (;;) {
    if (!net_status(&avail, &conn, &ds)) return; /* link lost */
    if (!conn && avail == 0) return; /* disconnected */

    if (avail != 0) {
        got = net_read(avail > NET_READ_MAX ? NET_READ_MAX : avail);
        term_puts(FN_REPLY, got); /* draw straight out of the window,
                                   before any other transaction */
    }
    ev = in_read();
    if (ev == IN_KEYHASH || ev == IN_FIRE) compose_and_send();
    else if (ev == IN_KEYSTAR) { net_close(); return; }
}
```

Scrolling is honest work: the TMS9918 in Graphics 1 has no scroll register, so `netterm.c` copies every row up one through OS7's `read_vram / write_vram` and clears the last. And because there is no RAM to shadow the pane in, composing a line is the one clever move: the on-screen keyboard borrows the whole screen, so NETCAT saves the pane to *spare VRAM* first (`term_save`) and restores it after (`term_restore`) — the conversation survives the detour without ever occupying console RAM. The five `N:` commands live in `netlib.c` ; the shared mailbox, display, input and editor come from `../common` . Full listings: Appendix C.

CHAPTER 16

5 Card Stud

Ante up — the table is world-wide. 5 CARD STUD joins live multiplayer poker at `5card.carr-designs.com`, against players on Ataris, Apples, ADAMs, Intellivisions and the rest of the FujiNet family. The **server owns the rules**: every legal move arrives over the wire with its label, so the client carries no poker logic at all — and inherits rule changes without a re-burn.

How to play

Build with `./make-exp coleco` and run. Enter a name on the on-screen keyboard, pick a table, and the keypad presses whichever move the server dealt you. The stick and both fire buttons drive the table; the keypad's `1–5` map to refresh, help, colour, name and quit.

Inside the program

5 CARD STUD is the C client this whole platform was brought up to prove, and every one of its ColecoVision-specific decisions is forced by the same fact: one kilobyte of RAM, and a game state that is 418 bytes of it. So the state is **never copied into RAM**. It is read in place out of the reply window — which is cartridge ROM — through a single audacious macro.

SRC/MISC.H AND SRC/COLECO/VARS.H -- THE STATE IS THE WINDOW

```
#define COLECO_REPLY_WINDOW 0xF800
/* The 418-byte game state lives in the cartridge's reply window,
   not in RAM. It is READ-ONLY -- a store here goes nowhere -- and
   valid only until the next fuji_* call of any kind. */
#define clientState (*(ClientState *) COLECO_REPLY_WINDOW)
#define state clientState.game
```

Two rules follow from that macro, and the network layer is built around them:

SRC/COLECO/NETWORK.C -- ONE TRANSACTION, AND NOTHING AFTER IT

```
uint8_t getResponse(char *url, unsigned char *buffer, uint16_t max_len)
{
    if (channelOpen) { network_close(url); channelOpen = 0; }
    if (network_open(url, OPEN_MODE_HTTP_GET_H, OPEN_TRANS_NONE)
        != FN_ERR_OK) return 0;
    channelOpen = 1;
    /* ONE transaction: network_read_nb waits for the whole body to be
       available, then issues exactly one read. network_read()'s loop
       would copy its second chunk to buffer+count -- cartridge ROM --
       and lose the tail silently. And nothing runs after: the close is
       deferred to the top of the NEXT call, because every transaction
       repaints the window we are about to render from. */
    return network_read_nb(url, buffer, max_len) > 0;
}
```

The rest of the port is a study in living within the budget. The vblank NMI handler (`myInt`) does nothing but bump two counters — it must never touch `$F800` and up, so it is kept deliberately tiny. `conbuf.asm` cuts z88dk's 256-byte console scroll buffer to sixteen, safe only because the game disables vertical scrolling. The seat-position tables are `const`, so they live in ROM rather than spending 96 bytes of RAM. And `coleco-romstamp.py` enforces the cartridge contract at build time: exactly 32K, the `55 AA` header, and the `FUJI` claim at `$FCFC` — without which the cartridge shuts the mailbox down for the session.

When you quit, the game hands the console back to the FujiNet Lobby: it finds `ec.tnfs.io` among the host slots, mounts `colemco/lobby.rom`, watches `FN_BOOTPCT` climb, and takes the swap. Full listings: Appendix C.

CHAPTER 17

Battleship

You sank my — well, you know. BATTLESHIP is up to four players, live over the network at `battleship.carr-designs.com`, on boards small enough that all of them fit the screen at once: four 10×10 quadrants. You are always quadrant 0 — the server rotates its player list so every client sees itself first — and the game meets the Intellivision port at the same tables.

How to play

Name, table, then place your five ships (the placement screen rejects overlaps and offers a random legal seed), press ready, and when your turn lights up steer the target cursor and fire. The keypad's `1`–`5` map to refresh, help, sink-a-ship, name and quit.

Inside the program

BATTLESHIP shares 5 CARD STUD's shape — state in the reply window, one transaction per read — but reaches the mailbox through its own hook, enabled with `-DCUSTOM_FUJINET_CALLS`, so the shared game core calls a single function the ColecoVision port defines:

SRC/COLECO/FUJINET.C -- THE CUSTOM NETWORK HOOK

```
int16_t custom_network_call(char *url, uint8_t *buffer, uint16_t max_len)
{
    if (channelOpen) { network_close(url); channelOpen = 0; }
    if (network_open(url, OPEN_MODE_HTTP_GET, OPEN_TRANS_NONE) != FN_ERR_OK)
        return -1;
    channelOpen = 1;
    return network_read_nb(url, buffer, max_len);    /* one transaction */
}
```

Two economies are worth the look. The graphics run GRAPHICS II as a **cellmap**: z88dk lays down an identity name table once and never touches it, so every 32×24 cell owns eight pattern bytes and eight colour bytes, and drawing a cell is two VRAM writes — no glyph budget, no RAM shadow. And the 100-cell “was this square empty” shadow, needed only to trigger the shot animation, is packed to **one bit per cell** — thirteen bytes per player instead of a hundred. The poll loop backs off when the server errors (waiting longer after each failure) and, on the sound side, routes explosion sub-bass to the noise channel because it falls below the tone chip's floor. Server and wire format are documented with the game; full listings: Appendix C.

CHAPTER 18

Config

CONFIG is the program your FujiNet cartridge wakes up in — the adapter’s face on the ColecoVision, and the only client in this book that speaks to device `$70`. It lists host slots, renames them on an on-screen keyboard, browses directories, scans WiFi and joins a network, and boots what you pick, progress bar and all.

It is also the one program here that ships *inside* the cartridge: `build-cart.sh` packs the built client into `fujiconfigrom.h`, a C array compiled into the RP2040 firmware, which serves it as the power-on image — and after CONFIG network-boots something else, the swap replaces it entirely.

A note on where CONFIG lives

The CONFIG in this book is `fujicfg.c`, from the firmware’s own `testrom/`. It is the working browse-and-boot client the cartridge serves today, and its own header says it is the stand-in for the full `fujinet-config` port that will one day live in that project’s `colecto/` directory. It is documented here because it is the real thing — the code on the cartridge — not a sketch.

Around the screens

Three pages: HOSTS (pick one to open, `9` to rename, `#` for the config screen), FILES (browse and boot), and CONFIG (adapter info, a WiFi scan, and joining). The selection bar is not a colour change — Graphics 1 has no per-cell colour — but a second copy of the character set at pattern `$80`, so highlighting a row means re-pointing its name-table bytes at the inverse glyphs.

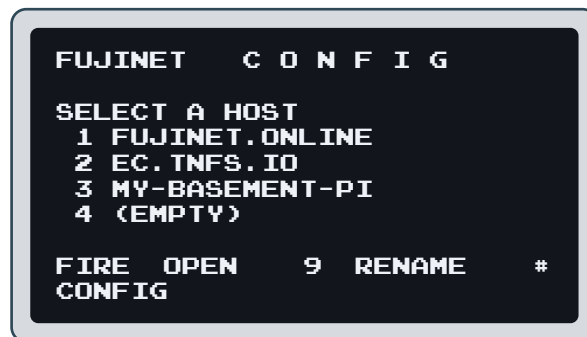


Figure 11. CONFIG’s host page.

Two contracts

CONFIG leans on two rules this book has already met. The on-screen keyboard (`fujiedit.c`) runs **no mailbox transactions** — a hard contract, so that a caller can hold data in the reply window across an edit; that is what lets `rename_host` re-read eight host slots, edit one on screen, and stream all eight back out of the window. And the file browser never keeps a filename in RAM: it draws each entry straight from the window and, to boot one, streams it straight back into the next transaction’s payload with `fn_tx_path`.

FUJICFG.C -- READ A DIRECTORY ENTRY, AND KNOW WHEN TO STOP

```
static bool read_entry(unsigned char maxlen)
{
    volatile unsigned char *r;
    fn_start(FN_DEV_FUJINET, CMD_READ_DIR_ENTRY);
    fn_param8(maxlen); /* the crunch width */
    fn_param8(0);
    if (fn_commit() != FN_OK || !fn_acked()) return false;

    r = FN_REPLY; /* through a local pointer, never FN_REPLY[i] */
    if (r[0] == 0x7F && r[1] == 0x7F) return false; /* end of dir */
    /* an SD host hands back '.' and '..' forever instead of $7F$7F,
       and reading past the end poisons SET_DIRECTORY_POSITION for the
       session -- so stop on those too, and never read past it. */
    if (r[0] == '.' && (r[1] == 0 || (r[1] == '.' && r[2] == 0)))
        return false;
    return true;
}
```

Deliberately not here, and left to the full CONFIG port: descending into subdirectories, disk-slot management, app keys. This is the browse-and-boot core, proven against a live adapter. Its helper modules — the OS7 display layer `fujidisp.c`, the POLLER-based input `fujiiin.c`, the on-screen keyboard `fujiedit.c`, the mandatory-first sound silencer `fujisnd.c`, and the boot splash — are the same ones NETCAT and the milestone clients share, and all are printed in Appendix C.

CHAPTER 19

Always More To Come

Your FujiNet cartridge does not only ship four programs — it opens the ColecoVision to virtually hundreds of program possibilities. Here is where to go next.

The Network Protocol Handbook

The companion volume from this series documents every URL scheme the `N:` device speaks — `TCP:`, `TELNET:`, `HTTP:` / `HTTPS:`, `TNFS:`, `JSON:`, `SSH:`, `SMB:`, `FTP:`, `UDP:`, `GMAIL:`, `GCAL:` and more — with the complete devicespec grammar, every mode and error code, and worked examples. Everything in it runs on the ColecoVision through the five commands of the Network Device chapter.

FujiNet Go Coleco Desktop

If you want to develop without touching hardware or hand-assembling a MAME tree, the next chapter covers *FujiNet Go Coleco Desktop*: a self-contained ColecoVision with a FujiNet built in, a Z80 and VDP debugger, and drag-and-drop mounting — the friendliest bench there is.

The sources of truth

<code>fuji_mailbox.h</code>		the mailbox spec — firmware	<code>pico/coleco/</code>
<code>fujilib.c</code> / <code>fujimail.inc</code>		the C and Z80 transports (Appendix C)	
<code>fujiCommandID.h</code>		every command opcode, canonically	
<code>fujitest.c</code> , <code>acfg.asm</code>		the minimal round trip, both languages	

The programs

Each program's repository carries its full build system, its MAME smoke tests, and a README with its own war stories: `netcat` (this book's, in the manuals repo), `fujinet-5cardstud`, `fujinet-battleship`, and the firmware's `pico/coleco/testrom` for CONFIG — all under `github.com/FujiNetWIFI` and friends. The game servers are `carr-designs.com` productions.

So enjoy your FujiNet: you now have a whole new dimension of home computing at your fingertips.

CHAPTER 20

FujiNet Go Coleco Desktop

The whole stack in this book — a ColecoVision, a FujiNet, and the network between them — fits in one desktop application. **FujiNet Go Coleco Desktop** is a self-contained ColecoVision with a FujiNet built in: browse a network host from the console, boot images over the network, and let the booted program keep talking to the network, all in one window. It is the newest member of the FujiNet Go desktop family, and for a FujiNet programmer it is the friendliest bench there is.

For users

Download a build for your platform from the releases page:

```
github.com/FujiNetWIFI/fujinet-go-coleco-desktop/releases
```

There are Linux packages for both GNOME and KDE (`.deb` , `.rpm` , `.tar.gz`), Windows (a `.zip` and a per-user NSIS installer that needs no administrator), macOS (an `.app` bundle, Apple Silicon and Intel), and two single-file Flatpaks. Once it is running: drag a cartridge (`.rom` , `.col` , `.bin`) onto the window to load it, or a disk or tape image to hand it to the built-in FujiNet. Every real ColecoVision mapper works — MegaCart, Activision, X-in-1, and the Opcode Super Game Module, which is fitted by default. Both hand controllers are drawn on screen and clickable, and every control is remappable.

IMPORTANT

You must supply your own `OS7.rom`. The ColecoVision OS-7 BIOS is copyrighted Coleco firmware and is **not** redistributed with the app. Put an **exactly 8192-byte** `OS7.rom` in the ROM directory (`~/local/share/fujinet-go-coleco/roms/` on Linux, `%LOCALAPPDATA%\fujinet-go-coleco` on Windows) or use the **Import BIOS...** menu item. The app rejects a file of any other size and, without a BIOS, boots to the FujiNet CONFIG client. See the project's `COMPLIANCE.md`.

The menus

Machine	open / eject a cartridge, reset, reset to CONFIG, import BIOS, preferences
View	the controllers panel, the debugger, TV aspect, smooth scaling
FujiNet	the adapter's configuration and a live console log

For developers

The app is the fastest way to see a client run without a MAME graft. It builds with CMake and provides its two dependencies — the emulator core and the FujiNet firmware — automatically:

BUILDING THE DESKTOP APP

```
cmake -B build
cmake --build build
ctest --test-dir build --output-on-failure
# against working checkouts instead of the pinned commits:
cmake -B build -DADAMCORE_SRC=~/.Workspace/adamcore \
    -DFUJINET_SRC=~/.Workspace/fujinet-firmware
```

The emulator is **adamcore**, a clean-room GPLv3 ColecoVision core (its Z80 validated against Tom Harte's tests and ZEXALL, with a TMS9928A VDP, an SN76489, and a working Super Game Module). The FujiNet is compiled in-process as `libfujinet` from the firmware's `colecovision-bringup` branch — the same mailbox sources this whole book describes, exercised over a loopback socket inside the one process. FujiNet's bus-over-IP listener is on port 11502 and its web admin UI on 11503; unusually for the family, FujiNet listens and the emulator's cartridge dials in, so the session starts FujiNet first.

Every frontend — GNOME (GTK4), KDE (Qt6), macOS (AppKit), Windows (Win32) — carries the same **Z80 and VDP debugger** over one shared engine: CPU, VDP and trace tabs, breakpoints by symbol, a trace ring, a pokeable live VRAM dump, and a full VDP decode (name table, pattern banks, sprites and their attribute table, the palette). When a client misbehaves, this is where you watch the mailbox transactions happen a frame at a time.

APPENDIX A

Mailbox Quick Reference

Status bytes (painted ROM)

Addr	Meaning
\$F800	reply window, 1024 bytes
\$FC00	ACKSEQ — echoes SEQ when ready
\$FC01	STATUS — bit0 link up
\$FC02	ERR — transport verdict
\$FC03	REPLYCMD — \$06 ACK / \$15 NAK
\$FC04	RXLEN low
\$FC05	RXLEN high
\$FC06	BOOTSTAT
\$FC07	BOOTPCT 0–100
\$FC08	BOOTERR
\$FC09	F (presence)
\$FC0A	N
\$FC0B	PROTOVER (1)

Hotspots (reading is writing)

Read	Effect
\$FD00+r	arm register r (r < \$80)
\$FDfE	swap to staged image (armed only)
\$FE00+v	armed register ← v
\$FF00+v	append v to TX (max 320)

Registers

Reg	Use
\$00	DEVICE
\$01	COMMAND
\$02	NPARAM
\$05	DATA_RST (rewind TX)
\$10	SEQ (ACKSEQ+1, wrap 255->1)
\$11	BOOTLOCK (\$B5)

The five moves

1. BEGIN: rewind TX; DEVICE, COMMAND, NPARAM.
2. STREAM: params (size 1/2/4 + LE value) then payload.
3. COMMIT: SEQ = ACKSEQ+1 (wrap 255->1).
4. WAIT: poll ACKSEQ; then ERR, REPLYCMD, RXLEN.
5. READ: the reply is at \$F800 , in place.

Layout contract (cart offsets)

Code below \$7800 · claim FUJI at \$7CFC · image exactly 32768 bytes · reply max 1024 · TX max 320 · NMI must never read \$F800 +.

APPENDIX B

Error and Status Codes

FN_ERR — transport verdict (FC02H)

Value	Meaning
0	OK — carried perfectly
1	NOLINK — ESP32-S3 not enumerated
2	TIMEOUT — adapter did not answer
3	BADFRAME — SLIP/checksum failure
4	TOOBIG — reply exceeded 1024 bytes
\$FF	EWAIT (client) — the cart never answered

FN_REPLYCMD (FC03H)

Value	Meaning
\$06	ACK — accepted, reply painted
\$15	NAK — refused: bad params, missing target, unsupported opcode

FN_BOOTSTAT (FC06H)

Value	Meaning
0	IDLE — no push in progress
1	XFER — streaming; watch BOOTPCT
2	READY — staged; arm and swap
\$80	FAILED — see BOOTERR

FN_BOOTERR (FC08H)

Value	Meaning
1	too big for the bank store
2	truncated push
3	no mapping fits the size
4	the only fitting store is busy

NET_STATUS device byte

Value	Meaning
1	SUCCESS
136	END_OF_FILE
other	per-protocol codes (see the Network Protocol Handbook)

GET_WIFISTATUS reply

Value	Meaning
3	connected
other	not connected — keep polling

APPENDIX C

Program Listings

The programs, complete, from the `listings/` snapshots — byte-identical to the source repositories at the commits in the README's sources-of-truth table. The shared client library (C.1) is used by NETCAT, CONFIG and the milestone clients; the games (C.6, C.7) reach the mailbox through the portable fujinet-lib instead, so only their ColecoVision-specific ports are printed here — the cross-platform game core is identical across eight machines and lives in each game's repository. Generated art (`charset.c` , `udg.h` , `font.bin`) is not printed.

C.1 The shared client library

FUJILIB.H -- THE MAILBOX CLIENT, IN C

```
1 /* fujilib.h -- the console's half of the FujiNet
2 * mailbox, in C.
3 * Mirrors firmware/include/fuji_mailbox.h by hand; keep
4 * the two in step. Every
5 * address here is a CONSOLE address (image offset +
6 * $8000).
7 * Both directions of the mailbox are cartridge READS,
8 * because the ColecoVision
9 * cartridge port has no write strobe -- see
10 * fuji_mailbox.h for why. So:
11 *
12 * console -> cart    the ADDRESS carries the payload.
13 * Reading $FD05 arms  register 5; reading $FE2A then
14 * sets it to 0x2A. Reading $FF41 appends 'A' to the outgoing
15 * stream.
16 * cart -> console    the cartridge repaints its own
17 * ROM, so the reply is just bytes at $F800 by the time we
18 * look.
19 *
20 * The reply window is the whole 1K in one piece rather
21 * than a paged 256-byte
22 * slice as on the Astrocade and the Arcadia. That is the
23 * one place this port
24 * spends its bigger window on the console's behalf: the
25 * ColecoVision has about
26 * 700 usable bytes of RAM after OS7's tables and the
27 * stack, so a directory
28 * entry that can be read in place, out of cartridge ROM,
29 * is worth more than
30 * the 1K of client space it costs.
31 */
32 #ifndef FUJILIB_H
33 #define FUJILIB_H
34
35 #include <stdbool.h>
36
37 /* ---- cart -> console: repainted ROM ---- */
38 #define FN_REPLY ((volatile unsigned char *)0xF800) /
39 * 1024 bytes */
40 #define FN_REPLY_MAX 1024
41
42 #define FN_ACKSEQ (*(volatile unsigned char *)0xFC00)
43 #define FN_STATUS (*(volatile unsigned char *)0xFC01)
44 #define FN_ERRCODE (*(volatile unsigned char *)0xFC02)
45 #define FN_REPLYCMD (*(volatile unsigned char *)0xFC03)
46 #define FN_RXLEN_LO (*(volatile unsigned char *)0xFC04)
47 #define FN_RXLEN_HI (*(volatile unsigned char *)0xFC05)
48 #define FN_BOOTSTAT (*(volatile unsigned char *)0xFC06)
49 #define FN_BOOTPCT (*(volatile unsigned char *)0xFC07)
50 #define FN_BOOTERR (*(volatile unsigned char *)0xFC08)
51 #define FN_MAGIC0 (*(volatile unsigned char *)0xFC09)
52 #define FN_MAGIC1 (*(volatile unsigned char *)0xFC0A)
53 #define FN_PROTOCOL (*(volatile unsigned char *)0xFC0B)
54
55 /* ---- console -> cart: hotspot reads ---- */
56 #define FN_REGSEL ((volatile unsigned char *)0xFD00)
57 #define FN_REGDAT ((volatile unsigned char *)0xFE00)
58 #define FN_TXPAGE ((volatile unsigned char *)0xFF00)
59 #define FN_SWAP (*(volatile unsigned char *)0xFFFE)
60
61 /* Register file */
62 #define FNR_DEVICE 0x00
63
64 #define FNR_CMD 0x01
65 #define FNR_NPARAM 0x02
66 #define FNR_DATA_RST 0x05
67 #define FNR_RXSLICE 0x06
68 #define FNR_SEQ 0x10
69 #define FNR_BOOTLOCK 0x11
70
71 #define FN_BOOTLOCK_MAGIC 0xB5
72
73 /* FujiBus */
74 #define FN_DEV_FUJINET 0x70
75 #define FN_ACK 0x06
76 #define FN_NAK 0x15
77
78 /* fn_commit() results: FN_ERR_* from fuji_mailbox.h,
79 plus our own timeout. */
80 #define FN_OK 0
81 #define FN_ENOLINK 1
82 #define FN_ETIMEOUT 2
83 #define FN_EBADFRAME 3
84 #define FN_ETOOBIG 4
85 #define FN_EWAIT 0xFF /* the cart never answered at
86 all */
87
88 /* Boot states */
89 #define FNB_IDLE 0
90 #define FNB_XFER 1
91 #define FNB_READY 2
92 #define FNB_FAILED 0x80
93
94 /* Is a FujiNet cartridge actually underneath us? On a
95 plain ROM cart these
96 two addresses are whatever the game put there (or
97 0xFF). */
98 bool fn_present(void);
99
100 /* Begin a transaction: rewind the outgoing stream and
101 address a device. */
102 void fn_start(unsigned char device, unsigned char
103 command);
104
105 /* Parameters, in order, before any payload. */
106 void fn_param8(unsigned char v);
107 void fn_param16(unsigned int v);
108
109 /* Payload. fn_tx_padded streams a NUL-terminated string
110 and then pads with
111 * NULs to 'total' bytes -- SET_DEVICE_FULLPATH and
112 OPEN_DIRECTORY take a
113 * fixed 256-byte buffer and a short one is rejected on
114 the ESP32 side. */
115 void fn_tx(unsigned char b);
116 void fn_tx_bytes(const unsigned char *p, unsigned int n);
117 void fn_tx_padded(const char *s, unsigned int total);
118
119 /* Launch, and wait for the reply. Returns FN_OK / FN_E*.
120 *
121 * The sequence number comes from the cartridge's own
122 FN_ACKSEQ + 1, never from
123 * a variable of ours. A console RESET restarts this
124 program and re-zeroes
125 * everything it owns, but does NOT reset the cartridge
126 -- deriving the
127 * sequence locally means every reset replays the same
128 number, the cart sees a
129 * sequence it has already acknowledged, and no request
130 is ever sent again
131 * while the stale reply sits there looking like success.
```

```

That bug cost the
107 * Intellivision bring-up a whole session; it is designed
out here. */
108 unsigned char fn_commit(void);
109
110 /* Did the far side ACK? Only meaningful after
fn_commit() == FN_OK. */
111 bool fn_acked(void);
112
113 /* Length of the reply currently in FN_REPLY. */
114 unsigned int fn_reply_len(void);
115
116 /* Arm and take the ROM swap. Does not return. */
117 void fn_boot_swap(void);
118
119 #endif /* FUJILIB_H */
120
121 /* --- streaming out of the reply window ---
122 *
123 * SET_DEVICE_FULLPATH and OPEN_DIRECTORY take a fixed
256-byte buffer, and the
124 * name to put in it is already sitting in the reply
window as cartridge ROM.
125 * The reply is only repainted by a SEQ commit or an
RXSLICE select (see the
126 * invariant in fuji_mailbox.h), so it survives while the
next transaction's TX
127 * stream is being built -- and a directory entry can go
from the cartridge to
128 * the cartridge without ever occupying any of the
console's ~700 bytes of RAM.
129 */
130 void fn_tx_path(const char *prefix, volatile unsigned
char *name,
131                unsigned int total);
132
133 /* Stream `n` bytes of the reply window, starting at
`off`, into the outgoing
134 * payload. This is what lets WRITE_HOST_SLOTS -- which
takes all eight 32-byte
135 * slots in one 256-byte payload, with no per-slot form
-- be answered without a
136 * 256-byte mirror in a machine that has under a kilobyte
of RAM: read the eight
137 * slots, then stream them straight back with the edited
one substituted. */
138 void fn_tx_from_reply(unsigned int off, unsigned int n);
139

```

FUJILIB.C

```

1 #include <string.h>
2
3 #include <os7.h>
4
5 #include "fujilib.h"
6
7 /* Every mailbox access is a READ whose value we do not
want -- the address is
8 * the message. Writing that as `(void)FN_REGSEL[reg];`
looks obviously right
9 * and is silently wrong: sccz80 emits the address
calculation and then throws
10 * the load away, volatile or not. The generated fn_regwr
was
11 *
12 *      ld de,64768      ; $FD00, computed...
13 *      ld de,65024      ; ...and immediately overwritten
by $FE00
14 *
15 * -- the REGSEL half of every register write simply did
not happen, so no
16 * transaction ever launched and the client sat polling
ACKSEQ forever. Storing
17 * the byte somewhere volatile is what makes the read a
side effect the
18 * compiler has to keep. Check the generated code if this
is ever rewritten:
19 * `zcc +coleco -O2 -a fujilib.c` and look for two loads,
not one. */
20 volatile unsigned char fn_sink;
21
22 #define FN_TOUCH(addr) (fn_sink = *(volatile unsigned
char *) (addr))
23
24 /* A register write is exactly two reads, back to back:
one to arm the register
25 * number, one to deliver the value. The cartridge
disarms after a single use,
26 * so a stray read of the data page on its own can never
change anything --
27 * which is what makes the vblank NMI harmless here. The
NMI can land between
28 * these two lines; the armed register simply waits. */
29 static void fn_regwr(unsigned char reg, unsigned char
val)
30 {
31     FN_TOUCH(FN_REGSEL + reg);
32     FN_TOUCH(FN_REGDAT + val);
33 }
34
35 bool fn_present(void)
36 {
37     return FN_MAGIC0 == 'F' && FN_MAGIC1 == 'N';
38 }
39
40 void fn_tx_path(const char *prefix, volatile unsigned
char *name,
41                unsigned int total)
42 {
43     unsigned int n = 0;
44
45     while (*prefix && n < total) {
46         FN_TOUCH(FN_TXPAGE + (unsigned char)*prefix++);
47         n++;
48     }
49     while (n < total) {
50         unsigned char c = *name++;
51
52         if (c == 0)
53             break;
54         FN_TOUCH(FN_TXPAGE + c);
55         n++;
56     }
57     while (n++ < total)
58         FN_TOUCH(FN_TXPAGE);
59 }
60
61 void fn_tx_from_reply(unsigned int off, unsigned int n)
62 {
63     volatile unsigned char *r = FN_REPLY + off;
64
65     while (n--)
66         FN_TOUCH(FN_TXPAGE + *r++);
67 }
68
69 /* Parameters ride the same stream, each as {size,
value...} little-endian,
70 * with NPARAM counting them. Interleaving the count

```

```

update with the stream is
71 * safe: the register pages and the TX page are decoded
separately, so a
72 * REGSEL/REGDATA pair between two payload bytes disturbs
nothing. */
73 static unsigned char nparam;
74
75 void fn_start(unsigned char device, unsigned char
command)
76 {
77     nparam = 0;
78     fn_regwr(FNR_DATA_RST, 0);
79     fn_regwr(FNR_DEVICE, device);
80     fn_regwr(FNR_CMD, command);
81     fn_regwr(FNR_NPARAM, 0);
82 }
83
84 void fn_tx(unsigned char b)
85 {
86     FN_TOUCH(FN_TXPAGE + b);
87 }
88
89 void fn_tx_bytes(const unsigned char *p, unsigned int n)
90 {
91     while (n--)
92         FN_TOUCH(FN_TXPAGE + *p++);
93 }
94
95 void fn_tx_padded(const char *s, unsigned int total)
96 {
97     unsigned int n = 0;
98
99     while (*s && n < total) {
100         FN_TOUCH(FN_TXPAGE + (unsigned char)*s++);
101         n++;
102     }
103     while (n++ < total)
104         FN_TOUCH(FN_TXPAGE);
105 }
106
107 void fn_param8(unsigned char v)
108 {
109     FN_TOUCH(FN_TXPAGE + 1);
110     FN_TOUCH(FN_TXPAGE + v);
111     fn_regwr(FNR_NPARAM, ++nparam);
112 }
113
114 void fn_param16(unsigned int v)
115 {
116     FN_TOUCH(FN_TXPAGE + 2);
117     FN_TOUCH(FN_TXPAGE + (unsigned char)(v & 0xFF));
118     FN_TOUCH(FN_TXPAGE + (unsigned char)(v >> 8));
119     fn_regwr(FNR_NPARAM, ++nparam);
120 }
121
122 unsigned char fn_commit(void)
123 {
124     unsigned char want = (unsigned char)(FN_ACKSEQ + 1);
125     unsigned int outer, inner;
126
127     if (want == 0)
128         want = 1; /* 0 means "never used"
129 */
130     nparam = 0;
131     fn_regwr(FNR_SEQ, want);
132
133     /* The cartridge's own transaction budget is 5 s (60
s for MOUNT_IMAGE);
134     * wait longer than that so a real timeout is
reported as the cart's error
135     * code rather than as ours. The loop is deliberately
dumb: OS7's timers
136     * run off the vblank NMI, and a client that has not
installed one yet --
137     * fujitest has not -- would wait forever on them. */
138     for (outer = 0; outer < 2400u; outer++) {
139         for (inner = 0; inner < 250u; inner++) {
140             if (FN_ACKSEQ == want)
141                 return FN_ERRCODE;
142         }
143     }
144     return FN_WAIT;
145 }
146
147 bool fn_acked(void)
148 {
149     return FN_REPLYCMD == FN_ACK;
150 }
151
152 unsigned int fn_reply_len(void)
153 {
154     return (unsigned int)FN_RXLEN_LO | ((unsigned
int)FN_RXLEN_HI << 8);
155 }
156
157 /* The swap stub, and why it is shaped like this.
158 * The cartridge replaces every byte of $8000-$FFFF

```

```

between one read and the
159 * next, so whatever triggers it must not itself be in
cartridge space. It goes
160 * in our own BSS -- NOT at a hard-coded $6000, which is
where z88dk puts the
161 * BSS anyway ($702C mirrors to $602C in the 1K of RAM).
162 *
163 * The vblank interrupt is on the Z80's /NMI and cannot
be masked with DI, and
164 * the BIOS vectors it straight to $8021 -- a byte that
belongs to the NEXT
165 * image the moment we swap. So the stub silences the VDP
first, clears the
166 * flag that is already latched, lets any in-flight NMI
be taken while our own
167 * handler is still there, and only then swaps.
168 *
169 * It also resets the VDP registers before jumping. The
BIOS's $55AA path is
170 * LD HL,($800A) / JP (HL) with no VDP or RAM
initialisation at all, so a game
171 * that assumes a cold VDP would otherwise inherit
whatever mode CONFIG left
172 * behind. Silencing the sound chip is the same courtesy.
173 */
174 static unsigned char stub[64];
175
176 static const unsigned char stub_src[] = {
177     /* Screen off, VDP interrupt off: write $00 to VDP
register 1. */
178     0x3E, 0x00, /* ld a,$00 */
179     0xD3, 0xBF, /* out ($bf),a */
180     0x3E, 0x81, /* ld a,$81 */
181     /* register 1 */
182     0xD3, 0xBF, /* out ($bf),a */
183     /* Clear the VDP's latched interrupt flag (and de-
assert /NMI). */
184     0xDB, 0xBF, /* in a,($bf) */
185     /* Give any NMI already in flight a moment to be
taken while OUR $8021
186     * handler is still the live one. */
187     0x00, 0x00, 0x00, 0x00, /* nop x4 */
188     0x00, 0x00, 0x00, 0x00, /* Silence the SN76489: attenuate all four channels.
*/
189     0x3E, 0x9F, 0xD3, 0xFF, /* ld a,$9f / out ($ff),a */
190     0x3E, 0xBF, 0xD3, 0xFF,
191     0x3E, 0xDF, 0xD3, 0xFF,
192     0x3E, 0xFF, 0xD3, 0xFF,
193     /* THE SWAP: one read of $FDfE. Every byte of
cartridge space changes
194     * between this instruction and the next. */
195     0x3A, 0xFE, 0xFD, /* ld a,($fdfe) */
196     /* Registers 0..7 to a power-on-like state, so a
$55AA game that skips the
197     * BIOS title screen does not inherit our video mode.
*/
198     0x21, 0x00, 0x80, /* ld hl,$8000 */
199     /* h=reg data, l=reg # */
200     0x06, 0x08, /* ld b,8 */
201     0x7D, /* vloop: ld a,l */
202     0xD3, 0xBF, /* out ($bf),a */
203     /* data byte (0) -- see below */
204     0x7C, /* ld a,h */
205     0xD3, 0xBF, /* out ($bf),a */
206     /* $80|reg */
207     0x24, /* inc h */
208     0x10, 0xF8, /* djnz vloop */
209     /* Cold start: the BIOS re-reads the new image's
$8000 header, so $55AA and
210     * $AA55 behave exactly as they would on a real
cartridge. */
211     0xC3, 0x00, 0x00 /* jp $0000 */
212 };
213
214 void fn_boot_swap(void)
215 {
216     memcpy(stub, stub_src, sizeof stub_src);
217     fn_regwr(FNR_BOOTLOCK, FN_BOOTLOCK_MAGIC);
218     ((void (*)(void))stub)();
219     for (;;)
220         ; /* not reached */
221 }

```

FUJIDISP.H -- OS7 TEXT DISPLAY

```

1  /* fujidisp.h -- the small display and input layer every
client shares.
2  *
3  * Everything here goes through OS7, the ColecoVision's
own BIOS, rather than
4  * touching the VDP directly: mode_1 lays out the
standard Graphics-1 tables,
5  * load_ascii fills the pattern generator with the BIOS's
own ASCII set, and
6  * put_vram writes into the pattern name table. That
gives a 32x24 text screen
7  * for free -- roomier than any other console in this
family -- and it is the
8  * point of building the client against os7lib in the
first place.
9  */
10
11 #ifndef FUJIDISP_H
12 #define FUJIDISP_H
13
14 #include <os7.h>
15 #include <stdbool.h>
16
17 #define DISP_COLS 32
18 #define DISP_ROWS 24
19
20 /* The inverse charset lives at pattern 0x80, a verbatim
copy of the ASCII set
21 * at 0x20 with the colour nibbles swapped. See
disp_init(). */
22 #define DISP_INVERSE 0x60
23
24 /* Graphics 1, the BIOS ASCII charset, white on dark
blue, screen and vblank
25 * interrupt both on. Safe to call more than once and in
any order relative to
26 * in_init(). */
27 void disp_init(void);
28 void disp_cls(void);
29 /* Write 's' at (col,row), clipped to the row. Not NUL-
padded: whatever was
30 * there stays, so callers that need a clean row call
disp_row_clear first. */
31 void disp_at(unsigned char col, unsigned char row, const
char *s);
32 void disp_row_clear(unsigned char row);
33 /* One character. Used to paint names straight out of the
cartridge's reply
34 * window, which is volatile and therefore not a 'const
char *'. */
35 void disp_char(unsigned char col, unsigned char row, char
c);
36 /* Write a string, optionally drawn from the inverse
charset. Cheaper than
37 * disp_row_invert for anything whose text we already
have in hand -- no VRAM
38 * read-back -- and it works on part of a row, which the
row version cannot. */
39 void disp_at_hi(unsigned char col, unsigned char row,
const char *s, bool hi);
40
41 /* One character, optionally inverse. */
42 void disp_char_hi(unsigned char col, unsigned char row,
char c, bool hi);
43
44 /* Turn a whole row inverse (dark blue on white) or back
again. This is the
45 * selection bar: the ADAM CONFIG gets the same effect
from Graphics 2's
46 * per-cell colour table, which we do not have in
Graphics 1, so instead the
47 * row's characters are re-pointed at an inverted copy of
the charset.
48 *
49 * The row is read back out of the name table rather than
redrawn, because
50 * there is nothing to redraw it FROM -- the file browser
paints names straight
51 * out of the mailbox reply window and never keeps a
copy. So moving the bar
52 * costs two 32-byte VRAM round trips and no network
traffic. Idempotent in
53 * both directions. */
54 void disp_row_invert(unsigned char row, bool on);
55
56 /* Right-aligned unsigned decimal, for lengths and error
codes. */
57 void disp_at_u16(unsigned char col, unsigned char row,
unsigned int v);
58 void disp_at_hex8(unsigned char col, unsigned char row,
unsigned char v);
59
60 #endif /* FUJIDISP_H */
61

```

FUJIDISP.C

```

1 #include <string.h>
2
3 #include "fujidisp.h"
4
5 /* Shared by the one-off charset copy at init and by
every bar move afterwards;
6 * the two are never live at the same time and RAM here
is 952 bytes total. */
7 static unsigned char vbuf[64];
8
9 void disp_init(void)
10 {
11     /* Clear OS7's own RAM variables before calling into
it.
12     *
13     * A cartridge whose header magic is $55AA gets `LD
HL,($800A) / JP (HL)
14     * with no BIOS initialisation of any kind -- which
is what we want on a
15     * cold boot, and a trap on a warm one. A console
RESET restarts the Z80
16     * but does not clear RAM, so OS7's variables still
hold the previous run's
17     * values while z88dk's crt0 clears only its own BSS
from $702C up. OS7's
18     * WRITE_VRAM and friends read DEFER_WRITES and
VDP_STATUS_BYTE without
19     * ever initialising them -- on a real cartridge the
BIOS title screen does
20     * that, and the $55AA path skips it. Three bytes of
hygiene, cheap enough
21     * not to argue about. */
22     VDP_STATUS_BYTE = 0;
23     DEFER_WRITES = 0;
24     MUX_SPRITES = 0;
25
26     mode_1();
27     load_ascii();
28     /* White on dark blue, for every pattern group and
the border alike --
29     * the same palette OS7 paints its own skill/players
select screen in.
30     * Verified rather than guessed: GAME_OPT's colour
table reads 0xF4 in
31     * all 32 entries (foreground 15 white, background 4
dark blue), so a
32     * FujiNet client looks like it belongs on the
machine instead of
33     * announcing itself. */
34     /* Build the inverse charset. LOAD_ASCII puts 96
patterns at index 0x1B, so
35     * pattern index == ASCII code and everything from
0x7D up is free; copying
36     * 0x20-0x7F to 0x80-0xDF (+0x60, and group-aligned)
gives a second,
37     * identical set whose only difference is the colour
entry it lands under.
38     *
39     * The glyphs are copied VERBATIM -- not bit-
inverted. Inverting the bits
40     * and swapping the colours would cancel out and hand
back normal video.
41     * The 5x7 glyph cells leave their right columns and
bottom row clear, and
42     * those pixels take the background colour, which is
what makes adjacent
43     * inverse cells butt together into one unbroken bar.
44     *
45     * Counts here are in PATTERNS, not bytes: put_vram/
get_vram scale by 8 for
46     * the generator table (and, in Graphics 1 only, not
at all for the colour
47     * table). */
48     {
49         unsigned char i;
50
51         for (i = 0; i < 12; i++) {
52             get_vram(PATTERN_GENERATOR_TABLE,
(unsigned short)(0x20 + (i << 3)),
vbuf, 8);
53             put_vram(PATTERN_GENERATOR_TABLE,
(unsigned short)(0x80 + (i << 3)),
vbuf, 8);
54         }
55     }
56
57     /* Groups 0-15 (patterns 0x00-0x7F) white on dark
blue; groups 16-27
60     * (0x80-0xDF, the inverse set) dark blue on white.
*/
61     fill_vram(0x2000, 32, 0xF4);
62     fill_vram(0x2010, 12, 0x4F);
63     write_register(REGISTER_BACKGROUND, DARK_BLUE);
64     disp_cls();

```

```

65
66     /* Screen on, vblank interrupt on -- LAST, because
mode_1() above resets
67     * VDP register 1 and would otherwise silently switch
the interrupt back
68     * off. That is not hypothetical: adding the splash
moved in_init() to
69     * before disp_init(), and this line is what stops
the second mode_1() from
70     * killing the NMI that POLLER rides on, leaving the
client alive but deaf
71     * to the controller. */
72     blank(true);
73 }
74
75 void disp_cls(void)
76 {
77     unsigned char row;
78
79     for (row = 0; row < DISP_ROWS; row++)
80         disp_row_clear(row);
81 }
82
83 void disp_row_clear(unsigned char row)
84 {
85     static const char blank[DISP_COLS + 1] =
86         " ";
87
88     put_vram(PATTERN_NAME_TABLE, (unsigned short)row *
DISP_COLS,
89             (void *)blank, DISP_COLS);
90 }
91
92 void disp_at(unsigned char col, unsigned char row, const
char *s)
93 {
94     unsigned int len = (unsigned int)strlen(s);
95
96     if (col >= DISP_COLS)
97         return;
98     if (len > (unsigned int)(DISP_COLS - col))
99         len = (unsigned int)(DISP_COLS - col);
100     if (len == 0)
101         return;
102     put_vram(PATTERN_NAME_TABLE, (unsigned short)row *
DISP_COLS + col,
103             (void *)s, (unsigned short)len);
104 }
105
106 void disp_char(unsigned char col, unsigned char row, char
c)
107 {
108     static char one[1];
109
110     if (col >= DISP_COLS)
111         return;
112     one[0] = (c < 32 || c > 126) ? ' ' : c;
113     put_vram(PATTERN_NAME_TABLE, (unsigned short)row *
DISP_COLS + col,
114             (void *)one, 1);
115 }
116
117 void disp_char_hi(unsigned char col, unsigned char row,
char c, bool hi)
118 {
119     unsigned char b = (unsigned char)((c < 32 || c >
126) ? ' ' : c);
120
121     if (hi && b >= 0x20 && b < 0x80)
122         b = (unsigned char)(b + DISP_INVERSE);
123     vbuf[0] = b;
124     put_vram(PATTERN_NAME_TABLE, (unsigned short)row *
DISP_COLS + col,
125             vbuf, 1);
126 }
127
128 void disp_at_hi(unsigned char col, unsigned char row,
const char *s, bool hi)
129 {
130     unsigned int n = 0;
131
132     while (s[n] != '\0' && col + n < DISP_COLS) {
133         unsigned char b = (unsigned char)s[n];
134
135         if (b < 32 || b > 126)
136             b = ' ';
137         if (hi && b >= 0x20 && b < 0x80)
138             b = (unsigned char)(b + DISP_INVERSE);
139         vbuf[n] = b;
140         n++;
141     }
142     if (n != 0)
143         put_vram(PATTERN_NAME_TABLE, (unsigned short)row
* DISP_COLS + col,

```

```

144         vbuf, (unsigned short)n);
145     }
146
147 void disp_row_invert(unsigned char row, bool on)
148 {
149     unsigned short base = (unsigned short)row *
DISP_COLS;
150     unsigned char i;
151
152     get_vram(PATTERN_NAME_TABLE, base, vbuf, DISP_COLS);
153     for (i = 0; i < DISP_COLS; i++) {
154         unsigned char c = vbuf[i];
155
156         /* Range-checked both ways, so calling this twice
is harmless. */
157         if (on) {
158             if (c >= 0x20 && c < 0x80)
159                 vbuf[i] = (unsigned char)(c +
DISP_INVERSE);
160             } else if (c >= 0x80) {
161                 vbuf[i] = (unsigned char)(c - DISP_INVERSE);
162             }
163         }
164     put_vram(PATTERN_NAME_TABLE, base, vbuf, DISP_COLS);
165 }
166
167 void disp_at_u16(unsigned char col, unsigned char row,
unsigned int v)
168 {
169     char buf[6];
170     unsigned char i = 5;
171
172     buf[5] = '\0';
173     do {
174         buf[--i] = (char)('0' + (v % 10u));
175         v /= 10u;
176     } while (v != 0 && i != 0);
177     disp_at(col, row, buf + i);
178 }
179
180 void disp_at_hex8(unsigned char col, unsigned char row,
unsigned char v)
181 {
182     static const char hex[] = "0123456789ABCDEF";
183     char buf[3];
184
185     buf[0] = hex[(v >> 4) & 0x0F];
186     buf[1] = hex[v & 0x0F];
187     buf[2] = '\0';
188     disp_at(col, row, buf);
189 }
190

```

FUJII.H -- OS7 CONTROLLER INPUT

```

1  /* fujiin.h -- controller input, entirely through OS7.
2  *
3  * OS7's POLLER does the scanning and debouncing and writes into the cartridge
4  * header's controller data area -- the buffer z88dk reserves as
5  * os7_bios_controller and os7lib maps as ControllerData. So a client's whole
6  * input layer is: call poller() from the vblank NMI, read the struct from the
7  * main loop, and turn levels into edges. No port banging, no debounce of our
8  * own, no scan table.
9  */
10
11 #ifndef FUJII_H
12 #define FUJII_H
13
14 #include <stdbool.h>
15
16 /* Edge-triggered events, one per call. */
17 #define IN_NONE    0
18 #define IN_UP      1
19 #define IN_DOWN    2
20 #define IN_LEFT    3
21 #define IN_RIGHT   4
22 #define IN_FIRE    5
23 #define IN_KEY0    0x10    /* + digit, so IN_KEY0 + 7 is keypad 7 */
24 #define IN_KEYSTAR 0x1A
25 #define IN_KEYHASH 0x1B
26
27 /* Install the NMI handler (which calls poller() and decays the sound click)
28 * and enable both controllers.
29 * The handler touches only the BIOS, the VDP ports and RAM -- never the
30 * mailbox pages -- which is what makes the vblank NMI safe to leave running
31 * through a transaction. */
32 void in_init(void);
33
34 /* Vblanks since in_init(). Wraps at 256, which is fine for the short waits it
35 * is used for -- the splash's minimum hold. */
36 unsigned char in_frames(void);

```

```

37
38 /* One event, or IN_NONE. Auto-repeats a held direction so paging a long
39 * directory does not need 40 separate pushes. */
40 unsigned char in_read(void);
41
42 #endif /* FUJIIIN_H */
43

```

FUJIIIN.C

```

1 #include <os7.h>
2 #include <interrupt.h>
3 #include <arch/coleco.h>
4
5 #include "fujiiin.h"
6 #include "fujisnd.h"
7
8 #define JOY_UP 0x01
9 #define JOY_RIGHT 0x02
10 #define JOY_DOWN 0x04
11 #define JOY_LEFT 0x08
12 #define BTN_FIRE 0x40
13
14 /* OS7 reports "no key" as a value outside the keypad's
own 0-9, *, # range. */
15 #define KEY_NONE_MIN 0x0C
16
17 #define REPEAT_FIRST 20 /* vblanks before a held
direction repeats */
18 #define REPEAT_NEXT 6 /* vblanks between repeats
after that */
19
20 static ControllerData *ctl = (ControllerData
*)os7_bios_controller;
21
22 static unsigned char last_dir;
23 static unsigned char last_fire;
24 static unsigned char last_key;
25 static unsigned char hold;
26
27 /* Bumped by the NMI, read by in_read(). The repeat delay
has to be counted in
28 * VBLANKS, not in calls: the main loop polls in_read()
thousands of times a
29 * second, so ageing the counter per call would run the
auto-repeat at loop
30 * speed and fire the cursor across a whole page in one
press. (Audible the
31 * moment cursor movement got a click -- a single held
direction produced a
32 * burst of them.) */
33 static volatile unsigned char frames;
34 static unsigned char last_frame;
35
36 /* The vblank handler. Reading the VDP status register is
what clears the
37 * interrupt; poller() is what makes the controller
struct mean anything. Both
38 * live in the BIOS, so this never touches $F800 and up
-- the one rule the
39 * mailbox needs from an NMI it cannot mask. */
40 static void nmi(void)
41 {
42     M_PRESERVE_ALL;
43     frames++;
44     poller();
45     snd_tick();
46     VDP_STATUS_BYTE = read_register();
47     M_RESTORE_ALL;
48 }
49
50 void in_init(void)
51 {
52     ctl->player1_enable = 0xFF;
53     ctl->player2_enable = 0xFF;
54     add_raster_int(nmi);
55     /* VDP register 1: screen on, 16K, vblank interrupt
enabled. */
56     write_register(0x01, 0xE0);
57 }
58
59 unsigned char in_frames(void)
60 {
61     return frames;
62 }
63
64 unsigned char in_read(void)
65 {
66     unsigned char now = frames;
67     bool vblank = (now != last_frame);
68     unsigned char joy = ctl->player1.joystick;
69     unsigned char fire = (unsigned char)((ctl-
>player1.left_button
70 | ctl-
>player1.right_button)
71 & BTN_FIRE);
72     unsigned char key = ctl->player1.keyboard;
73     unsigned char dir = 0;
74
75     last_frame = now;
76
77     if (joy & JOY_UP) dir = IN_UP;
78     else if (joy & JOY_DOWN) dir = IN_DOWN;
79     else if (joy & JOY_LEFT) dir = IN_LEFT;
80     else if (joy & JOY_RIGHT) dir = IN_RIGHT;
81
82     if (dir != 0) {
83         if (dir != last_dir) {
84             last_dir = dir;
85             hold = REPEAT_FIRST;
86             return dir;
87         }
88         if (vblank && hold != 0 && --hold == 0) {
89             hold = REPEAT_NEXT;
90             return dir;
91         }
92     } else {
93         last_dir = 0;
94     }
95
96     if (fire != 0 && last_fire == 0) {
97         last_fire = fire;
98         return IN_FIRE;
99     }
100     if (fire == 0)
101         last_fire = 0;
102
103     if (key < KEY_NONE_MIN) {
104         if (key != last_key) {
105             last_key = key;
106             return (unsigned char)(IN_KEY0 + key);
107         }
108     } else {
109         last_key = 0xFF;
110     }
111
112     return IN_NONE;
113 }
114

```

FUJIEDIT.H -- THE ON-SCREEN KEYBOARD

```

1 /* fujiedit.h -- the on-screen keyboard.
2 *
3 * A ColecoVision controller has twelve keys and a stick, so any text a user
4 * has to type -- a host name, a WiFi password -- gets typed on screen. This is
5 * the same grid the Intellivision client's grid_entry and the Astrocade's
6 * edit.inc use, and the reason all three look alike is the same insight: all
7 * 64 printable characters from space to underscore fit in 16 columns by 4
8 * rows, so the cursor position IS the character and there is no shift key and
9 * no paging.
10 *
11 * Two rules inherited deliberately from those two:
12 *
13 * - CASE is a toggle that redraws the letter rows in real lowercase, not a
14 * shift modifier. You can see what you are about to type.
15 * - Cancel is the ESC cell and nothing else. The backspace key must never
16 * discard the edit, because reaching for it repeatedly while correcting a
17 * typo is exactly when a reflexive extra press happens.
18 *
19 * The editor runs NO mailbox transactions. That is a contract, not an
20 * accident: callers hold data in the cartridge's reply window across the call
21 * -- the host-slot writer reads 8 slots, edits one here, and streams all 8
22 * back out of that same window -- and any transaction would repaint it.
23 */
24
25 #ifndef FUJIEDIT_H
26 #define FUJIEDIT_H
27
28 #include <stdbool.h>
29
30 /* The edit buffer. The caller preloads it NUL-terminated and reads it back
31 * afterwards; it is mutated either way, so copy out only on accept. Sized for
32 * the longest thing anyone types here, a 63-character WiFi password. */
33 #define FN_ENTRY_MAX 64
34 extern char fn_entry[FN_ENTRY_MAX];
35
36 /* Run the editor over fn_entry. `maxlen` is the longest string to allow, not
37 * counting the NUL. Returns true on accept, false on cancel. */
38 bool fn_edit(const char *title, unsigned char maxlen);
39
40 #endif /* FUJIEDIT_H */
41

```

FUJIEDIT.C

```

1 #include <string.h>
2
3 #include <os7.h>
4
5 #include "fujidisp.h"
6 #include "fujiedit.h"
7 #include "fujii.h"
8 #include "fujisnd.h"
9
10 /* Screen geometry. The grid is 16 wide on a 32-column
11 screen, so it sits in
12 * the middle with 8 columns of margin either side. */
13 #define TITLE_ROW 1
14 #define VALUE_ROW 4
15 #define VALUE_COL 1
16 #define VALUE_W 30 /* the value line's
17 visible window */
18 #define GRID_COL0 8
19 #define GRID_ROW0 8
20 #define GRID_PITCH 2 /* a blank line between
21 rows, for legibility */
22 #define GRID_COLS 16
23 #define GRID_ROWS 4
24 #define ACT_ROW 17
25 #define ACT_CELLS 5
26 #define FOOT_ROW 22
27
28 char fn_entry[FN_ENTRY_MAX];
29
30 static unsigned char glen; /* current length */
31 static unsigned char gmax; /* caller's limit */
32 static unsigned char gcase; /* 0 upper, 1 lower */
33 static unsigned char gx, gy; /* cursor; gy ==
34 GRID_ROWS is the action row */
35
36 static const char *act_text[ACT_CELLS] = { "CASE", "SPC",
37 "DEL", "OK", "ESC" };
38 static const unsigned char act_col[ACT_CELLS] = { 1, 8,
39 13, 19, 24 };
40
41 /* The character a grid cell types, after the CASE
42 toggle. Only A-Z is folded;
43 * digits and punctuation are the same either way. */
44 static char cell_char(unsigned char cx, unsigned char cy)
45 {
46     char c = (char)(0x20 + cy * GRID_COLS + cx);
47
48     if (gcase && c >= 'A' && c <= 'Z')
49         c = (char)(c + 0x20);
50     return c;
51 }
52
53 static unsigned char grid_screen_row(unsigned char cy)
54 {
55     return (unsigned char)(GRID_ROW0 + cy * GRID_PITCH);
56 }
57
58 static void draw_cell(unsigned char cx, unsigned char cy,
59 bool sel)
60 {
61     disp_char_hi((unsigned char)(GRID_COL0 + cx),
62 grid_screen_row(cy),
63 cell_char(cx, cy), sel);
64 }
65
66 static void draw_grid(void)
67 {
68     unsigned char cx, cy;
69
70     for (cy = 0; cy < GRID_ROWS; cy++)
71         for (cx = 0; cx < GRID_COLS; cx++)
72             draw_cell(cx, cy, (bool)(gy == cy && gx ==
73 cx));
74 }
75
76 static void draw_actions(void)
77 {
78     unsigned char i;
79
80     for (i = 0; i < ACT_CELLS; i++)
81         disp_at_hi(act_col[i], ACT_ROW, act_text[i],
82 (bool)(gy == GRID_ROWS && gx == i));
83 }

```

```

73 }
74
75 /* The value line: the tail of the buffer, plus one
inverse cell sitting at the
76 * append point so it is obvious where the next character
lands. There is no
77 * left/right text cursor -- editing is append and
backspace only. */
78 static void draw_value(void)
79 {
80     unsigned char start = 0;
81     unsigned char i;
82
83     if (glen > VALUE_W - 1)
84         start = (unsigned char)(glen - (VALUE_W - 1));
85
86     disp_row_clear(VALUE_ROW);
87     for (i = 0; i < VALUE_W; i++) {
88         unsigned char idx = (unsigned char)(start + i);
89         bool cursor = (bool)(idx == glen);
90
91         disp_char_hi((unsigned char)(VALUE_COL + i),
VALUE_ROW,
92                     idx < glen ? fn_entry[idx] : ' ',
cursor);
93     }
94 }
95
96 static void move_to(unsigned char nx, unsigned char ny)
97 {
98     /* Un-highlight where we were. */
99     if (gy == GRID_ROWS)
100         disp_at_hi(act_col[gx], ACT_ROW, act_text[gx],
false);
101     else
102         draw_cell(gx, gy, false);
103
104     gx = nx;
105     gy = ny;
106
107     if (gy == GRID_ROWS)
108         disp_at_hi(act_col[gx], ACT_ROW, act_text[gx],
true);
109     else
110         draw_cell(gx, gy, true);
111     snd_click();
112 }
113
114 static void type_char(char c)
115 {
116     if (glen >= gmax)
117         return; /* full: silently ignore,
as the family does */
118     fn_entry[glen++] = c;
119     fn_entry[glen] = '\0';
120     draw_value();
121     snd_click();
122 }
123
124 static void backspace(void)
125 {
126     if (glen == 0)
127         return;
128     fn_entry[--glen] = '\0';
129     draw_value();
130     snd_click();
131 }
132
133 bool fn_edit(const char *title, unsigned char maxlen)
134 {
135     if (maxlen > FN_ENTRY_MAX - 1)
136         maxlen = FN_ENTRY_MAX - 1;
137     gmax = maxlen;
138     fn_entry[maxlen] = '\0';
139     glen = (unsigned char)strlen(fn_entry);
140     gcase = 0;
141     gx = 0;
142     gy = 2; /* start on the A-0 row,
where letters are */
143
144     disp_cls();
145     disp_at(1, TITLE_ROW, title);
146     disp_at(1, FOOT_ROW, "FIRE PICK * DEL # OK");
147     draw_value();
148     draw_grid();
149     draw_actions();
150
151     for (;;) {
152         unsigned char ev = in_read();
153
154         switch (ev) {
155             case IN_NONE:
156                 break;
157
158             case IN_UP:
159                 if (gy == GRID_ROWS)
160                     /* Out of the 5-cell action row back into
the 16-cell grid;

```

```

161         * spread the column so the cursor lands
near where it looked
162         * like it was, rather than snapping to
the left edge. */
163         move_to((unsigned char)(gx * 3 + 1),
GRID_ROWS - 1);
164         else if (gy > 0)
165             move_to(gx, (unsigned char)(gy - 1));
166         break;
167
168     case IN_DOWN:
169         if (gy < GRID_ROWS - 1)
170             move_to(gx, (unsigned char)(gy + 1));
171         else if (gy == GRID_ROWS - 1)
172             move_to((unsigned char)(gx * ACT_CELLS /
GRID_COLS), GRID_ROWS);
173         break;
174
175     case IN_LEFT:
176         if (gx > 0)
177             move_to((unsigned char)(gx - 1), gy);
178         break;
179
180     case IN_RIGHT:
181         if (gy == GRID_ROWS) {
182             if (gx + 1 < ACT_CELLS)
183                 move_to((unsigned char)(gx + 1), gy);
184             } else if (gx + 1 < GRID_COLS) {
185                 move_to((unsigned char)(gx + 1), gy);
186             }
187         break;
188
189     case IN_FIRE:
190         if (gy < GRID_ROWS) {
191             type_char(cell_char(gx, gy));
192             break;
193         }
194         switch (gx) {
195             case 0: /* CASE */
196                 gcase = (unsigned char)(gcase ^ 1);
197                 draw_grid();
198                 snd_click();
199                 break;
200             case 1: /* SPC */
201                 type_char(' ');
202                 break;
203             case 2: /* DEL */
204                 backspace();
205                 break;
206             case 3: /* OK */
207                 return true;
208             default: /* ESC -- the only way to
cancel */
209                 return false;
210         }
211         break;
212
213     case IN_KEYSTAR:
214         backspace();
215         break;
216
217     case IN_KEYHASH:
218         return true;
219
220     default:
221         if (ev >= IN_KEY0 && ev <= IN_KEY0 + 9)
222             type_char((char)('0' + (ev - IN_KEY0)));
223         break;
224     }
225 }
226 }
227

```

FUJISND.H -- THE SN76489, KEPT QUIET

```

1 /* fujisnd.h -- the SN76489, kept quiet.
2 *
3 * A ColecoVision powers its sound chip up with all four channels at
4 * attenuation 0 -- full volume -- and their frequency registers at zero. The
5 * result is an immediate, permanent buzz. Every commercial cartridge is
6 * spared this because the BIOS title screen silences the chip before the game
7 * runs; a client whose header magic is $55AA skips the title screen and gets
8 * the noise instead. So snd_init() is not optional politeness, it is the first
9 * thing any client here must do.
10 *
11 * The click is done with direct port writes rather than OS7's SOUND_INIT /
12 * PLAY_IT / SOUND_MAN. That trio wants a note list and a per-sound RAM data
13 * area, which on a machine with ~700 usable bytes is a lot to spend on a
14 * cursor blip -- and the blip is two register writes. Everything else in these
15 * clients goes through OS7; this is the one place where it costs more than it
16 * saves.
17 */
18
19 #ifndef FUJISND_H
20 #define FUJISND_H
21
22 /* Silence all four channels. Call before anything else. */
23 void snd_init(void);
24
25 /* A short click, for cursor movement. Decays through snd_tick(). */
26 void snd_click(void);
27
28 /* Call once per vblank, from the NMI handler. */
29 void snd_tick(void);
30
31 #endif /* FUJISND_H */
32

```

FUJISND.C

```

1 #include <arch/z80.h>
2
3 #include "fujisnd.h"
4
5 #define PSG_PORT 0xFF
6
7 /* SN76489 command bytes. A latch byte is 1-cc-t-dddd: channel in bits 6:5,
8 * type in bit 4 (1 = attenuation, 0 = tone), low four data bits. A following
9 * 0-ddddd byte carries the tone's upper six bits. Attenuation 15 is off. */
10 #define ATTEN(ch, a) (((unsigned char)(0x90 | ((ch) << 5) | (a)))
11 #define TONE_LO(ch, n) (((unsigned char)(0x80 | ((ch) << 5) | ((n) & 0x0F)))
12 #define TONE_HI(n) ((unsigned char)(((n) >> 4) & 0x3F))
13
14 /* 3579545 / (32 * N) Hz. 93 is about 1200 Hz -- high enough to read as a UI
15 * click rather than a note. */
16 #define CLICK_N 93
17 #define CLICK_ATTEN 4
18 #define CLICK_TICKS 2 /* vblanks; ~33ms */
19
20 static unsigned char ticks;
21
22 void snd_init(void)
23 {
24     z80_outp(PSG_PORT, ATTEN(0, 15));
25     z80_outp(PSG_PORT, ATTEN(1, 15));
26     z80_outp(PSG_PORT, ATTEN(2, 15));
27     z80_outp(PSG_PORT, ATTEN(3, 15));
28     ticks = 0;
29 }
30
31 void snd_click(void)
32 {
33     z80_outp(PSG_PORT, TONE_LO(0, CLICK_N));
34     z80_outp(PSG_PORT, TONE_HI(CLICK_N));
35     z80_outp(PSG_PORT, ATTEN(0, CLICK_ATTEN));
36     ticks = CLICK_TICKS;
37 }
38
39 void snd_tick(void)
40 {
41     if (ticks != 0 && --ticks == 0)
42         z80_outp(PSG_PORT, ATTEN(0, 15));
43 }
44

```

C.2 The Z80 assembler library

FUJIMAIL.INC -- THE MAILBOX, IN ASSEMBLY

```

1 ;
=====
2 ; fujimapil.inc -- the FujiNet mailbox, in Z80 assembly.
3 ;
4 ; The console's half of the cartridge mailbox, hand-
mirrored from
5 ; firmware/include/fuji_mailbox.h. Every address here is
a CONSOLE address
6 ; (the cartridge image offset plus $8000).
7 ;
8 ; Both directions of the mailbox are cartridge READS --
the ColecoVision
9 ; cartridge port has no write strobe, so the ADDRESS
carries the payload:
10 ;
11 ; console -> cart   reading $FD05 arms register 5;
reading $FE2A then
12 ;
13 ; Reading $FF41 appends           delivers the value $2A to it.
TX stream.                         the byte $41 ('A') to the outgoing
14 ; cart -> console   the cartridge repaints its own ROM,
so by the time the
15 ;
16 ; Z80 looks, the reply at $F800 is
just bytes.
17 ;
18 ; In C this took a volatile sink to stop sccz80
discarding the "read whose
19 ; value we throw away". In assembly there is no such
hazard: 'ld a,(hl)'
20 ;
21 ; always performs the read. That is the one real
advantage assembly has here.
22 ;
23 ; Calling conventions are register-based and documented
at each routine. The
24 ; two scratch bytes NPARAM and WANT live in low RAM; see
the demo's equates.
=====
25 ; ---- cart -> console: the painted status page and reply
window ----
26 FN_REPLY equ $F800           ; the whole 1K reply, in one
piece
27 FN_ACKSEQ equ $FC00           ; echoes our SEQ when the
reply is ready
28 FN_STATUS equ $FC01           ; bit0 link up, bit1 busy
29 FN_ERR equ $FC02              ; transport verdict of the
last transaction
30 FN_REPLYCMD equ $FC03          ; $06 ACK / $15 NAK
31 FN_RXLEN equ $FC04            ; reply length, little-
endian word
32 FN_BOOTSTAT equ $FC06
33 FN_BOOTPCT equ $FC07
34 FN_BOOTERR equ $FC08
35 FN_MAGIC0 equ $FC09           ; 'F' -- cartridge presence
36 FN_MAGIC1 equ $FC0A           ; 'N'
37
38 ; ---- console -> cart: the hotspot pages ----
39 FN_REGSELP equ $FD            ; high byte: read $FD00+r
arms register r
40 FN_REGDATP equ $FE            ; high byte: read $FE00+v
delivers v
41 FN_DATAP equ $FF              ; high byte: read $FF00+v
appends v to TX
42
43 ; ---- register file ----
44 FNR_DEVICE equ $00
45 FNR_CMD equ $01
46 FNR_NPARAM equ $02
47 FNR_DATA_RST equ $05
48 FNR_SEQ equ $10
49 FNR_BOOTLOCK equ $11
50 FN_BOOTLOCK_MAGIC equ $B5
51
52 ; ---- FujiBus reply codes ----
53 FN_ACK equ $06
54 FN_NAK equ $15
55
56 ;
=====
57 ; FNREGWR -- write one mailbox register.
58 ; in: C = register number, A = value
59 ; out: - clobbers: A, H, L
60 ;
61 ; A register write is exactly two reads, back to back:
one to arm the
62 ; register number, one to deliver the value. The
cartridge disarms after a
63 ; single use, so a lone stray read of the data page
changes nothing -- which
64 ; is what makes the unmaskable vblank NMI harmless

```

between these two lines.

```

65 ;
=====
66 FNREGWR:
67     push    af                ; hold the value
68     ld      h, FN_REGSELP     ; $FD00 + register: arm
it
69     ld      l, c
70     ld      a, (hl)
71     pop     af
72     ld      h, FN_REGDATP     ; $FE00 + value: deliver
it
73     ld      l, a
74     ld      a, (hl)
75     ret
76
77 ;
=====
78 ; FNTXB -- append one byte to the TX stream.
79 ; in: A = byte clobbers: H, L (A preserved
on exit? no -- see)
80 ;
=====
81 FNTXB:
82     ld      h, FN_DATAP       ; $FF00 + value
83     ld      l, a
84     ld      a, (hl)
85     ret
86
87 ;
=====
88 ; FNSTART -- begin a transaction.
89 ; in: D = device, E = command clobbers: A, H, L
90 ;
91 ; Rewinds the TX write pointer and sets DEVICE, COMMAND
and NPARAM. D and E
92 ; survive because FNREGWR touches only A, H and L.
Afterwards the stream is
93 ; empty and waiting for parameters and payload.
94 ;
=====
95 FNSTART:
96     xor     a
97     ld      (NPARAM), a
98     ld      c, FNR_DATA_RST   ; any value rewinds
the TX pointer
99     ld      a, 0
100    call    FNREGWR
101    ld      c, FNR_DEVICE
102    ld      a, d
103    call    FNREGWR
104    ld      c, FNR_CMD
105    ld      a, e
106    call    FNREGWR
107    ld      c, FNR_NPARAM
108    xor     a
109    call    FNREGWR
110    ret
111
112 ;
=====
113 ; FNP8 -- append a one-byte parameter and bump NPARAM.
114 ; in: A = value clobbers: A, B, C, H, L
115 ; The parameter grammar is {size byte, value bytes...}; a
byte is {1, v}.
116 ;
=====
117 FNP8:
118     ld      b, a                ; save value
119     ld      a, 1                ; size byte
120     call    FNTXB
121     ld      a, b
122     call    FNTXB                ; the value
123     ld      a, (NPARAM)
124     inc     a
125     ld      (NPARAM), a
126     ld      c, FNR_NPARAM       ; A already = new count
127     call    FNREGWR
128     ret
129
130 ;
=====
131 ; FNP16 -- append a two-byte (little-endian) parameter,
bump NPARAM.
132 ; in: HL = value clobbers: A, C, H, L
133 ;
=====
134 FNP16:
135     push    hl
136     ld      a, 2                ; size byte
137     call    FNTXB
138     pop     hl

```

```

139      push    hl
140      ld      a,l          ; low byte
141      call    FNTXB
142      pop     hl
143      ld      a,h          ; high byte
144      call    FNTXB
145      ld      a,(NPARAM)
146      inc     a
147      ld      (NPARAM),a
148      ld      c,FNR_NPARAM
149      call    FNREGWR
150      ret
151
152 ;
=====
153 ; FNTXPAD -- append a NUL-terminated string, then pad
with NULs to a fixed
154 ; width. SET_DEVICE_FULLPATH and OPEN_DIRECTORY want a
fixed 256-byte buffer;
155 ; a short one is rejected on the ESP32 side.
156 ; in: HL = string, DE = total byte count
clobbers: A, B, D, E, H, L
157 ;
=====
158 FNTXPAD:
159 FNTXP1:
160      ld      a,(hl)
161      or      a
162      jr      z,FNTXP2      ; NUL: switch to padding
163      push    hl
164      push    de
165      call    FNTXB          ; A = the character
166      pop     de
167      pop     hl
168      inc     hl
169      dec     de
170      ld      a,d
171      or      e
172      jr      nz,FNTXP1
173      ret                    ; filled exactly, no
padding needed
174 FNTXP2:
175      ld      a,d
176      or      e
177      ret     z              ; padded to width
178      push    de
179      xor     a              ; a NUL pad byte
180      call    FNTXB
181      pop     de
182      dec     de
183      jr      FNTXP2
184
185 ;
=====
186 ; FNCOMMIT -- launch the transaction and wait for the
reply.
187 ; out: A = FN_ERR (transport verdict); CARRY set if the
cart never answered.
188 ; clobbers: A, B, C, D, E, H, L
189 ;
190 ; The sequence number comes from the cartridge's own
FN_ACKSEQ + 1, wrapping
191 ; 255 -> 1 (0 is reserved), never from a local counter --
a console RESET
192 ; restarts this program but not the cartridge, and a
local counter would
193 ; replay a sequence the cart has already answered. Then
poll FN_ACKSEQ until
194 ; it echoes what we wrote.
195 ;
=====
196 FNCOMMIT:
197      ld      a,(FN_ACKSEQ)
198      inc     a
199      jr      nz,FNCMI
200      inc     a              ; 255 wrapped to 0; step
past the reserved 0
201 FNCMI:
202      ld      (WANT),a
203      ld      c,FNR_SEQ      ; A = want
204      call    FNREGWR
205      ld      b,40           ; outer quanta; inner is
a full 16-bit spin
206 FNCMO:
207      ld      de,0
208 FNCMI:
209      ld      a,(FN_ACKSEQ)
210      ld      hl,WANT
211      cp      (hl)
212      jr      z,FNCMOK
213      dec     de
214      ld      a,d
215      or      e
216      jr      nz,FNCMI
217      djnz   FNCMO
218      scf                    ; timed out -- the cart
never answered
219      ret
220 FNCMOK:
221      ld      a,(FN_ERR)
222      or      a              ; clears carry, sets Z if
FN_ERR == 0
223      ret
224
225 ;
=====
226 ; FNACKED -- did the far side ACK? out: Z set if
FN_REPLYCMD == FN_ACK.
227 ;
=====
228 FNACKED:
229      ld      a,(FN_REPLYCMD)
230      cp      FN_ACK
231      ret
232
233 ;
=====
234 ; FNPRES -- is a FujiNet cartridge underneath us?
235 ; out: Z set if the 'F','N' magic is painted.
clobbers: A
236 ;
=====
237 FNPRES:
238      ld      a,(FN_MAGIC0)
239      cp      'F'
240      ret     nz
241      ld      a,(FN_MAGIC1)
242      cp      'N'
243      ret
244

```

ACFG.ASM -- THE ASSEMBLY FIRST-CONTACT CARTRIDGE

```

1 ;
=====
2 ; acfg.asm -- GET_ADAPTERCONFIG_EXTENDED, in Z80
assembly.
3 ;
4 ; The assembly-language counterpart to the C fujitest:
the smallest program
5 ; that runs one whole mailbox transaction and shows what
came back. It is a
6 ; hand-written ColecoVision cartridge with its own
header, but for the screen
7 ; it does what every client in this book does -- it uses
OS7, the console's own
8 ; BIOS, through its jump table at $1Fxx. MODE_1 lays out
Graphics I, LOAD_ASCII
9 ; fills the pattern generator with the BIOS font so a
name-table byte is simply
10 ; its ASCII code, and PUT_VRAM writes the text. No font
of our own, no VDP
11 ; register banging: the mailbox is the only thing this
program does the hard
12 ; way, because the mailbox is the only thing OS7 does not
already know how to
13 ; do.
14 ;
15 ; It asks the adapter for its extended configuration
(device $70, command
16 ; $C4), then prints the SSID, IP address and firmware
version straight out of
17 ; the reply window at $F800.
18 ;
19 ; Assemble with build.sh (z88dk-z80asm), which pads the
image to 32768 bytes
20 ; and stamps the "FUJI" claim; run it in the grafted MAME
against a live
21 ; fujinet-pc.
22 ;
=====
23 ;
24 ; ---- OS7 BIOS entry points (the jump table at the top
of the BIOS ROM) ----
25 MODE_1 equ $1F85 ; set up Graphics I
tables
26 LOAD_ASCII equ $1F7F ; load the BIOS ASCII
font into the generator
27 FILL_VRAM equ $1F82 ; HL=addr, DE=count,
A=value
28 PUT_VRAM equ $1FBE ; A=table, DE=index,
HL=data, IY=count
29 ;
30 ; PUT_VRAM table codes
31 VDP_NAME equ 2 ; the pattern name table
(the text screen)
32 ;
33 ; ---- OS7 RAM variables PUT_VRAM consults; a $55,$AA
cart must clear them,
34 ; because the BIOS title screen that normally would is
skipped. ----
35 VDP_STATUS_BYTE equ $73C5
36 DEFER_WRITES equ $73C6
37 MUX_SPRITES equ $73C7
38 ;
39 ; ---- low-RAM scratch (the 1K at $6000 mirrors up to
$73FF) ----
40 SPRITES equ $7000 ; header-required RAM
pointers; unused here
41 SPRTORD equ $7080
42 WORKBUF equ $70A0
43 CTRLMAP equ $70C0
44 NPARAM equ $70E0 ; the mailbox library's
two scratch bytes
45 WANT equ $70E1
46 ;
47 ; a name-table index is row*32 + column
48 COLS equ 32
49 ;
50 org $8000
51 ;
52 ;
=====
53 ; Cartridge header. $55,$AA tells the BIOS to skip its
title screen and jump
54 ; straight to the address in $800A, so this program owns
the machine from its
55 ; first instruction -- and therefore has to set the
screen up itself.
56 ;
=====
57 defb $55,$AA ; skip the BIOS title
screen
58 defb SPRITES ; sprite name table (RAM)
59 defb SPRTORD ; sprite order table
60 defb WORKBUF ; work buffer
61 defb CTRLMAP ; controller map
62 defw START ; entry point ($55,$AA
jumps here)
63 jp RSTV ; rst 08
64 jp RSTV ; rst 10
65 jp RSTV ; rst 18
66 jp RSTV ; rst 20
67 jp RSTV ; rst 28
68 jp RSTV ; rst 30
69 jp RSTV ; rst 38
70 jp NMIH ; NMI (vblank) -- must be
valid; see below
71 ;
72 ;
=====
73 ; Entry.
74 ;
=====
75 START:
76 di ; the mailbox contract:
no maskable interrupts
77 ld sp,$73B8 ; stack below the BIOS
scratch at $73B9
78 ;
79 xor a ; clear the OS7 variables
the skipped title
80 ld (VDP_STATUS_BYTE),a ; screen would
normally have initialised
81 ld (DEFER_WRITES),a
82 ld (MUX_SPRITES),a
83 ;
84 call MODE_1 ; Graphics I
85 call LOAD_ASCII ; BIOS font: name-table
byte == ASCII code
86 ld hl,$2000 ; colour table: white on
dark blue, all groups
87 ld de,32
88 ld a,$F4
89 call FILL_VRAM
90 ;
91 ld hl,MTITLE ; row 0: the title
92 ld de,0
93 ld iy,MTITLE_L
94 call SHOW
95 ;
96 call FNPRES ; is a FujiNet cartridge
actually here?
97 jr nz,NOCART
98 ;
99 ld d,$70 ; device: the Fuji
adapter
100 ld e,$C4 ; command:
GET_ADAPTERCONFIG_EXTENDED
101 call FNSTART
102 call FNCOMMIT
103 jr c,ETMO ; carry: the cart never
answered
104 or a
105 jr nz,EERR ; FN_ERR nonzero: no
link, timeout, bad frame
106 call FNACKED
107 jr nz,ENAK ; the adapter refused the
command
108 ;
109 ; The reply is a 240-byte AdapterConfigExtended in the
window at $F800. Its
110 ; friendliest fields are text at fixed offsets: SSID at
+0, the firmware
111 ; version at +125, and the dotted-decimal IP at +140.
FNLEN measures each one
112 ; so PUT_VRAM writes only the real characters, not the
NUL padding behind them.
113 ld hl,LSSID
114 ld de,2*COLS
115 ld iy,LSSID_L
116 call SHOW
117 ld hl,FN_REPLY+0
118 ld de,3*COLS+2
119 ld b,32
120 call SHOWFLD
121 ;
122 ld hl,LIP
123 ld de,5*COLS
124 ld iy,LIP_L
125 call SHOW
126 ld hl,FN_REPLY+140
127 ld de,6*COLS+2
128 ld b,16
129 call SHOWFLD
130 ;
131 ld hl,LFW
132 ld de,8*COLS
133 ld iy,LFW_L
134 call SHOW
135 ld hl,FN_REPLY+125

```

```

136         ld      de,9*COLS+2
137         ld      b,15
138         call    SHOWFLD
139
140         ld      hl,MOK
141         ld      de,11*COLS
142         ld      iy,MOK_L
143         call    SHOW
144 HALT1:    jr      HALT1
145
146
147 NOCART:  ld      hl,MNOCART
148         ld      iy,MNOCART_L
149         jr      SHOWERR
150 ETMO:    ld      hl,MTMO
151         ld      iy,MTMO_L
152         jr      SHOWERR
153 EERR:    ld      hl,MERR
154         ld      iy,MERR_L
155         jr      SHOWERR
156 ENAK:    ld      hl,MNAK
157         ld      iy,MNAK_L
158 SHOWERR: ld      de,2*COLS
159         ld      call    SHOW
160         jr      HALT1
161
162
163 ;
=====
164 ; SHOW -- PUT_VRAM a fixed-length string to the name
table.
165 ; in: HL = data, DE = name index, IY = length
clobbers: everything
166 ; SHOWFLD -- the same, but for a NUL-terminated field of
at most B bytes out of
167 ; the reply window: measure it first, then draw only that
many.
168 ; in: HL = field, DE = name index, B = max length
169 ;
=====
170 SHOW:
171         ld      a,VDP_NAME
172         call    PUT_VRAM
173         ret
174
175 SHOWFLD:
176         ; count characters up to a NUL or B, without
disturbing HL/DE
177         push    hl
178         push    de
179         ld      c,0          ; running length
180 SF1:
181         ld      a,(hl)
182         or      a
183         jr      z,SF2
184         inc     hl
185         inc     c
186         djnz    SF1
187 SF2:
188         pop     de
189         pop     hl
190         ld      a,c
191         or      a
192         ret     z          ; empty field: draw
nothing
193         ld      iyl,c      ; IY = length for
PUT_VRAM
194         ld      iyh,0
195         ld      a,VDP_NAME
196         call    PUT_VRAM
197         ret
198
199 ; The vblank NMI cannot be masked, and the BIOS vectors
it here. MODE_1 leaves
200 ; the VDP interrupt disabled, so it should never fire --
but the connector
201 ; carries no reset line and the VDP powers up in an
unknown state, so the
202 ; vector must be valid regardless. Reading the status
port drops a latched
203 ; interrupt.
204 NMIIH:
205         push    af
206         in      a,($BF)
207         pop     af
208         retn
209
210 RSTV:
211         ret
212
213 ; ---- text (with assembled lengths, so SHOW needs no
strlen) ----
214 MTITLE:  defm "FUJINET ACFG DEMO"
215 MTITLE_L equ $ - MTITLE
216 LSSID:   defm "SSID"
217 LSSID_L  equ $ - LSSID
218 LIP:     defm "IP ADDRESS"
219 LIP_L    equ $ - LIP
220 LFW:     defm "FIRMWARE"
221 LFW_L    equ $ - LFW
222 MOK:     defm "OK"
223 MOK_L    equ $ - MOK
224 MNOCART: defm "NO FUJINET CART"
225 MNOCART_L equ $ - MNOCART
226 MTMO:    defm "TIMEOUT - NO ANSWER"
227 MTMO_L   equ $ - MTMO
228 MERR:    defm "TRANSPORT ERROR"
229 MERR_L   equ $ - MERR
230 MNAK:    defm "COMMAND REFUSED (NAK)"
231 MNAK_L   equ $ - MNAK
232
233 ; The mailbox library.
234         include "fujimail.inc"
235

```

C.3 NETCAT

NETLIB.H -- THE FIVE N: COMMANDS

```

1 /* netlib.h -- the N: network device, five commands, on
top of fujilib.
2 *
3 * Where fujilib is the transport (registers, TX stream,
the SEQ handshake),
4 * this is the network unit N1: (FujiBus device 0x71)
that rides on it: open a
5 * devicespec, ask how many bytes are waiting, read them,
write some back, hang
6 * up. Every one of these is one fujilib transaction --
fn_start, parameters,
7 * fn_commit -- so read fujilib.h first and this file is
nothing new.
8 *
9 * NETCAT uses only N1:. The eight network units
0x71-0x78 (N1:-N8:) are
10 * independent connections; a program that wanted two at
once would talk to two
11 * device ids, everything else the same.
12 */
13
14 #ifndef NETLIB_H
15 #define NETLIB_H
16
17 #include <stdbool.h>
18
19 /* The network device ids and the five command opcodes.
The opcodes are the
20 * ASCII letters of their names -- 'O'pen, 'S'tatus,
'R'ead, 'W'rite, 'C'lose --
21 * a habit inherited from the Atari SIO handlers. */
22 #define NET_DEV_N1 0x71
23 #define NET_OPEN 0x4F /* 'O' */
24 #define NET_STATUS 0x53 /* 'S' */
25 #define NET_READ 0x52 /* 'R' */
26 #define NET_WRITE 0x57 /* 'W' */
27 #define NET_CLOSE 0x43 /* 'C' */
28
29 /* NET_OPEN parameter 0, fileAccessMode_t (lib/network-
protocol/Protocol.h):
30 * READ 4, WRITE 8, READWRITE 12. A socket is read/write.
*/
31 #define NET_MODE_READ 4
32 #define NET_MODE_WRITE 8
33 #define NET_MODE_READWRITE 12
34
35 /* NET_OPEN parameter 1, netProtoTranslation_t: 0 none, 1
CR, 2 LF, 3 CR/LF.
36 * NETCAT sends 0 and does its own end-of-line handling
in the terminal. */
37 #define NET_TRANS_NONE 0
38
39 /* The fixed devicespec buffer the ESP32 expects for
NET_OPEN. A short payload
40 * is padded up to this; the transaction layer refuses
anything shorter. */
41 #define NET_SPEC_LEN 256
42
43 /* NET_READ / NET_WRITE ceiling. The reply window is 1K,
but the whole point of
44 * a terminal is to show bytes as they arrive, so a
modest read keeps the loop
45 * responsive; a write has to leave room in the 320-byte
TX stream for the
46 * length parameter. */
47 #define NET_READ_MAX 512
48 #define NET_WRITE_MAX 120
49
50 /* NET_STATUS byte 3 (device status): the two a terminal
cares about. */
51 #define NET_DS_SUCCESS 1
52 #define NET_DS_EOF 136
53
54 /* One-time init: bring up the mailbox client. Safe to
call more than once. */
55 bool net_present(void);
56
57 /* Open the devicespec (e.g. "N:TELNET://bbs.example.
com/") read/write. Returns
58 * true on ACK. */
59 bool net_open(const char *spec);
60
61 /* Hang up. Best-effort -- a terminal calls it on the way
out and does not
62 * care whether the far side was still there. */
63 void net_close(void);
64
65 /* Poll the connection. Fills *avail (bytes waiting),
*conn (socket up), and
66 * *devstatus (NET_DS_*). Returns false if the
transaction itself failed. */
67 bool net_status(unsigned int *avail, unsigned char *conn,
unsigned char *devstatus);
68
69
70 /* Read up to `want` bytes into the reply window. Returns
how many landed (from
71 * the transaction's own length bytes); the data is at
FN_REPLY, valid until the
72 * next transaction of any kind. */
73 unsigned int net_read(unsigned int want);
74
75 /* Write `len` bytes. Returns true on ACK. `len` is
capped at NET_WRITE_MAX. */
76 bool net_write(const unsigned char *buf, unsigned int
len);
77
78 #endif /* NETLIB_H */
79

```

NETLIB.C

```

1  /* netlib.c -- the five N: commands. See netlib.h.
2  *
3  * The one thing worth saying that is not in fujilib: the
status reply, and the
4  * read reply, both land in the reply window and are only
valid until the next
5  * transaction. NETCAT's session loop is written around
that -- status, then
6  * read, then draw, with nothing in between that would
repaint the window.
7  */
8
9  #include <stdint.h>
10
11 #include "fujilib.h"
12 #include "netlib.h"
13
14 bool net_present(void)
15 {
16     return fn_present();
17 }
18
19 bool net_open(const char *spec)
20 {
21     fn_start(NET_DEV_N1, NET_OPEN);
22     fn_param8(NET_MODE_READWRITE);
23     fn_param8(NET_TRANS_NONE);
24     fn_tx_padded(spec, NET_SPEC_LEN);
25     return fn_commit() == FN_OK && fn_acked();
26 }
27
28 void net_close(void)
29 {
30     fn_start(NET_DEV_N1, NET_CLOSE);
31     (void)fn_commit(); /* best effort */
32 }
33
34 bool net_status(unsigned int *avail, unsigned char *conn,
35                 unsigned char *devstatus)
36 {
37     volatile unsigned char *r;
38
39     fn_start(NET_DEV_N1, NET_STATUS);
40     /* The RS232 build reads the request type from
parameter 1 and wants both
41     * present; parameter 0 is legacy and ignored. Two
zeroes ask for the
42     * open channel's own status. */
43     fn_param8(0);
44     fn_param8(0);
45     if (fn_commit() != FN_OK || !fn_acked())
46         return false;
47
48     /* Four bytes: avail lo, avail hi, connected, device
status. Read through a
49     * local pointer -- never FN_REPLY[i] -- for the
sccz80 reason in fujilib. */
50     r = FN_REPLY;
51     *avail = (unsigned int)r[0] | ((unsigned int)r[1] <<
8);
52     *conn = r[2];
53     *devstatus = r[3];
54     return true;
55 }
56
57 unsigned int net_read(unsigned int want)
58 {
59     if (want > NET_READ_MAX)
60         want = NET_READ_MAX;
61
62     fn_start(NET_DEV_N1, NET_READ);
63     fn_param16(want);
64     if (fn_commit() != FN_OK || !fn_acked())
65         return 0;
66
67     /* The firmware always sends exactly `want` bytes on
ACK, so this equals
68     * what we asked for; capture it from the transaction
rather than trusting
69     * the argument, because a NAK read comes back short.
*/
70     return fn_reply_len();
71 }
72
73 bool net_write(const unsigned char *buf, unsigned int
len)
74 {
75     if (len > NET_WRITE_MAX)
76         len = NET_WRITE_MAX;
77
78     fn_start(NET_DEV_N1, NET_WRITE);
79     fn_param16(len);
80     fn_tx_bytes(buf, len);
81     return fn_commit() == FN_OK && fn_acked();
82 }
83

```

NETTERM.H -- THE SCROLLING PANE

```

1  /* netterm.h -- a scrolling text pane.
2  *
3  * The TMS9918 in Graphics 1 has no hardware scroll, so a terminal has to move
4  * its own pixels: when the cursor falls off the bottom, every row is copied up
5  * one and the last is cleared. That is a VRAM-to-VRAM block move a row at a
6  * time, done through OS7's read_vram / write_vram (the name table is at 0x1800
7  * in the layout mode_1 sets up).
8  *
9  * There is no RAM shadow of the pane -- a ColecoVision does not have the bytes
10 * for one. When the on-screen keyboard has to borrow the screen to compose a
11 * line, the pane is saved to spare VRAM and restored afterwards, so the
12 * conversation survives the interruption without ever occupying console RAM.
13 */
14
15 #ifndef NETTERM_H
16 #define NETTERM_H
17
18 /* The pane: rows TERM_TOP through TERM_BOT of the 32x24 screen, full width.
19 * Row 0 is the title, row 23 the key legend. */
20 #define TERM_TOP    2
21 #define TERM_BOT    21
22 #define TERM_ROWS   (TERM_BOT - TERM_TOP + 1)
23 #define TERM_COLS   32
24
25 /* Clear the pane and home the cursor. */
26 void term_init(void);
27
28 /* Put one character. Handles CR (column 0), LF (next line, scrolling at the
29 * bottom), backspace, and printable ASCII; anything else is shown as a dot so
30 * a control-heavy stream still looks like something. */
31 void term_putc(char c);
32
33 /* Put '\n' bytes -- one whole NET_READ, straight out of the reply window, which
34 * is why the argument is volatile. */
35 void term_puts(volatile unsigned char *p, unsigned int n);
36
37 /* Save the pane to spare VRAM before the on-screen keyboard takes the screen,
38 * and put it back after. */
39 void term_save(void);
40 void term_restore(void);
41
42 #endif /* NETTERM_H */
43

```

NETTERM.C

```

1  #include <os7.h>
2
3  #include "fujidisp.h"
4  #include "netterm.h"
5
6  /* The name table's VRAM address in the layout mode_1()
7  * lays down: 32x24 bytes
8  * at 0x1800. A row is 32 bytes at 0x1800 + row*32. */
9  #define NT_BASE    0x1800
10
11 /* Where the pane is stashed while the keyboard borrows
12 * the screen. The colour
13 * table is 32 bytes at 0x2000 and the sprite generator
14 * does not start until
15 * 0x3800, so 0x2800 is free VRAM the display code never
16 * touches. */
17 #define SAVE_BASE  0x2800
18
19 static unsigned char col;      /* 0..TERM_COLS-1 */
20 static unsigned char row;      /* TERM_TOP..TERM_BOT */
21 static unsigned char line[TERM_COLS];
22
23 void term_init(void)
24 {
25     unsigned char r;
26
27     for (r = TERM_TOP; r <= TERM_BOT; r++)
28         disp_row_clear(r);
29     col = 0;
30     row = TERM_TOP;
31 }
32
33 /* Copy the pane up one row and clear the bottom.
34 * read_vram/write_vram take raw
35 * VRAM addresses; a row at a time keeps every count well
36 * under the 8-bit
37 * truncation that bites larger put_vram calls. */
38 static void scroll(void)
39 {
40     unsigned char r;
41
42     for (r = TERM_TOP; r < TERM_BOT; r++) {
43         read_vram(line, (unsigned short)(NT_BASE + (r +
44         1) * TERM_COLS),
45                 TERM_COLS);
46         write_vram(line, (unsigned short)(NT_BASE + r *
47         TERM_COLS), TERM_COLS);
48     }
49     disp_row_clear(TERM_BOT);
50 }
51
52 static void newline(void)
53 {
54     col = 0;
55     if (row < TERM_BOT)
56         row++;
57     else
58         scroll();
59 }
60
61 void term_putc(char c)
62 {
63     if (c == '\r') {
64         col = 0;
65         return;
66     }
67     if (c == '\n') {
68         newline();
69         return;
70     }
71     if (c == '\b') {
72         if (col > 0)
73             col--;
74         disp_char(col, row, ' ');
75         return;
76     }
77     if (c < 0x20 || c > 0x7E)
78         c = '.';
79     /* keep control bytes

```

```

visible, not disruptive */
71
72     disp_char(col, row, c);
73     if (++col >= TERM_COLS)
74         newline();
75 }
76
77 void term_puts(volatile unsigned char *p, unsigned int n)
78 {
79     while (n--)
80         term_putc((char)*p++);
81 }
82
83 void term_save(void)
84 {
85     unsigned char r;
86
87     for (r = 0; r < 24; r++) {
88         read_vram(line, (unsigned short)(NT_BASE + r *
TERM_COLS), TERM_COLS);
89         write_vram(line, (unsigned short)(SAVE_BASE + r *
TERM_COLS), TERM_COLS);
90     }
91 }
92
93 void term_restore(void)
94 {
95     unsigned char r;
96
97     for (r = 0; r < 24; r++) {
98         read_vram(line, (unsigned short)(SAVE_BASE + r *
TERM_COLS), TERM_COLS);
99         write_vram(line, (unsigned short)(NT_BASE + r *
TERM_COLS), TERM_COLS);
100     }
101 }
102

```

NETCAT.C

```

1  /* netcat.c -- a terminal for the FujiNet N: device.
2  *
3  * Give it any N: devicespec and it pumps bytes between
the connection and a
4  * scrolling pane, forever: call a telnet BBS, watch a
TCP echo server, talk to
5  * anything that talks back. It is the network-device
counterpart to fujitest --
6  * the smallest program that exercises N: open/status/
read/write/close end to
7  * end -- and the one client in this handbook that holds
a single connection
8  * open for its whole life rather than opening one per
request.
9  *
10 * The session loop is three moves in a fixed order:
11 *
12 * 1. NET_STATUS -- how many bytes wait, and are we
still connected;
13 * 2. NET_READ -- pull them into the reply window
(if any);
14 * 3. draw them -- straight out of the window, before
anything else runs.
15 *
16 * Nothing may fall between the read and the draw: every
transaction repaints
17 * the reply window, so a stray status poll there would
erase the bytes we were
18 * about to show. That is the whole discipline; the rest
is a keyboard.
19 */
20
21 #include <os7.h>
22
23 #include "fujidisp.h"
24 #include "fujilin.h"
25 #include "fujiedit.h"
26 #include "fujisnd.h"
27 #include "fujilib.h"
28 #include "netlib.h"
29 #include "netterm.h"
30
31 /* The devicespec NETCAT dials unless you edit it. A
telnet BBS makes the best
32 * demonstration: it answers in ASCII, in lowercase, and
it scrolls. */
33 static char spec[NET_SPEC_LEN] = "N:TELNET://bbs.
fozztexx.com/";
34
35 static void title(const char *s)
36 {
37     disp_row_clear(0);
38     disp_at(1, 0, s);
39 }
40
41 static void legend(const char *s)
42 {
43     disp_row_clear(23);
44     disp_at(1, 23, s);
45 }
46
47 /* The dial screen: edit the devicespec on the on-screen
keyboard, then open
48 * it. Returns when a connection is live. fn_edit
preloads and reads back
49 * fn_entry, so the spec round-trips through the 64-byte
editor buffer -- which
50 * is why a devicespec here cannot exceed FN_ENTRY_MAX-1.
That is plenty for a
51 * host and path; the 256-byte NET_OPEN buffer is the
firmware's limit, not the
52 * user's. */
53 static void dial(void)
54 {
55     unsigned int i;
56
57     for (;;) {
58         /* Copy the current spec into the editor buffer,
edit, copy back. */
59         for (i = 0; i < FN_ENTRY_MAX - 1 && spec[i]; i++)
60             fn_entry[i] = spec[i];
61         fn_entry[i] = '\0';
62
63         if (fn_edit("DIAL N: DEVICESPEC", FN_ENTRY_MAX -
1)) {
64             for (i = 0; i < FN_ENTRY_MAX - 1 &&
fn_entry[i]; i++)
65                 spec[i] = fn_entry[i];
66             spec[i] = '\0';
67         }
68
69         disp_cls();
70         title("NETCAT");
71         disp_at(1, TERM_TOP, "DIALING...");
72
73         if (net_open(spec))
74             return;
75
76         disp_at(1, TERM_TOP + 1, "CONNECT FAILED");
77         disp_at(1, TERM_TOP + 3, "FIRE TO EDIT AND
RETRY");
78         while (in_read() != IN_FIRE)
79             ;
80     }
81 }
82
83 /* Compose a line on the on-screen keyboard and send it
with a trailing CR.
84 * The pane is saved first because fn_edit clears the
screen, and restored

```

```

85  * after so the conversation is unbroken. fn_edit runs no
transactions, so the
86  * connection is undisturbed by the detour. */
87  static void compose_and_send(void)
88  {
89      unsigned char n;
90
91      term_save();
92      fn_entry[0] = '\0';
93      if (fn_edit("SEND (# sends)", NET_WRITE_MAX - 1)) {
94          for (n = 0; fn_entry[n]; n++)
95              ;
96          fn_entry[n++] = '\r'; /* the line terminator
most hosts expect */
97          term_restore();
98          net_write((const unsigned char *)fn_entry, n);
99      } else {
100          term_restore();
101      }
102  }
103
104  static void session(void)
105  {
106      unsigned int avail, got;
107      unsigned char conn, ds, ev;
108
109      term_init();
110      title("NETCAT CONNECTED");
111      legend("# SEND * HANG UP");
112
113      for (;;) {
114          if (!net_status(&avail, &conn, &ds)) {
115              title("NETCAT LINK LOST");
116              return;
117          }
118          if (!conn && avail == 0) {
119              title("NETCAT DISCONNECTED");
120              return;
121          }
122
123          if (avail != 0) {
124              got = net_read(avail > NET_READ_MAX ?
NET_READ_MAX : avail);
125              /* Draw immediately, straight out of the
reply window, before any
126               * other transaction can repaint it. */
127              term_puts(FN_REPLY, got);
128          }
129
130          ev = in_read();
131          if (ev == IN_KEYHASH || ev == IN_FIRE)
132              compose_and_send();
133          else if (ev == IN_KEYSTAR) {
134              net_close();
135              title("NETCAT HUNG UP");
136              return;
137          }
138      }
139  }
140
141  void main(void)
142  {
143      snd_init(); /* the PSG powers up
buzzing; see fujisnd.h */
144      disp_init();
145      in_init();
146
147      if (!net_present()) {
148          disp_at(4, 8, "NO FUJINET CART");
149          for (;;)
150              ;
151      }
152
153      for (;;) {
154          dial();
155          session();
156      }
157  }
158

```

BUILD.SH

```

1  #!/usr/bin/env bash
2  # build.sh -- assemble NETCAT into a 32768-byte
ColecoVision cartridge.
3  #
4  # ./build.sh
5  #
6  # This is the firmware testrom's build recipe (pico/
coleco/build.sh), narrowed
7  # to one client. NETCAT builds in C with z88dk's +coleco
target against the
8  # user's own OS7 binding library (os7lib) and the shared
FujiNet client library
9  # from ../common. The link line's three FujiNet-specific

```

additions are:

```

10 # -Cz--romsize/--rombase force exactly 32768 bytes
based at $8000;
11 # --code-fence/--data-fence warn if anything reaches
the mailbox pages;
12 # CLIB_DEFAULT_SCREEN_MODE=-1 keep the crt0's VDP init
out of OS7's way.
13 #
14 # Environment overrides:
15 # Z88DK z88dk root (default ~/Workspace/z88dk)
16 # OS7LIB os7lib checkout (default ~/Workspace/os7lib)
17
18 set -euo pipefail

```

```

19 cd "$(dirname "$0")"
20
21 Z88DK=${Z88DK:-$HOME/Workspace/z88dk}
22 OS7LIB=${OS7LIB:-$HOME/Workspace/os7lib}
23 export ZCCCFG="$Z88DK/lib/config"
24 export PATH="$Z88DK/bin:$PATH"
25
26 WINDOW=32768
27 FENCE=0xF800
28 COMMON=../common
29
30 [ -x "$Z88DK/bin/zcc" ] || { echo "no zcc at $Z88DK/bin/
zcc" >&2; exit 1; }
31 [ -f "$OS7LIB/os7.lib" ] || { echo "no os7.lib at
$OS7LIB" >&2; exit 1; }
32
33 mkdir -p build
34
35 CFLAGS="+colec -O2 -I$OS7LIB/src -I$COMMON -I."
36 LDFLAGS="+colec -m -s \
37 -pragma-define:CRT_ORG_BSS=0x702C \
38 -pragma-define:CRT_COLECO_SPRITE_NAME_SIZE=128 \
39 -pragma-define:CRT_COLECO_SPRITE_ORDER_SIZE=32 \
40 -pragma-define:CRT_COLECO_BIOS_BUFFER_SIZE=32 \
41 -pragma-define:CRT_COLECO_BIOS_CONTROLLER_SIZE=12 \
42 -pragma-define:CRT_ENABLE_STDIO=0 \
43 -pragma-define:REGISTER_SP=0x73B8 \
44 -pragma-define:CLIB_FOPEN_MAX=0 \
45 -pragma-define:fputc_cons=0 \
46 -pragma-define:CLIB_DEFAULT_SCREEN_MODE=-1 \
47 -Cz--romsize=$WINDOW -Cz--rombase=$WINDOW \
48 -Cz--code-fence=$FENCE -Cz--data-fence=$FENCE \
49 -L$OS7LIB -los7"
50
51 SHARED="$COMMON/fujilib.c $COMMON/fujidisp.c $COMMON/
fujin.c \
52 $COMMON/fujiedit.c $COMMON/fujisnd.c"
53
54 rm -f build/netcat.bin
55 # shellcheck disable=SC2086
56 zcc $CFLAGS $LDFLAGS netcat.c netlib.c netterm.c $SHARED
\
57 -o build/netcat -create-app
58 mv build/netcat.rom build/netcat.bin 2>/dev/null || mv
build/netcat build/netcat.bin
59
60 # Stamp the "FUJI" claim at offset 0x7CFC and check the
layout. checkrom.py
61 # lives in the firmware tree; point CHECKROM at it, or
drop a copy beside this.
62 CHECKROM=${CHECKROM:-$HOME/Workspace/fujinet-firmware/
pico/coleco/tools/checkrom.py}
63 if [ -f "$CHECKROM" ]; then
64     python3 "$CHECKROM" --stamp build/netcat.bin
65 else
66     echo "warning: no checkrom.py at $CHECKROM -- image
is unstamped" >&2
67 fi
68
69 echo "built build/netcat.bin"
70

```

C.4 The milestone clients

HELLO.C -- M0: TOOLCHAIN AND DISPLAY

```

1 /* hello.c -- milestone 0: prove the toolchain, the cartridge header and the
2 * display layer, with no mailbox involved at all.
3 *
4 * If this shows up in MAME the z88dk +colec link line is right, os7lib is
5 * linked, the image is a valid 32K cartridge, and build.sh's padding and claim
6 * stamp did not break the header. That is the whole job.
7 */
8
9 #include <os7.h>
10
11 #include "fujidisp.h"
12 #include "fujisnd.h"
13
14 void main(void)
15 {
16     snd_init(); /* the PSG powers up buzzing; see fujisnd.h */
17     disp_init();
18     disp_at(6, 8, "FUJINET COLECOVISION");
19     disp_at(6, 10, "HELLO FROM OS7");
20     disp_at(6, 12, "MILESTONE 0");
21
22     for (;;)
23     ;
24 }
25

```

FUJITEST.C -- M1: ONE ROUND TRIP

```

1  /* fujitest.c -- milestone 1: one real FujiBus round
trip.
2  *
3  * Asks the ESP32 for its adapter config and puts the
live SSID, IP address and
4  * firmware version on screen. If those are right they
came off an actual
5  * network stack, through USB CDC, through the
cartridge's mailbox and back out
6  * of repainted cartridge ROM -- there is nothing in the
client that could have
7  * invented them.
8  *
9  * It doubles as the hardware diagnostic: with no
cartridge underneath, the
10 * magic bytes read as whatever a plain ROM has there and
the first line says
11 * NO FUJINET CART rather than hanging.
12 *
13 * Reset survival is the second thing this proves, and
the subtler one. Press
14 * RESET and the sequence number must come back HIGHER,
with a fresh `Fuji cmd:`
15 * on the ESP32 side -- not the same number replaying the
same stale reply. The
16 * number is on screen for exactly that reason.
17 */
18
19 #include <os7.h>
20 #include <string.h>
21
22 #include "fujidisp.h"
23 #include "fujisnd.h"
24 #include "fujilib.h"
25
26 #define CMD_GET_ADAPTERCONFIG_EXTENDED 0xC4
27
28 /* AdapterConfigExtended, from lib/device/fujiDevice/
fujiDevice.h. */
29 #define ACX_SSID 0
30 #define ACX_VERSION 125
31 #define ACX_SLOCALIP 140
32
33 static char buf[34];
34
35 static void show_field(unsigned char row, const char
*label,
36                      unsigned int off, unsigned int
max)
37 {
38     unsigned int i;
39
40     for (i = 0; i < max; i++) {
41         char c = (char)FN_REPLY[off + i];
42
43         if (c == '\0')
44             break;
45         buf[i] = (c < 32 || c > 126) ? ' ' : c;
46     }
47     buf[i] = '\0';
48     disp_row_clear(row);
49     disp_at(1, row, label);
50     disp_at(9, row, buf);
51 }
52
53 void main(void)
54 {
55     unsigned char err;
56
57     snd_init(); /* the PSG powers up buzzing; see
fujisnd.h */
58     disp_init();
59     disp_at(4, 1, "FUJINET COLECOVISION TEST");
60
61     if (!fn_present()) {
62         disp_at(4, 4, "NO FUJINET CART");
63         for (;;)
64             ;
65     }
66
67     disp_at(1, 3, "PROTO");
68     disp_at_u16(9, 3, FN_PROTOVER);
69     disp_at(1, 5, "ASKING...");
70
71     fn_start(FN_DEV_FUJINET,
CMD_GET_ADAPTERCONFIG_EXTENDED);
72     err = fn_commit();
73
74     disp_row_clear(5);
75     if (err != FN_OK) {
76         disp_at(1, 5, "ERR");
77         disp_at_hex8(9, 5, err);
78         for (;;)
79             ;
80     }
81     if (!fn_acked()) {
82         disp_at(1, 5, "NAK");
83         for (;;)
84             ;
85     }
86
87     show_field(7, "SSID", ACX_SSID, 32);
88     show_field(9, "IP", ACX_SLOCALIP, 15);
89     show_field(11, "FW", ACX_VERSION, 14);
90
91     disp_at(1, 14, "REPLY");
92     disp_at_u16(9, 14, fn_reply_len());
93
94     /* The reset-survival readout: this is the
cartridge's own persisted
95     * counter, not ours. It must be strictly larger
after every RESET. */
96     disp_at(1, 16, "SEQ");
97     disp_at_u16(9, 16, FN_ACKSEQ);
98
99     for (;;)
100         ;
101 }
102

```

FUJIBOOT.C -- M2: NETWORK BOOT

```

1  /* fujiboot.c -- milestone 2: mount a cartridge image
over the network and boot
2  * it.
3  *
4  * MOUNT_HOST -> SET_DEVICE_FULLPATH -> MOUNT_IMAGE, and
then the interesting
5  * part: the ESP32 streams the image straight to the
cartridge as an unsolicited
6  * DBC push while our MOUNT_IMAGE is still outstanding.
The cartridge stages it,
7  * publishes progress, and says READY; we arm the swap,
run a stub out of RAM,
8  * and the ColecoVision comes back up running a
completely different cartridge.
9  *
10 * Host and path come from build/bootcfg.h, which
build.sh regenerates from
11 * $BOOT_HOST / $BOOT_PATH on every build.
12 */
13
14 #include <os7.h>
15
16 #include "fujidisp.h"
17 #include "fujisnd.h"
18 #include "fujilib.h"
19 #include "bootcfg.h"
20
21 #define CMD_MOUNT_HOST      0xF9
22 #define CMD_MOUNT_IMAGE    0xF8
23 #define CMD_SET_DEVICE_FULLPATH 0xE2
24
25 #define DEVSLOT      0
26 #define MODE_READ 1
27
28 /* SET_DEVICE_FULLPATH takes a fixed 256-byte buffer; a
short payload is
29 * rejected on the ESP32 side. */
30 #define PATH_LEN 256
31
32 static void die(const char *what, unsigned char code)
33 {
34     disp_row_clear(10);
35     disp_at(1, 10, what);
36     disp_at_hex8(20, 10, code);
37     for (;;)
38         ;
39 }
40
41 static void step(const char *what)
42 {
43     disp_row_clear(8);
44     disp_at(1, 8, what);
45 }
46
47 /* Every transaction is "commit, then insist on an ACK":
a FujiBus NAK is a
48 * perfectly successful round trip that happens to mean
no. */
49 static void need_ack(const char *what)
50 {
51     unsigned char err = fn_commit();
52
53     if (err != FN_OK)
54         die(what, err);
55     if (!fn_acked())
56         die(what, FN_NAK);
57 }
58
59 void main(void)
60 {
61     unsigned char pct = 0xFF;
62
63     snd_init(); /* the PSG powers up buzzing; see
fujisnd.h */
64     disp_init();
65     disp_at(4, 1, "FUJINET COLECOVISION BOOT");
66
67     if (!fn_present())
68         die("NO CART", 0);
69
70     disp_at(1, 4, "HOST");
71     disp_at_u16(9, 4, BOOT_HOST);
72     disp_at(1, 5, "PATH");
73     disp_at(9, 5, BOOT_PATH);
74
75     step("MOUNT HOST");
76     fn_start(FN_DEV_FUJINET, CMD_MOUNT_HOST);
77     fn_param8(BOOT_HOST);
78     need_ack("EHOST");
79
80     step("SET PATH");
81     fn_start(FN_DEV_FUJINET, CMD_SET_DEVICE_FULLPATH);
82     fn_param8(DEVSLOT);
83     fn_param8(BOOT_HOST);
84     fn_param8(MODE_READ);
85     fn_tx_padded(BOOT_PATH, PATH_LEN);
86     need_ack("EPATH");
87
88     step("MOUNT IMAGE");
89     fn_start(FN_DEV_FUJINET, CMD_MOUNT_IMAGE);
90     fn_param8(DEVSLOT);
91     fn_param8(MODE_READ);
92     need_ack("EMOUNT");
93
94     /* The push is asynchronous: MOUNT_IMAGE has already
been answered, and the
95     * image arrives afterwards frame by frame. Watch the
cartridge's own
96     * progress counter rather than guessing at a delay.
*/
97     step("LOADING");
98     for (;;) {
99         unsigned char st = FN_BOOTSTAT;
100
101         if (st == FNB_READY)
102             break;
103         if (st == FNB_FAILED)
104             die("ELOAD", FN_BOOTERR);
105         if (FN_BOOTPCT != pct) {
106             pct = FN_BOOTPCT;
107             disp_at_u16(14, 8, pct);
108         }
109     }
110
111     step("BOOTING");
112     fn_boot_swap(); /* does not return: the cartridge
changes underneath */
113 }
114

```

C.5 CONFIG

FUJICFG.C -- THE BROWSE-AND-BOOT CLIENT

```

1  /* fujicfg.c -- milestone 3: the CONFIG browser.
2  *
3  * Pick a host, walk its root directory, boot what you
4  * land on. The stand-in
5  * for the real CONFIG (which will live in ~/Workspace/
6  * fujinet-config/coleco/);
7  * this one exists to prove the browse-and-boot core
8  * against a live FujiNet,
9  * and to be driven headlessly by emu/drive.lua.
10 *
11 * Everything on screen goes through OS7 -- mode_1,
12 * load_ascii, put_vram -- and
13 * everything from the controller comes back through
14 * OS7's POLLER. What makes
15 * that possible at all on a machine with ~700 bytes of
16 * usable RAM is that no
17 * filename is ever copied into it: names are displayed
18 * straight out of the
19 * cartridge's reply window and streamed straight back
20 * into the next
21 * transaction's payload (see fn_tx_path).
22 *
23 * Not here, on purpose: descending into subdirectories,
24 * host renaming, disk
25 * slots. Those are the real CONFIG's job -- this is the
26 * browse core.
27 */
28 #include <os7.h>
29
30 #include "fujidisp.h"
31 #include "fujisnd.h"
32 #include "fuiin.h"
33 #include "fujisplash.h"
34 #include "fujiedit.h"
35 #include "fujilib.h"
36
37 #define CMD_READ_HOST_SLOTS      0xF4
38 #define CMD_WRITE_HOST_SLOTS    0xF3
39 #define CMD_MOUNT_HOST          0xF9
40 #define CMD_OPEN_DIRECTORY      0xF7
41 #define CMD_READ_DIR_ENTRY      0xF6
42 #define CMD_CLOSE_DIRECTORY     0xF5
43 #define CMD_SET_DIRECTORY_POSITION 0xE4
44 #define CMD_SET_DEVICE_FULLPATH 0xE2
45 #define CMD_MOUNT_IMAGE         0xF8
46 #define CMD_ADAPTERCONFIG_EXT   0xC4
47 #define CMD_SCAN_NETWORKS       0xFD
48 #define CMD_GET_SCAN_RESULT     0xFC
49 #define CMD_SET_SSID            0xFB
50
51 #define HOST_SLOTS 8
52 #define HOST_STRIDE 32 /* READ_HOST_SLOTS: 8 x
53 32 bytes */
54
55 #define LIST_TOP 5 /* first list row on
56 screen */
57 #define LIST_ROWS 16
58 #define NAMELEN 29 /* display width: col
59 2..30 */
60 #define FULLLEN 120 /* re-read width when
61 building a boot path */
62 #define PATH_LEN 256 /* the fixed buffer the
63 ESP32 expects */
64
65 #define DEV_SLOT 0
66 #define MODE_READ 1
67
68 #define PAGE_HOSTS 0
69 #define PAGE_FILES 1
70 #define PAGE_CONFIG 2
71
72 /* AdapterConfigExtended field offsets, from lib/device/
73 fujiDevice/fujiDevice.h */
74 #define ACX_SSID 0
75 #define ACX_VERSION 125
76 #define ACX_SLOCALIP 140
77 #define ACX_SMAC 204
78
79 /* SSIDInfo: char ssid[33]; uint8_t rssi -- 34 bytes, one
80 per GET_SCAN_RESULT */
81 #define SSID_LEN 33
82 #define SSID_STRIDE 34
83
84 #define CFG_TOP 11 /* first network row */
85 #define CFG_ROWS 8
86 #define PASS_MAX 63
87
88 static unsigned char page;
89 static unsigned char host;
90
91 static unsigned char cur; /* cursor row within the
92 visible window */
93 static unsigned int top; /* index of the first
94 visible entry */
95 static unsigned char nrows; /* rows actually filled
96 */
97 static unsigned char at_end; /* the last row on screen
98 is the last entry */
99
100 static void status(const char *s)
101 {
102     disp_row_clear(22);
103     disp_at(1, 22, s);
104 }
105
106 static void oops(const char *what, unsigned char code)
107 {
108     disp_row_clear(22);
109     disp_at(1, 22, what);
110     disp_at_hex8((unsigned char)(28), 22, code);
111 }
112
113 /* Commit and insist on an ACK. Returns 0 on success; on
114 failure it has already
115 * said so on the status line, so callers just bail out
116 to the redraw. */
117 static unsigned char need_ack(const char *what)
118 {
119     unsigned char err = fn_commit();
120     if (err != FN_OK) {
121         oops(what, err);
122         return err;
123     }
124     if (!fn_acked()) {
125         oops(what, FN_NAK);
126         return FN_NAK;
127     }
128     return 0;
129 }
130
131 static void draw_frame(void)
132 {
133     disp_cls();
134     disp_at(1, 1, "FUJINET C O N F I G");
135 }
136
137 /* The selection bar. Always applied AFTER a row's text
138 is drawn, because it
139 * works by re-pointing the characters already in the
140 name table. */
141 static void draw_cursor(unsigned char row, bool on)
142 {
143     unsigned char top = (page == PAGE_CONFIG) ? CFG_TOP :
144 LIST_TOP;
145     disp_row_invert((unsigned char)(top + row), on);
146 }
147
148 /* ---- host page ---- */
149
150 static void host_page(void)
151 {
152     unsigned char i;
153
154     draw_frame();
155     disp_at(1, 3, "SELECT A HOST");
156     status("FIRE OPEN 9 RENAME # CONFIG");
157
158     fn_start(FN_DEV_FUJINET, CMD_READ_HOST_SLOTS);
159     if (need_ack("EHOSTS") != 0) {
160         nrows = 0;
161         return;
162     }
163
164     nrows = 0;
165     for (i = 0; i < HOST_SLOTS; i++) {
166         volatile unsigned char *name = FN_REPLY +
167 (unsigned int)i * HOST_STRIDE;
168         unsigned char row = (unsigned char)(LIST_TOP +
169 i);
170         unsigned char col;
171
172         disp_row_clear(row);
173         disp_at_u16(1, row, (unsigned int)(i + 1));
174         if (*name == 0) {
175             disp_at(3, row, "(empty)");
176             continue;
177         }
178         /* Straight out of the reply window, a character

```

```

at a time -- there is
153      * nowhere in RAM to put it and no reason to. */
154      for (col = 0; col < HOST_STRIDE && name[col] !=
0; col++)
155          disp_char((unsigned char)(3 + col), row,
(char)name[col]);
156          nrows = (unsigned char)(i + 1);
157      }
158      if (nrows == 0)
159          nrows = HOST_SLOTS; /* still navigable, just
all empty */
160      at_end = 1;
161      if (cur >= nrows)
162          cur = 0;
163      draw_cursor(cur, true);
164  }
165
166  /* ---- configuration page ---- */
167
168  /* Draw a label and a NUL-terminated field out of the
reply window. Always via
169  * a local pointer -- sccz80 mis-indexes the FN_REPLY
macro's cast constant. */
170  static void draw_field(unsigned char row, const char
*label,
171                        unsigned int off, unsigned int
max)
172  {
173      volatile unsigned char *r = FN_REPLY + off;
174      unsigned int i;
175
176      disp_row_clear(row);
177      disp_at(1, row, label);
178      for (i = 0; i < max; i++) {
179          char c = (char)r[i];
180
181          if (c == '\0')
182              break;
183          disp_char((unsigned char)(10 + i), row, c);
184      }
185  }
186
187  /* RSSI arrives as a uint8_t but means dBm, so it has to
be read back as
188  * signed or every network looks like a very strong +200.
*/
189  static void draw_rssi(unsigned char row, unsigned char
rrow)
190  {
191      signed char dbm = (signed char)row;
192      unsigned int mag = (unsigned int)(dbm < 0 ? -dbm :
dbm);
193
194      disp_at(26, row, dbm < 0 ? "-" : " ");
195      disp_at_ul6(27, row, mag);
196  }
197
198  static void config_page(void)
199  {
200      volatile unsigned char *r;
201      unsigned char i;
202
203      draw_frame();
204      status("SCANNING...");
205
206      /* Adapter info first, and drawn completely before
anything else runs: the
207      * scan below repaints the reply window this is being
read out of. */
208      fn_start(FN_DEV_FUJINET, CMD_ADAPTERCONFIG_EXT);
209      if (need_ack("EINFO") != 0) {
210          nrows = 0;
211          return;
212      }
213      draw_field(3, "SSID", ACX_SSID, 32);
214      draw_field(4, "IP", ACX_SLOCALIP, 15);
215      draw_field(5, "MAC", ACX_SMAC, 17);
216      draw_field(6, "FW", ACX_VERSION, 14);
217
218      disp_at(1, 9, "NETWORKS");
219
220      fn_start(FN_DEV_FUJINET, CMD_SCAN_NETWORKS);
221      if (need_ack("ESCAN") != 0) {
222          nrows = 0;
223          return;
224      }
225      r = FN_REPLY;
226      nrows = r[0] > CFG_ROWS ? CFG_ROWS : r[0];
227
228      for (i = 0; i < nrows; i++) {
229          unsigned char row = (unsigned char)(CFG_TOP + i);
230          unsigned char col;
231
232          fn_start(FN_DEV_FUJINET, CMD_GET_SCAN_RESULT);
233          fn_param8(i);
234          if (fn_commit() != FN_OK || !fn_acked())
235              break;
236          r = FN_REPLY;
237
238          disp_row_clear(row);
239          for (col = 0; col < 24 && r[col] != 0; col++)
240              disp_char((unsigned char)(2 + col), row,
(char)r[col]);
241          draw_rssi(row, r[SSID_LEN]);
242      }
243
244      status("FIRE CONNECT * BACK");
245      if (nrows == 0) {
246          disp_at(2, CFG_TOP, "(none found)");
247          return;
248      }
249      cur = 0;
250      at_end = 1;
251      draw_cursor(cur, true);
252
253      /* Join the highlighted network. SET_SSID wants nparam >=
1 (the value is
254      * ignored) and a payload of exactly ssid[33] +
password[64]. The SSID is
255      * re-fetched so it is sitting in the reply window, and
the editor runs no
256      * transactions, so it is still there afterwards to
stream straight back. */
257      static void connect_network(void)
258      {
259          fn_start(FN_DEV_FUJINET, CMD_GET_SCAN_RESULT);
260          fn_param8(cur);
261          if (need_ack("ESCAN") != 0)
262              return;
263
264          fn_entry[0] = '\0';
265          if (!fn_edit("WIFI PASSWORD", PASS_MAX)) {
266              config_page();
267              return;
268          }
269
270          status("CONNECTING...");
271          fn_start(FN_DEV_FUJINET, CMD_SET_SSID);
272          fn_param8(0);
273          fn_tx_from_reply(0, SSID_LEN);
274          fn_tx_padded(fn_entry, 64);
275          if (need_ack("ESSID") != 0)
276              return;
277
278          config_page();
279      }
280
281      /* Rename the highlighted host slot.
282      *
283      * WRITE_HOST_SLOTS takes no parameters and one 256-byte
payload -- all eight
284      * 32-byte slots, every time; there is no per-slot write.
Rather than mirror
285      * 256 bytes into a machine with under a kilobyte of RAM,
re-read the slots and
286      * stream them straight back out of the reply window with
the edited one
287      * substituted. The editor runs no transactions, so the
window it was called
288      * with would still be intact; the re-read is one round
trip that removes the
289      * need to reason about that at all. */
290      static void rename_host(void)
291      {
292          volatile unsigned char *name;
293          unsigned char i;
294
295          fn_start(FN_DEV_FUJINET, CMD_READ_HOST_SLOTS);
296          if (need_ack("EHOSTS") != 0)
297              return;
298
299          name = FN_REPLY + (unsigned int)cur * HOST_STRIDE;
300          for (i = 0; i < HOST_STRIDE - 1 && name[i] != 0; i++)
301              fn_entry[i] = (char)name[i];
302          fn_entry[i] = '\0';
303
304          if (!fn_edit("EDIT HOST NAME", HOST_STRIDE - 1)) {
305              host_page(); /* cancel leaves the slot
untouched */
306              return;
307          }
308
309          fn_start(FN_DEV_FUJINET, CMD_READ_HOST_SLOTS);
310          if (need_ack("EHOSTS") != 0)
311              return;
312
313          fn_start(FN_DEV_FUJINET, CMD_WRITE_HOST_SLOTS);
314          for (i = 0; i < HOST_SLOTS; i++) {
315              if (i == cur)
316                  fn_tx_padded(fn_entry, HOST_STRIDE);
317              else
318                  fn_tx_from_reply((unsigned int)i *
HOST_STRIDE, HOST_STRIDE);
319          }
320          if (need_ack("EWRITE") != 0)
321              return;

```

```

322     host_page();
323 }
324 }
325
326 /* ---- file page ---- */
327
328 /* Read one directory entry into the reply window.
Returns false at end of
329 * directory or on any error. maxlen is the crunch width
the ESP32 applies --
330 * ask for the display width when drawing and the full
width when booting,
331 * because READ_DIR_ENTRY really does truncate to what
you asked for. */
332 static bool read_entry(unsigned char maxlen)
333 {
334     volatile unsigned char *r;
335     unsigned char b0, b1, b2;
336
337     fn_start(FN_DEV_FUJINET, CMD_READ_DIR_ENTRY);
338     fn_param8(maxlen);
339     fn_param8(0);
340     if (fn_commit() != FN_OK || !fn_acked())
341         return false;
342
343     /* Read through a local pointer, never `FN_REPLY[i]`
directly. sccz80
344     * indexes the macro's cast constant differently from
a pointer variable,
345     * and gets it wrong: the same expression that draws
a filename correctly
346     * from a `volatile unsigned char *` local reads back
zero when it is
347     * written against the macro. Same family of trap as
the discarded volatile
348     * read in fujilib.c -- assume nothing about this
compiler and volatile.
349     *
350     * End of directory is two 0x7F bytes: that is what
fujidevice
returns when
351     * dir_nextfile() runs out. But not every host stops
there -- reading past
352     * the end of an SD root gives back "." over and
over instead, and doing
353     * that also poisons SET_DIRECTORY_POSITION for the
rest of the session, so
354     * every later seek NAKs. Neither "." nor ".." is
ever something to boot,
355     * so treat them as the end too and never read past
it. */
356     r = FN_REPLY;
357     b0 = r[0];
358     b1 = r[1];
359     b2 = r[2];
360
361     if (b0 == 0x7F && b1 == 0x7F)
362         return false;
363     if (b0 == '.' && (b1 == 0 || (b1 == '.' && b2 == 0)))
364         return false;
365     return true;
366 }
367
368 static bool seek_dir(unsigned int pos)
369 {
370     fn_start(FN_DEV_FUJINET, CMD_SET_DIRECTORY_POSITION);
371     fn_param8((unsigned char)pos);
372     return fn_commit() == FN_OK && fn_acked();
373 }
374
375 static void file_page(void)
376 {
377     volatile unsigned char *name;
378     unsigned char i;
379
380     draw_frame();
381     disp_at(1, 3, "SELECT A FILE");
382     status("FIRE BOOT * BACK");
383
384     /* Position once, then read sequentially:
READ_DIR_ENTRY advances the
385     * cursor itself, so a page costs one seek plus one
read per
row rather
386     * than a seek per row. */
387     if (!seek_dir(top)) {
388         oops("ESEEK", 0);
389         nrows = 0;
390         return;
391     }
392
393     nrows = 0;
394     at_end = 0;
395     for (i = 0; i < LIST_ROWS; i++) {
396         unsigned char row = (unsigned char)(LIST_TOP +
i);
397         unsigned char col;
398
399         disp_row_clear(row);
400         if (!read_entry(NAMELEN)) {

```

```

401             at_end = 1;
402             break;
403         }
404         name = FN_REPLY;
405         for (col = 0; col < NAMELEN && name[col] != 0;
col++)
406             disp_char((unsigned char)(2 + col), row,
(char)name[col]);
407         nrows = (unsigned char)(i + 1);
408     }
409
410     if (nrows == 0) {
411         disp_at(2, LIST_TOP, "(empty)");
412         return;
413     }
414     if (cur >= nrows)
415         cur = (unsigned char)(nrows - 1);
416     draw_cursor(cur, true);
417 }
418
419 /* ---- actions ---- */
420
421 static void enter_host(void)
422 {
423     host = cur;
424
425     status("MOUNTING HOST...");
426     fn_start(FN_DEV_FUJINET, CMD_MOUNT_HOST);
427     fn_param8(host);
428     if (need_ack("EHOST") != 0)
429         return;
430
431     fn_start(FN_DEV_FUJINET, CMD_OPEN_DIRECTORY);
432     fn_param8(host);
433     fn_tx_padded("/", PATH_LEN);
434     if (need_ack("EOPEN") != 0)
435         return;
436
437     page = PAGE_FILES;
438     top = 0;
439     cur = 0;
440     file_page();
441 }
442
443 static void leave_host(void)
444 {
445     fn_start(FN_DEV_FUJINET, CMD_CLOSE_DIRECTORY);
446     (void)fn_commit(); /* best effort: the host
page redraws anyway */
447     page = PAGE_HOSTS;
448     /* `cur` is the file cursor right now; put it back on
the host we came
449     * from, or coming back from a long listing lands on
an arbitrary slot. */
450     cur = host;
451     host_page();
452 }
453
454 static void boot_selected(void)
455 {
456     volatile unsigned char *name;
457     unsigned char pct = 0xFF;
458
459     /* Re-read the entry at full width: what is on screen
was crunched to the
460     * display width, and a boot path built from that
would be a filename that
461     * does not exist. */
462     status("READING...");
463     if (!seek_dir(top + cur) || !read_entry(FULLLEN)) {
464         oops("EREAD", 0);
465         return;
466     }
467     name = FN_REPLY;
468     if (name[0] == 0) {
469         oops("EEMPTY", 0);
470         return;
471     }
472
473     /* Directories come back with a trailing '/'.
Descending into them is the
474     * real CONFIG's job; here, say so rather than NAKING
mysteriously. */
475     {
476         unsigned int n = 0;
477
478         while (n < FULLLEN && name[n] != 0)
479             n++;
480         if (n > 0 && name[n - 1] == '/') {
481             oops("EISDIR", 0);
482             return;
483         }
484     }
485
486     status("SET PATH...");
487     fn_start(FN_DEV_FUJINET, CMD_SET_DEVICE_FULLPATH);
488     fn_param8(DEVSLOT);
489     fn_param8(host);

```

```

490     fn_param8(MODE_READ);
491     fn_tx_path("/", name, PATH_LEN);
492     if (need_ack("EPATH") != 0)
493         return;
494
495     status("MOUNTING...");
496     fn_start(FN_DEV_FUJINET, CMD_MOUNT_IMAGE);
497     fn_param8(DEVSLOT);
498     fn_param8(MODE_READ);
499     if (need_ack("EMOUNT") != 0)
500         return;
501
502     status("LOADING");
503     for (;;) {
504         unsigned char st = FN_BOOTSTAT;
505
506         if (st == FNB_READY)
507             break;
508         if (st == FNB_FAILED) {
509             oops("ELOAD", FN_BOOTERR);
510             return;
511         }
512         if (FN_BOOTPCT != pct) {
513             pct = FN_BOOTPCT;
514             disp_at_u16(24, 22, pct);
515         }
516     }
517
518     status("BOOTING");
519     fn_boot_swap(); /* does not return */
520 }
521
522 /* ---- main ---- */
523
524 /* Every accepted move clicks; a move that goes nowhere
-- the top of the first
525 * page, the bottom of the last -- stays silent, so the
sound says whether the
526 * cursor actually went anywhere rather than just that a
direction was held. */
527 static void move_cursor(signed char delta)
528 {
529     if (nrows == 0)
530         return;
531
532     if (delta < 0) {
533         if (cur > 0) {
534             draw_cursor(cur, false);
535             cur--;
536             draw_cursor(cur, true);
537             snd_click();
538         } else if (page == PAGE_FILES && top >=
LIST_ROWS) {
539             top -= LIST_ROWS;
540             cur = LIST_ROWS - 1;
541             snd_click();
542             file_page();
543         }
544     } else {
545         if (cur + 1 < nrows) {
546             draw_cursor(cur, false);
547             cur++;
548             draw_cursor(cur, true);
549             snd_click();
550         } else if (page == PAGE_FILES && !at_end) {
551             top += LIST_ROWS;
552             cur = 0;
553             snd_click();
554             file_page();
555         }
556     }
557 }
558
559 void main(void)
560 {
561     snd_init(); /* the PSG powers up buzzing; see
fujisnd.h */
562
563     /* Splash first: it owns the VDP, loading the logo
glyphs over the pattern
564     * range disp_init() wants for its inverse charset.
disp_init() afterwards
565     * puts both back. */
566     splash_show();
567     in_init();
568     {
569         unsigned char start = in_frames();
570
571         while ((unsigned char)(in_frames() - start) <
120)
572             if (in_read() == IN_FIRE)
573                 break;
574     }
575
576     disp_init();
577
578     if (!fn_present()) {
579         disp_at(4, 8, "NO FUJINET CART");
580
581         for (;;)
582             ;
583     }
584
585     page = PAGE_HOSTS;
586     host_page();
587
588     for (;;) {
589         unsigned char ev = in_read();
590
591         switch (ev) {
592             case IN_NONE:
593                 break;
594             case IN_UP:
595                 move_cursor(-1);
596                 break;
597             case IN_DOWN:
598                 move_cursor(1);
599                 break;
600             case IN_LEFT:
601                 if (page == PAGE_FILES && top >= LIST_ROWS) {
602                     top -= LIST_ROWS;
603                     cur = 0;
604                     snd_click();
605                     file_page();
606                 }
607                 break;
608             case IN_RIGHT:
609                 if (page == PAGE_FILES && !at_end) {
610                     top += LIST_ROWS;
611                     cur = 0;
612                     snd_click();
613                     file_page();
614                 }
615                 break;
616             case IN_FIRE:
617                 if (page == PAGE_HOSTS)
618                     enter_host();
619                 else if (page == PAGE_CONFIG)
620                     connect_network();
621                 else
622                     boot_selected();
623                 break;
624             case IN_KEystar:
625                 if (page == PAGE_FILES) {
626                     leave_host();
627                 } else if (page == PAGE_CONFIG) {
628                     page = PAGE_HOSTS;
629                     cur = host;
630                     host_page();
631                 }
632                 break;
633             case IN_KEYHASH:
634                 /* # opens the configuration screen. */
635                 if (page == PAGE_HOSTS) {
636                     page = PAGE_CONFIG;
637                     config_page();
638                 }
639                 break;
640             case IN_KEY0 + 9:
641                 /* 9 renames the highlighted host slot. */
642                 if (page == PAGE_HOSTS)
643                     rename_host();
644                 break;
645             default:
646                 /* Keypad 1-8 jumps straight to a host slot.
*/
647                 if (page == PAGE_HOSTS && ev >= IN_KEY0 + 1
&& ev <= IN_KEY0 + HOST_SLOTS) {
648                     draw_cursor(cur, false);
649                     cur = (unsigned char)(ev - IN_KEY0 - 1);
650                     draw_cursor(cur, true);
651                     snd_click();
652                     enter_host();
653                 }
654                 break;
655         }
656     }
657 }
658 }
659

```

FUJISPLASH.H

```

1 /* fujisplash.h -- the boot splash.
2 *
3 * Every ColecoVision cartridge shows the OS7 title screen before it runs;
4 * ours skips it (the $55AA header) and so has to put something there itself.
5 * This is that screen, built to the same shape as the real one -- measured
6 * rather than remembered: OS7 draws a multicoloured banner near the top on a
7 * BLACK backdrop, with centred white text lower down, its colour table running
8 * 0xF0 (white on transparent) for the text groups and a magenta/red/yellow/
9 * green/blue sweep across the banner's own pattern groups.
10 *
11 * Here the FujiNet wordmark takes the banner's place, in the middle of the
12 * screen, with the same trick: its 96 glyphs span twelve colour groups and the
13 * artwork happens to allocate them roughly top-to-bottom, so assigning a
14 * gradient across those groups paints a sweep down the logo.
15 *
16 * The artwork is from ~/Workspace/fujinet-adam-logo (Thomas Cherryhomes, 2021,
17 * GPLv3); see fujisplash_data.c. Regenerate that file with:
18 *
19 *     python3 - reads logo.c, strips // comments, takes logo_patterns whole
20 *     and the first 320 bytes of logo_nametable (10 graphic rows, dropping the
21 *     ADAM strapline), and emits both as C arrays.
22 */
23
24 #ifndef FUJISPLASH_H
25 #define FUJISPLASH_H
26
27 #define SPLASH_ROW      6      /* top row of the 10-row logo */
28 #define SPLASH_PATTERN  0x80   /* baked into the name-table data */
29
30 extern const unsigned char splash_patterns[768];
31 extern const unsigned char splash_nametable[320];
32
33 /* Take over the screen and draw the splash. Leaves the VDP in Graphics 1 with
34 * the logo glyphs loaded at 0x80 -- so the caller must run disp_init() again
35 * before drawing anything else, which rebuilds the ASCII and inverse sets. */
36 void splash_show(void);
37
38 #endif /* FUJISPLASH_H */
39

```

FUJISPLASH.C

```

1 #include <os7.h>
2
3 #include "fujidisp.h"
4 #include "fujisplash.h"
5
6 /* Colour groups 16-27 cover patterns 0x80-0xDF, which is
7 the whole logo. The
8 * artwork allocates its glyphs roughly top-to-bottom, so
9 this reads on screen
10 * as a sweep down the wordmark: white at the mast,
11 warning through yellow to
12 * deep red at the base. Low nibble 0 is transparent,
13 which shows the black
14 * backdrop -- exactly how the OS7 banner is coloured. */
15 static const unsigned char logo_colors[12] = {
16     0xF0, 0xF0,      /* white */
17     0xB0, 0xB0,      /* light yellow */
18     0xA0, 0xA0,      /* dark yellow */
19     0x90, 0x90,      /* light red */
20     0x80, 0x80,      /* medium red */
21     0x60, 0x60,      /* dark red */
22 };
23
24 static void centre(unsigned char row, const char *s)
25 {
26     unsigned char n = 0;
27     while (s[n] != '\0')
28         n++;
29     disp_at((unsigned char)(n >= DISP_COLS ? 0 :
30 (DISP_COLS - n) / 2), row, s);
31 }
32
33 void splash_show(void)
34 {
35     mode_1();
36     load_ascii();
37
38     /* Text groups white on transparent, like the OS7
39 splash's own; the logo's
40 * groups take the gradient. */
41     fill_vram(0x2000, 32, 0xF0);
42     put_vram(PATTERN_COLOR_TABLE, 16, (void
43 *)logo_colors, 12);
44
45     write_register(REGISTER_BACKGROUND, BLACK);
46
47     disp_cls();
48     put_vram(PATTERN_GENERATOR_TABLE, SPLASH_PATTERN,
49 (void *)splash_patterns, 96);
50
51     /* One row at a time, because OS7's PUT_VRAM
52 truncates its count to 8 bits.
53 * Asking for all 320 bytes at once writes 320 & 0xFF
54 = 64 -- two rows --
55 * and returns as though it had done the job. Nothing
56 warns; the logo just
57 * comes out with its top two rows and nothing else.
58 The 96-pattern write
59 * above is safe because 96 is the number of ENTRIES,
60 not bytes, and OS7
61 * scales it internally to 768. Keep every count here
62 under 256. */
63 {
64     unsigned char r;
65     for (r = 0; r < 10; r++)
66         put_vram(PATTERN_NAME_TABLE,
67 (unsigned short)(SPLASH_ROW + r) *
68 DISP_COLS,
69 (void *) (splash_nametable +
70 (unsigned int)r * DISP_COLS),
71 DISP_COLS);
72 }
73
74 /* Exactly 32 characters, so it fills the row edge to
75 edge. */
76 disp_at(0, 18, "NETWORK ADAPTER FOR COLECOVISION");
77 centre(21, "PRESS FIRE");
78

```

C.6 5 CARD STUD (ColecoVision port)

MAIN.C

```

1  /**
2  * @brief 5-card-stud
3  * @author Eric Carr, Thomas Cherryhomes, (insert names
here)
4  * @license gpl v. 3
5  * @verbose main
6  */
7
8  #ifdef _CMOC_VERSION_
9  #include "coco/coco_bool.h"
10 #else
11 #include <stdlib.h>
12 #include <stdint.h>
13 #include <stdbool.h>
14 #include <string.h>
15 #endif /* _CMOC_VERSION_ */
16
17 #include "platform-specific/graphics.h"
18 #include "platform-specific/util.h"
19 #include "platform-specific/input.h"
20 #include "misc.h"
21 #include "platform-specific/network.h"
22 #include "platform-specific/appkey.h"
23 #include "platform-specific/sound.h"
24
25 #include "stateclient.h"
26 #include "gamelogic.h"
27 #include "screens.h"
28
29 // Store default server endpoint in case lobby did not
set app key
30 char serverEndpoint[50] = "https://5card.carr-
designs.com/";
31 //char serverEndpoint[50] = "http://127.0.0.1:8080/"; //
"N: for apple, but not C64"
32
33 char query[QUERY_LEN] = ""; //&player=ERICAPL2";
34 char playerName[12] = ""; // "eric";
35 //char playerName[12] = "ericmsx";
36
37 //GameState state;
38 // On the ColecoVision this is a macro for the
cartridge's reply window, not an
39 // object -- see src/misc.h.
40 #ifndef BUILD_COLECO
41 ClientState clientState;
42 #endif
43
44
45 // State helper vars
46 #ifdef __WATCOMC__
47 int inputKey;
48 #else
49 char inputKey;
50 #endif
51 unsigned char playerCount, prevPlayerCount,
validMoveCount, prevRound, tableCount, currentCard,
cardIndex, xOffset, fullFirst, cursorX, cursorY, waitCount,
wasViewing;
52 signed char inputDirX, inputDirY;
53 uint16_t prevPot, maxJifs;
54 bool noAnim, doAnim, finalFlip, inputTrigger;
55
56 unsigned char playerX[8], playerY[8], moveLoc[5];
57 signed char playerBetX[8], playerBetY[8], playerDir[8];
58
59 // Common local scope temp variables
60 unsigned char h, i, j, k, x, y, xx;
61 char tempBuffer[TEMP_BUFFER_LEN];
62
63 char prefs[4];
64
65 char *hand, *requestedMove;
66
67
68 #ifdef _CMOC_VERSION_
69 int main(void)
70 #else
71 void main(void)
72 #endif /* _CMOC_VERSION_ */
73 {
74     // uint8_t x, result;
75     // x=0;
76     // x=x+2;
77
78     // unsigned char i; while (1) { if (kbhit()) { inputKey =
cgetc(); printf("\nKEY: %u", inputKey); } for (i=0; i<10; i++)
{ pause(2); printf("."); } }
79     loadPrefs();
80     initGraphics();
81     initSound();
82
83     // Checking MSX C compiler bug
84     // x=10;
85     // result=(x>5)+10;
86     // itoa(result, tempBuffer, 10);
87
88     // cputs(tempBuffer);
89     // cgetc();
90
91     #ifdef USE_PLATFORM_SPECIFIC_INPUT
92     initPlatformKeyboardInput();
93     #endif
94
95     showWelcomeScreen();
96     showTableSelectionScreen();
97
98     // Main in-game loop
99     while (true) {
100
101         // Get latest state and draw on screen, then prompt
player for move if their turn
102         if (getStateFromServer()) {
103             showGameScreen();
104             requestPlayerMove();
105         } else {
106             // Wait a bit to avoid hammering the server if
getting bad responses
107             pause(30);
108         }
109
110         // Handle other key presses
111         readCommonInput();
112
113         switch(inputKey) {
114             case KEY_ESCAPE: // Esc
115             case KEY_ESCAPE_ALT: // Esc Alt
116                 showInGameMenuScreen();
117                 break;
118         }
119
120     }
121 }
122 #ifdef _CMOC_VERSION_
123 return 0;
124 #endif
125 }
126

```

MISC.H -- THE STATE-IN-WINDOW MACRO

```

1 #ifndef MISC_H
2 #define MISC_H
3
4 /* To test the game:
5 * VICE
6 * 1. In VICE, point DRIVE 11 to a local folder
7 * 2. Edit /support/c64/emulator/fuji_mock_network.py to
point to that same folder and run it
8 * 3. Uncomment the USE_EMULATOR define below. */
9
10 // #define USE_EMULATOR
11
12 /*
13 * Include CC65 style Joystick defines for Adam - There
is a probably a better way to do this.
14 */
15
16 #ifndef _CMOC_VERSION_
17 #include "coco/joystick.h"
18 #else
19 #ifndef __ADAM__
20 // conio.h must be parsed before the "#define state"
below: it drags in
21 // z88dk's X11 headers, which declare struct members
named "state".
22 #include <conio.h>
23 #include "adam/joystick.h"
24 #else
25 #ifndef BUILD_COLECO
26 // Same z88dk conio ordering hazard as the Adam, same
reason.
27 #include <conio.h>
28 #include "colecto/joystick.h"
29 #else
30 #ifndef __DOS__
31 #include <conio.h>
32 #endif /* __DOS__ */
33 #endif /* BUILD_COLECO */
34 #endif /* __ADAM__ */
35 #endif /* _CMOC_VERSION_ */
36 #include "platform-specific/graphics.h"
37 #include "platform-specific/input.h"
38 #ifdef _CMOC_VERSION_
39 #include <cmoc.h>
40 #include <coco.h>
41 #else
42 #include <stdbool.h>
43 #include <stdint.h>
44 #endif /* _CMOC_VERSION_ */
45 // FujiNet AppKey settings. These should not be changed
46 #define AK_LOBBY_CREATOR_ID 1 // FUJINET Lobby
47 #define AK_LOBBY_APP_ID 1 // Lobby Enabled Game
48 #define AK_LOBBY_KEY_USERNAME 0 // Lobby Username key
49 #define AK_LOBBY_KEY_SERVER 1 // 5 Card Stud Client
registered as Lobby appkey 1
50
51 // 5 Card Stud client
52 #define AK_CREATOR_ID 0xE41C // Eric Carr's creator
id
53 #define AK_APP_ID 1 // 5 Card/Poker App ID
54 #define AK_KEY_PREFS 0 // Preferences
55
56 #define PREF_HELP 0 // 1/2 seen help screen no/yes
57 #define PREF_COLOR 1 // 1/2 - Color mode (platform
specific)
58 // #define PREF_SOUND 2 // 1/2 - Sound Enabled yes/no
59
60 #ifndef __CC65__
61 #define _Packed
62 #endif
63
64 #ifndef _CMOC_VERSION_
65 #define _Packed
66 #endif
67
68 #ifndef BUILD_MSX
69 #define _Packed
70 #endif
71
72 #ifndef __ADAM__
73 #define _Packed
74 #endif
75
76 #ifndef BUILD_COLECO
77 #define _Packed
78 #endif
79
80 typedef _Packed struct {
81 char table [9];
82 char name [21];
83 char players [6];
84 } Table;
85
86 typedef _Packed struct {
87 char name [9];
88 uint8_t status;
89 uint16_t bet;
90 char move [8];
91 uint16_t purse;
92 char hand [11];
93 } Player;
94
95 typedef _Packed struct {
96 char move [3];
97 char name [10];
98 } ValidMove;
99
100 typedef _Packed struct {
101 char lastResult[81];
102 uint8_t round;
103 uint16_t pot;
104 int8_t activePlayer;
105 uint8_t moveTime;
106 uint8_t viewing;
107 uint8_t validMoveCount;
108 ValidMove validMoves[5];
109 uint8_t playerCount;
110 Player players[8];
111 } Game;
112
113 typedef _Packed struct {
114 uint8_t count;
115 Table table[10];
116 } Tables;
117
118 typedef union {
119 uint8_t firstByte;
120 Game game;
121 Tables tables;
122 } ClientState;
123
124
125 #ifndef __WATCOMC__
126 extern int inputKey;
127 #else
128 extern char inputKey;
129 #endif
130 extern unsigned char prevPlayerCount, prevRound,
currentCard, cardIndex, xOffset, fullFirst, cursorX, cursorY,
waitCount, wasViewing;
131 extern signed char inputDirX, inputDirY;
132
133 extern uint16_t prevPot, maxJifs;
134 #ifndef _CMOC_VERSION_
135 extern unsigned char noAnim, doAnim, finalFlip,
inputTrigger;
136 #else
137 extern bool noAnim, doAnim, finalFlip, inputTrigger;
138 #endif /* _CMOC_VERSION_ */
139 /*
140 Scratch buffer sizes. The ColecoVision has 1K of RAM in
total, so these are
141 cut to what each one actually has to hold rather than
to a round number.
142 tempBuffer cannot go below MAX_APPKEY_LEN+1: screens.c
reads app keys into it.
143 */
144 #ifndef BUILD_COLECO
145 #define TEMP_BUFFER_LEN 68
146 #define QUERY_LEN 24
147 #define URL_BUFFER_LEN 96
148 #else
149 #define TEMP_BUFFER_LEN 128
150 #define QUERY_LEN 50
151 #define URL_BUFFER_LEN 128
152 #endif
153
154 extern char tempBuffer[TEMP_BUFFER_LEN];
155 extern char query[QUERY_LEN];
156 extern char playerName[12];
157 extern char serverEndpoint[50];
158
159 #ifndef BUILD_COLECO
160 /*
161 The ColecoVision has 1K of RAM and this union is 418
bytes of it, so the
162 state is never copied down: it is read in place out of
the cartridge's reply
163 window, which is what that window is 1K and addressable
for. See
164 src/colecto/vars.h and src/colecto/network.c.
165
166 It is therefore READ-ONLY -- a store here goes nowhere,
the cartridge cannot
167 even see a write cycle -- and it is valid only until
the next fuji_* call of
168 any kind, because every transaction repaints the
window.

```

```

169 */
170 #define clientState (*(ClientState *)
COLECO_REPLY_WINDOW)
171 #else
172 extern ClientState clientState;
173 #endif
174 //extern GameState state;
175 #define state clientState.game
176
177 // Common local scope temp variables
178 extern unsigned char h, i, j, k, x, y, xx;
179 extern unsigned char playerX[8], playerY[8], moveLoc[5];
180 extern signed char playerBetX[8], playerBetY[8],
playerDir[8];
181 extern char *hand, *requestedMove;
182 extern char prefs[4];
183
184
185 // Screen specific player/bet coordinates. These are
declared by each platform's
186 // own vars.h as well; the ColecoVision makes them const
so they land in ROM
187 // rather than spending 96 bytes of a 1K machine on
tables nobody writes.
188 #ifndef BUILD_COLECO
189 extern unsigned char playerXMaster[] ;
190 extern unsigned char playerYMaster[] ;
191 extern char playerDirMaster[] ;
192 extern char playerBetXMaster[] ;
193 extern char playerBetYMaster[] ;
194
195 // Simple hard coded arrangment of players around the
table based on player count.
196 // These refer to index positions in the Master arrays
above
197 // Downside is new players will cause existing player
positions to move.
198 extern char playerCountIndex[] ;
199 #endif
200
201 void pause(unsigned char frames);
202 void clearCommonInput();
203 void readCommonInput();
204 void loadPrefs();
205 void savePrefs();
206
207 #endif /* MISC_H */
208
209

```

COLECO/NETWORK.C

```

1 #ifndef BUILD_COLECO
2
3 /**
4  * @brief Network functions
5  * @author Thomas Cherryhomes
6  * @email thom dot cherryhomes at gmail dot com
7  * @license gpl v. 3, see LICENSE for details
8  */
9
10 #include <stdint.h>
11 #include <fujinet-network.h>
12 #include "vars.h"
13
14 static uint8_t initialized = 0;
15 static uint8_t channelOpen = 0;
16
17 /*
18  The response is never copied into RAM. `buffer` is the
cartridge's reply
19  window (COLECO_REPLY_WINDOW, and see src/misc.h's
clientState) and the game
20  renders straight out of it, because the console has 1K
of RAM and the state
21  alone is 418 bytes. Two things follow, and both are the
reason this is not
22  simply src/adam/network.c:
23
24  1. The read has to be ONE transaction. network_read()'s
loop would copy its
25  second chunk to `buffer + count`, which is cartridge
ROM -- the store goes
26  nowhere and the tail is lost, silently.
27  network_read_nb() waits for the
28  adapter to report bytes available and then issues
exactly one FUJICMD_READ,
29  so what the cartridge paints is the whole body. The
window is 1K and the
30  largest response here is 418 bytes, so it always
fits; what has to hold on
31  the wire is that FujiNet has buffered the entire
body before it reports
32  any of it available, which it does for an HTTP GET.
33
34  2. Nothing may run after the read. Every mailbox
transaction repaints the
35  window, network_close() included, so closing here
would destroy the reply
36  before the caller ever looked at it. The channel is
closed at the top of
37  the NEXT call instead.
38  */
39 uint8_t getResponse(char *url, unsigned char *buffer,
uint16_t max_len)
40 {
41     int16_t count;
42
43     if (!initialized) {
44         if (network_init() != FN_ERR_OK)
45             return 0;
46         initialized = 1;
47     }
48
49     // The url handed in is the next request's, not the
one that was opened,
50     // but every url this client builds carries the bare
"N:" prefix and so
51     // resolves to the same network unit.
52     if (channelOpen) {
53         network_close(url);
54         channelOpen = 0;
55     }
56
57     // Don't read from a channel that never opened -
network_read_nb() would
58     // otherwise poll status on a dead unit and hand back
whatever it got.
59     if (network_open(url, OPEN_MODE_HTTP_GET_H,
OPEN_TRANS_NONE) != FN_ERR_OK)
60         return 0;
61     channelOpen = 1;
62
63     count = network_read_nb(url, buffer, max_len);
64
65     return count > 0;
66 }

```

```
67 #endif /* BUILD_COLECO */
68
```

COLECO/UTIL.C -- THE LOBBY HANDOFF

```
1 #ifndef BUILD_COLECO
2
3 /**
4  * @brief Utility Functions
5  * @author Thomas Cherryhomes
6  * @email thom dot cherryhomes at gmail dot com
7  * @license gpl v. 3, see LICENSE for details
8  */
9
10 #include <stdbool.h>
11 #include <stdint.h>
12 #include <string.h>
13 #include <stdlib.h>
14 #include <interrupt.h>
15 #include <fujinet-fuji.h>
16 #include <fujinet-coleco.h>
17 #include "vars.h"
18 #include "../platform-specific/graphics.h"
19
20 // The firmware's WRITE HOST SLOTS expects all 8 slots.
21 #define FUJI_HOST_SLOT_COUNT 8
22
23 #define LOBBY_DEVICE_SLOT 0
24 #define LOBBY_MODE_READ 1
25
26 volatile unsigned long currentTime;
27 static volatile bool vsync;
28
29 // Installed as the VDP interrupt hook by initGraphics()
30 // via add_raster_int().
31 // On this machine that hook is the vblank NMI: it cannot
32 // be masked, DI will not
33 // stop it, and it lands in the middle of mailbox
34 // transactions by design. That
35 // is harmless only for as long as it stays away from
36 // $F800 and up -- a handler
37 // that reads the mailbox pages corrupts whatever
38 // transaction it interrupted.
39 // Bumping two counters is safe; do not grow this.
40 void myInt(void)
41 {
42     currentTime++;
43     vsync=true;
44 }
45
46 void waitvsync(void)
47 {
48     while(!vsync);
49     vsync=false;
50 }
51
52 void resetTimer(void)
53 {
54     currentTime = 0;
55 }
56
57 int getTime(void)
58 {
59     return currentTime;
60 }
61
62 static bool sameHost(const char *a, const char *b)
63 {
64     while (*a && *b)
65     {
66         if ((*a | 0x20) != (*b | 0x20))
67             return false;
68         a++;
69         b++;
70     }
71     return *a == *b;
72 }
73
74 /*
75  * SET_DEVICE_FULLPATH is a fixed 256-byte payload -- a
76  * short one is rejected on
77  * the ESP32 side -- so the path is padded here rather
78  * than at the call site.
79  * In ROM, so the padding costs no RAM.
80  */
81 static const char lobbyPath[MAX_FILENAME_LEN] = "coleco/
82 lobby.rom";
83 static const char lobbyHost[] = "ec.tnfs.io";
84
85 /*
86  * The 8x32 host slot table is read IN PLACE out of the
87  * cartridge's reply window,
88  * the same way the game state is (see src/misc.h).
89  * READ_HOST_SLOTS is a single
90  * transaction and 256 bytes fit inside the 1K window, so
```

```
the whole table is
81 addressable ROM by the time the call returns -- which
82 is the only way it is
83 affordable at all on a 1K machine.
84 */
85 static uint8_t findLobbyHost(void)
86 {
87     HostSlot *slots = (HostSlot *) COLECO_REPLY_WINDOW;
88     uint8_t i;
89
90     if (!fuji_get_host_slots(slots, FUJI_HOST_SLOT_COUNT))
91         return FUJI_HOST_SLOT_COUNT;
92
93     for (i = 0; i < FUJI_HOST_SLOT_COUNT; i++)
94         if (sameHost(lobbyHost, (const char *) slots[i]))
95             return i;
96
97     return FUJI_HOST_SLOT_COUNT;
98 }
99
100 /*
101  * The slot is used as found; it is never created.
102  * WRITE_HOST_SLOTS is
103  * all-or-nothing over the whole 8x32 table, and the seven
104  * slots that would have
105  * to be preserved only exist in the reply window, which
106  * cannot be patched -- so
107  * creating one means staging 256 bytes somewhere. There
108  * is nowhere: BSS would
109  * cost a quarter of the machine permanently, and the C
110  * stack here is 289 bytes.
111  * A player who reached this client through the FujiNet
112  * Lobby already has the
113  * host; anyone else is told to add it in CONFIG, which is
114  * a better outcome than
115  * a stack overflow at the one moment the game is handing
116  * the console away.
117  */
118 static bool mountLobby(uint8_t slot)
119 {
120     if (!fuji_mount_host_slot(slot))
121         return false;
122     if (!fuji_set_device_filename(LOBBY_MODE_READ, slot,
123     LOBBY_DEVICE_SLOT,
124     (char *) lobbyPath))
125         return false;
126
127     // Only starts the transfer. The image is pushed to the
128     // cartridge
129     // asynchronously, after this has already been
130     // answered.
131     return fuji_mount_disk_image(LOBBY_DEVICE_SLOT,
132     LOBBY_MODE_READ) != 0;
133 }
134
135 void quit(void)
136 {
137     // Not `state`: src/misc.h defines that as a macro for
138     // the game state.
139     uint8_t bootState, slot, pct = 0xFF;
140     char pctText[5];
141
142     drawStatusText("LOADING LOBBY...");
143
144     if (!fuji_coleco_present())
145     {
146         drawStatusText("NO FUJINET CARTRIDGE");
147         return;
148     }
149
150     slot = findLobbyHost();
151     if (slot == FUJI_HOST_SLOT_COUNT)
152     {
153         drawStatusText("ADD EC.TNFS.IO IN CONFIG");
154         return;
155     }
156
157     if (!mountLobby(slot))
158     {
159         drawStatusText("LOBBY NOT AVAILABLE");
160         return;
161     }
162
163     // Watch the cartridge's own progress counter rather
164     // than guessing a delay.
165     for (;;)
166     {
167         bootState = fuji_coleco_boot_state();
168         if (bootState == FUJI_COLECO_BOOT_READY)
```

```

155     break;
156     if (bootState == FUJI_COLECO_BOOT_FAILED)
157     {
158         drawStatusText("LOBBY LOAD FAILED");
159         return;
160     }
161
162     if (fuji_coleco_boot_percent() != pct)
163     {
164         pct = fuji_coleco_boot_percent();
165         itoa(pct, pctText, 10);
166         strcat(pctText, "%");
167         drawStatusTextAt(18, pctText);
168     }
169 }
170
171 // Stop the firmware serving CONFIG at boot, then hand
the console over. The
172 // swap does not return: every byte of $8000-$FFFF
changes underneath it.
173 fuji_set_boot_config(0);
174 fuji_coleco_boot_swap();
175 }
176
177 #endif /* BUILD_COLECO */
178

```

COLECO/VARS.H

```

1  #ifndef BUILD_COLECO
2
3  #ifndef KEYMAP_H
4  #define KEYMAP_H
5
6  // Screen dimensions for platform
7
8  #define WIDTH 32
9  #define HEIGHT 24
10
11 #define SINGLE_BUFFER 1
12
13 #define POT_Y_MODIFIER -1
14 #define STATUS_TIMER_WIDTH 0
15 #define HOW_TO_PLAY_ROW_START 2
16
17 // The whole reply to the last mailbox transaction, as
directly addressable
18 // cartridge ROM. bus/coleco/fujinet-bus-coleco.h
(FN_REPLY) is the source of
19 // truth; keep the two in step.
20 //
21 // This is where ClientState lives on this platform (see
src/misc.h). The
22 // console has 1K of RAM and the state alone is 418
bytes, so the state is
23 // never copied down -- the game renders straight out of
the window. Two rules
24 // follow, and neither is optional:
25 //
26 // - it is READ-ONLY. A store here goes nowhere; the
cartridge cannot even
27 // see a write cycle.
28 // - it is valid only until the next fuji_* call of ANY
kind. Every
29 // transaction repaints the window, appkey reads and
writes included.
30 #define COLECO_REPLY_WINDOW 0xF800
31
32 // There is no keyboard: a stick, two fire buttons and a
twelve-key keypad.
33 // src/coleco/input.c synthesises key codes from the
keypad and the second fire
34 // button, and src/coleco/osk.c types text on screen.
35 #define USE_PLATFORM_SPECIFIC_INPUT 1
36 #define USE_PLATFORM_NAME_ENTRY 1
37
38 /**
39  * Platform specific key map for common input
40  */
41
42 // Direction comes from the stick, never from a key, so
the arrow codes are
43 // deliberately unreachable placeholders -- they exist
because readCommonInput()
44 // switches on all twelve of them.
45 #define KEY_LEFT_ARROW 0xF1
46 #define KEY_LEFT_ARROW_2 0xF2
47 #define KEY_LEFT_ARROW_3 0xF3
48
49 #define KEY_RIGHT_ARROW 0xF4
50 #define KEY_RIGHT_ARROW_2 0xF5
51 #define KEY_RIGHT_ARROW_3 0xF6
52
53 #define KEY_UP_ARROW 0xF7
54 #define KEY_UP_ARROW_2 0xF8
55 #define KEY_UP_ARROW_3 0xF9
56
57 #define KEY_DOWN_ARROW 0xFA
58 #define KEY_DOWN_ARROW_2 0xFB
59 #define KEY_DOWN_ARROW_3 0xFC
60
61 // Keypad # and the left fire button; keypad * and the
right fire button.
62 #define KEY_RETURN 0x0D
63 #define KEY_ESCAPE 0x1B
64 #define KEY_ESCAPE_ALT 0x03
65 #define KEY_SPACE 0x20
66 #define KEY_BACKSPACE 0x08
67
68 /*
69  Mapping for converting incoming ALT letters to a
standard case
70 */
71
72 #define LINE_ENDING 0x0A
73 #define ALT_LETTER_START 0x0
74 #define ALT_LETTER_END 0x0
75 #define ALT_LETTER_AND 0x0
76
77 #define QUERY_SUFFIX ""
78
79 /*
80  Screen related variables
81 */
82
83 // Screen specific player/bet coordinates. const, and
therefore in ROM: on a
84 // 1K machine 96 bytes of never-written seat tables is
worth the qualifier.
85 extern const unsigned char playerXMaster[];
86 extern const unsigned char playerYMaster[];
87
88 extern const char playerDirMaster[];
89 extern const char playerBetXMaster[];
90 extern const char playerBetYMaster[];
91
92 // Simple hard coded arrangement of players around the
table based on player count.
93 // These refer to index positions in the Master arrays
above
94 // Downside is new players will cause existing player
positions to move.
95
96 //
97 extern const char playerCountIndex[];
98
99
100 #endif /* KEYMAP_H */
101
102 #endif /* BUILD_COLECO */
103

```

COLECO/INPUT.C

```

1  #ifdef BUILD_COLECO
2
3  /**
4   * @brief ColecoVision input routines
5   * @author Thomas Cherryhomes
6   * @license gpl v.3
7   */
8
9  #include <games.h>
10 #include "joystick.h"
11 #include "../platform-specific/input.h"
12
13 /*
14  A ColecoVision controller is a four-way stick, two fire
15  buttons and a
16  twelve-key keypad. z88dk's joystick() is
17  coleco_joypad(): device 1 returns
18  the player-1 stick in the low byte, device 3 returns
19  the same stick plus the
20  keypad as an ASCII character in the high byte ('0'-'9',
21  '*', '#', and 'C'/'D'
22  for the two purple buttons).
23
24  Both buttons are deliberately reported as KEYS rather
25  than as stick bits.
26  readCommonInput() returns early whenever
27  joystick()'s value changes and
28  only reaches getPlatformKey() when it has not, so a
29  button that appeared in
30  both places would be seen twice: once as the joystick
31  edge, and again on the
32  next pass as a key, because the button is still held.
33  Keeping every button in
34  getPlatformKey() puts all of them behind one edge
35  detector.
36 */
37
38 #define PAD_KEYPAD 3          /* joystick() device: stick +
39 keypad, player 1 */
40 #define PAD_DIRS 0x0F        /* the four direction bits;
41 see joystick.h */
42
43 /* Synthetic "what is pressed" byte, so one comparison
44 covers the keypad and
45 both buttons. Real keypad characters are all >= '#',
46 so these cannot clash. */
47 #define EV_NONE 0x00
48 #define EV_FIRE1 0x01
49 #define EV_FIRE2 0x02
50
51 static unsigned char lastEvent;
52
53 unsigned char readJoystick()
54 {
55     return (unsigned char) joystick(PAD_KEYPAD) & PAD_DIRS;
56 }
57
58 void initPlatformKeyboardInput(void)
59 {
60     lastEvent = EV_NONE;
61 }
62
63 int getPlatformKey(void)
64 {
65     unsigned int pad;
66     unsigned char event;
67
68     pad = joystick(PAD_KEYPAD);
69
70     /* The keypad wins over the buttons: it is the
71     deliberate press. */
72     event = (unsigned char) (pad >> 8);
73     if (!event) {
74         if (pad & JOY_BTN_2_MASK)
75             event = EV_FIRE2;
76         else if (pad & JOY_BTN_1_MASK)
77             event = EV_FIRE1;
78     }
79
80     if (event == lastEvent)
81         return 0;
82     lastEvent = event;
83
84     switch (event) {
85     case EV_NONE: return 0;
86     case EV_FIRE1: return KEY_RETURN;
87     case EV_FIRE2: return KEY_ESCAPE;
88     }
89
90     /* The keypad stands in for the letter shortcuts the
91     other ports type.
92     src/screens.c labels these; keep the two in step.
93 */
94     case '1': return 'r'; /* refresh */
95     case '2': return 'h'; /* how to play */
96
97     case '3': return 'c'; /* colour */
98     case '4': return 'n'; /* name */
99     case '5': return 'q'; /* quit */
100
101     case '*': return KEY_ESCAPE;
102     case '#': return KEY_RETURN;
103 }
104
105 return 0;
106 }
107
108 #endif /* BUILD_COLECO */

```

COLECO/OSK.C

```

1 #ifndef BUILD_COLECO
2
3 /**
4  * @brief   ColecoVision on-screen keyboard
5  * @author   Thomas Cherryhomes
6  * @email    thom dot cherryhomes at gmail dot com
7  * @license  gpl v. 3, see LICENSE for details
8  */
9
10 #include <games.h>
11 #include <string.h>
12 #include "vars.h"
13 #include "joystick.h"
14 #include "../platform-specific/graphics.h"
15 #include "../platform-specific/sound.h"
16
17 /*
18  A ColecoVision controller has twelve keys and a stick,
19  so any text the player
20  has to type gets typed on screen. This is the same grid
21  the Intellivision
22  client's grid_entry, the Astrocade's edit.inc and the
23  ColecoVision CONFIG
24  client's fujiedit.c all use, and they look alike for
25  the same reason: the
26  cursor position IS the character, so there is no shift
27  key and no paging.
28
29  Two rules are inherited deliberately from those three:
30
31  - cancel is the OK cell's absence, not the backspace
32  key. Backspace must
33  never discard the edit, because reaching for it
34  repeatedly while fixing a
35  typo is exactly when a reflexive extra press
36  happens. There is no cancel
37  here at all: the caller only reaches this screen
38  when a name is required.
39  - the grid never runs a mailbox transaction. The
40  caller may be holding data
41  in the cartridge's reply window, and any
42  transaction would repaint it.
43
44  Where this one departs from fujiedit.c is the CASE
45  toggle, which is not
46  offered: drawText() runsstrupr() over everything it
47  prints, so lowercase is
48  not representable anywhere in this port's text layer.
49  Uppercase-only keeps
50  what is typed and what is displayed the same thing.
51  */
52
53 #define OSK_COLS 10
54 #define OSK_ROWS 4
55 #define OSK_PITCH 3 /* marker column
56 + two label columns */
57 #define OSK_X 1
58 #define OSK_Y 17
59
60 /* The last row is one short: its tenth cell does not
61 exist. */
62 #define OSK_LAST_ROW_COLS 9
63
64 #define CELL_SPACE (OSK_COLS * 3 + 6) /* index 36 */
65 #define CELL_DEL (CELL_SPACE + 1)
66 #define CELL_OK (CELL_SPACE + 2)
67
68 static const char cells[] =
69 "ABCDEFGHIJ"
70 "KLMNOPQRST"
71 "UVWXYZ0123"
72 "456789";
73
74 static const char *labelSpace = "SP";
75 static const char *labelDel = "<-";
76 static const char *labelOk = "OK";
77
78 /* The fire button, as a value the keypad never returns.
79 */
80 #define EV_FIRE 1
81
82 #define REPEAT_FIRST 18 /* vblanks before
83 a held stick repeats */
84 #define REPEAT_NEXT 5
85
86 static unsigned char curX, curY;
87
88 static unsigned char cellCols(unsigned char row)
89 {
90     return row == OSK_ROWS - 1 ? OSK_LAST_ROW_COLS :
91     OSK_COLS;
92 }
93
94 static void drawCell(unsigned char col, unsigned char
row, unsigned char marked)
95 {
96     unsigned char idx = row * OSK_COLS + col;
97     unsigned char x = OSK_X + col * OSK_PITCH;
98     char glyph[3];
99
100     drawText(x, OSK_Y + row, marked ? ">" : " ");
101
102     if (idx == CELL_SPACE)
103         drawText(x + 1, OSK_Y + row, labelSpace);
104     else if (idx == CELL_DEL)
105         drawText(x + 1, OSK_Y + row, labelDel);
106     else if (idx == CELL_OK)
107         drawText(x + 1, OSK_Y + row, labelOk);
108     else
109     {
110         glyph[0] = cells[idx];
111         glyph[1] = ' ';
112         glyph[2] = 0;
113         drawText(x + 1, OSK_Y + row, glyph);
114     }
115 }
116
117 static void drawGrid(void)
118 {
119     unsigned char row, col;
120
121     for (row = 0; row < OSK_ROWS; row++)
122         for (col = 0; col < cellCols(row); col++)
123             drawCell(col, row, row == curY && col == curX);
124 }
125
126 static void clearGrid(void)
127 {
128     unsigned char row;
129
130     for (row = 0; row < OSK_ROWS; row++)
131         drawText(0, OSK_Y + row, "
");
132 }
133
134 /* Repaint the field and its trailing cursor chip, the
135 same shape
136 inputFieldCycle() draws on the platforms that have a
137 keyboard. */
138 static void drawField(unsigned char x, unsigned char y,
unsigned char max,
const char *buffer)
139 {
140     unsigned char len = (unsigned char) strlen(buffer);
141
142     drawText(x, y, "
");
143     drawText(x, y, buffer);
144     if (len < max)
145         drawChip(x + len, y);
146 }
147
148 static void appendChar(char *buffer, unsigned char *len,
unsigned char max,
char ch)
149 {
150     if (*len >= max)
151     {
152         soundCursorInvalid();
153         return;
154     }
155     buffer[*len++] = ch;
156     buffer[*len] = 0;
157     soundCursor();
158 }
159
160 void platformNameEntry(unsigned char x, unsigned char y,
unsigned char max,
char *buffer)
161 {
162     unsigned char len = (unsigned char) strlen(buffer);
163     unsigned char hold = 0, lastDir = 0, lastEvent = 0;
164     unsigned int pad;
165     unsigned char dir, move, event, idx;
166
167     curX = 0;
168     curY = 0;
169     drawGrid();
170     drawField(x, y, max, buffer);
171
172     for (;;)
173     {
174         waitvsync();
175
176         pad = joystick(3);
177         dir = (unsigned char) pad & 0x0F;
178         event = (unsigned char) (pad >> 8);
179         if (lastEvent && (pad & JOY_BTN_1_MASK))
180             event = EV_FIRE;
181     }

```

```

163
164      /* Auto-repeat is counted in vblanks. This loop runs
once per frame, so
165      'hold' is already a frame count -- ageing it per
pass instead would run
166      the cursor across the whole grid on one push. */
167      move = 0;
168      if (dir != lastDir)
169      {
170          lastDir = dir;
171          hold = REPEAT_FIRST;
172          move = dir;
173      }
174      else if (dir && --hold == 0)
175      {
176          hold = REPEAT_NEXT;
177          move = dir;
178      }
179
180      if (move)
181      {
182          drawCell(curX, curY, 0);
183
184          if ((move & JOY_LEFT_MASK) && curX)
185              curX--;
186          else if ((move & JOY_RIGHT_MASK) && curX + 1 <
cellCols(curY))
187              curX++;
188          else if ((move & JOY_UP_MASK) && curY)
189              curY--;
190          else if ((move & JOY_DOWN_MASK) && curY + 1 <
OSK_ROWS)
191              curY++;
192
193          if (curX >= cellCols(curY))
194              curX = cellCols(curY) - 1;
195
196          drawCell(curX, curY, 1);
197          soundCursor();
198      }
199
200      if (event == lastEvent)
201          continue;
202      lastEvent = event;
203      if (!event)
204          continue;
205
206      /* The keypad is a shortcut over the grid, never a
way into it: a digit
207      types itself, * backspaces and # accepts, and the
fire button is what
208      takes whatever the cursor is sitting on. */
209      if (event >= '0' && event <= '9')
210      {
211          appendChar(buffer, &len, max, (char) event);
212          drawField(x, y, max, buffer);
213          continue;
214      }
215
216      idx = curY * OSK_COLS + curX;
217      if (event == '*')
218          idx = CELL_DEL;
219      else if (event == '#')
220          idx = CELL_OK;
221      else if (event != EV_FIRE)
222          continue;
223
224      if (idx == CELL_OK)
225      {
226          if (!len)
227          {
228              soundCursorInvalid();
229              continue;
230          }
231          break;
232      }
233
234      if (idx == CELL_DEL)
235      {
236          if (!len)
237          {
238              soundCursorInvalid();
239              continue;
240          }
241          buffer[--len] = 0;
242          soundCursor();
243      }
244      else
245          appendChar(buffer, &len, max, idx == CELL_SPACE ? '
' : cells[idx]);
246
247      drawField(x, y, max, buffer);
248  }
249
250  clearGrid();
251  }
252
253  #endif /* BUILD_COLECO */
254

```

COLECO/SOUND.C

```

1 #ifdef BUILD_COLECO
2
3 /**
4  * @brief ColecoVision Sound Routines (SN76489)
5  * @author Thomas Cherryhomes
6  * @email thom dot cherryhomes at gmail dot com
7  * @license gpl v. 3, see LICENSE for details
8  */
9
10 #include <stdint.h>
11
12 void waitvsync(void);
13
14 // The SN76489 sits at I/O port 0xFF (write only).
15 __sfr __at 0xFF psg_port;
16
17 static void sn_out(uint8_t b)
18 {
19     psg_port = b;
20 }
21
22 /**
23  * @brief Brain dead beep routine
24  * @param hz Frequency in Hz
25  * @param gate Delay in vertical blanks
26  * @param postGate Delay after tone off in vertical
27  * blanks
28  */
29 void beep(int hz, int gate, int postGate)
30 {
31     // 10-bit divider from the 3.58MHz color clock;
32     // lowest reachable
33     // tone is ~109Hz, so the sub-bass beeps clamp there.
34     uint16_t n = 3579545UL / (32UL * (uint16_t)hz);
35     if (n > 1023)
36         n = 1023;
37     sn_out(0x80 | (n & 0x0F)); // channel 0 tone,
38     // low 4 bits
39     sn_out((n >> 4) & 0x3F); // high 6 bits
40     sn_out(0x90 | 0x02); // channel 0
41     // attenuation -4dB
42     while (gate--)
43         waitvsync();
44     sn_out(0x9F); // channel 0 off
45     while (postGate--)
46         waitvsync();
47 }
48
49 void soundDealCard()
50 {
51     beep(150,1,5);
52 }
53
54 void soundJoinGame()
55 {
56     beep(430,5,8);
57     beep(340,5,0);
58     beep(500,5,0);
59 }
60
61 void soundPlayerLeft()
62 {
63     uint8_t i;
64     for (i=80;i>=50;i-=10)
65         beep(i,2,15);
66 }
67
68 void soundPlayerJoin()
69 {
70     uint8_t i;
71     for (i=50;i<=80;i+=10)
72         beep(i,2,15);
73 }
74
75 void soundGameDone()
76 {
77     beep(311,10,0);
78     beep(330,20,0);
79     beep(392,10,0);
80     beep(415,20,0);
81 }
82
83 void soundMyTurn()
84 {
85     beep(430,4,2);
86     beep(430,4,2);
87 }
88
89 void soundTick()
90 {
91     beep(50,2,0);
92 }
93
94 void soundCursor()
95 {
96     beep(300,2,0);
97 }
98
99 void soundCursorInvalid()
100 {
101     beep(100,2,0);
102 }
103
104 void soundSelectMove()
105 {
106     beep(300,3,1);
107     beep(350,3,0);
108 }
109
110 void initSound()
111 {
112     // Silence all four channels
113     sn_out(0x9F);
114     sn_out(0xBF);
115     sn_out(0xDF);
116     sn_out(0xFF);
117 }
118
119 void soundTakeChip(uint16_t counter)
120 {
121     beep(50+counter*20,2,2);
122 }
123
124 #endif /* BUILD_COLECO */

```

COLECO/APPKEY.C

```

1 #ifndef BUILD_COLECO
2
3 /**
4  * @brief   ColecoVision App Key Functions
5  * @author  Thomas Cherryhomes
6  * @email   thom dot cherryhomes at gmail dot com
7  * @license gpl v. 3, see LICENSE for details
8  */
9
10 #include <stdint.h>
11 #include <string.h>
12 #include <fujinet-fuji.h>
13
14 #include "../platform-specific/appkey.h"
15
16 void read_appkey(unsigned int creator_id, unsigned char app_id, unsigned char key_id, char *data)
17 {
18     uint16_t read = 0;
19
20     fuji_set_appkey_details(creator_id, app_id, DEFAULT);
21     if (!fuji_read_appkey(key_id, &read, (uint8_t *) data))
22         read = 0;
23
24     // Terminate as a string. Callers hand us the shared tempBuffer scratch
25     // global and then strlen() it, so this has to happen even on failure or
26     // they read back whatever the previous screen left behind.
27     data[read] = 0;
28 }
29
30 void write_appkey(unsigned int creator_id, unsigned char app_id, unsigned char key_id, const char *data)
31 {
32     fuji_set_appkey_details(creator_id, app_id, DEFAULT);
33     fuji_write_appkey(key_id, strlen(data), (uint8_t *) data);
34 }
35
36 #endif /* BUILD_COLECO */
37

```

COLECO/CONBUF.ASM

```

1 ; Override z88dk's 256-byte __tms9918_scroll_buffer.
2 ;
3 ; The library sizes that buffer for a whole GRAPHICS II text row -- 32 cells of
4 ; 8 pattern bytes -- because scrolling one row means staging one row. A quarter
5 ; of the ColecoVision's entire 1K of RAM, for a client that never scrolls: this
6 ; screen is drawn by absolute positioning, and initGraphics() sets bit 6 of
7 ; generic_console_flags, which is what makes the console skip its scrollup call
8 ; outright (see fputc_cons_generic.inc). The two go together -- shrink this
9 ; without disabling the scroll and a status line printed to the last column of
10 ; row 23 walks 256 bytes through the globals.
11 ;
12 ; What is left is the only other use of the symbol on this path:
13 ; __tms9918_mode2_printc stages one 8-byte glyph here before LDIRVM'ing it into
14 ; VRAM. 16 is that, with room to spare.
15 ;
16 ; Defining it here keeps the library module unreferenced, so the linker never
17 ; pulls it in.
18
19     SECTION bss_user
20
21     PUBLIC __tms9918_scroll_buffer
22
23 __tms9918_scroll_buffer:
24     defs    16
25
26
27 ; z88dk's console flags byte, under the name sccz80 gives a C extern. Bit 6
28 ; disables vertical scrolling; initGraphics() sets it. Same aliasing idiom
29 ; z88dk itself uses for joystick -> coleco_joypad.
30
31     SECTION code_user
32
33     EXTERN generic_console_flags
34     PUBLIC _generic_console_flags
35
36     defc    _generic_console_flags = generic_console_flags
37

```

C.7 BATTLESHIP (ColecoVision port)

MAIN.C

```

1  /**
2  * @brief   Fuji Battleship
3  * @author   Eric Carr, Thomas Cherryhomes, (insert names
here)
4  * @license  gpl v. 3
5  * @verbose  main
6  */
7
8  /
*****
9  *
10 * Do NOT include standard library headers (e.g. conio,
std*).
11 * Instead, add to standard_lib.h, which gets included in
misc.h
12 *
13
14 *****
15 #include "misc.h"
16 #include "stateclient.h"
17 #include "gamelogic.h"
18 #include "screens.h"
19
20 // Store default public server endpoint in case lobby did
not set app key
21 char serverEndpoint[50] = "https://battleship.carr-
designs.com/";
22 //char serverEndpoint[50] = "http://127.0.0.1:8080/";
23
24 // For local dev testing, instead of changing the
endpoint above,
25 // set 1st byte in the e41c0500 appkey to 0xff, which
will cause the below endpoint to be used
26 const char localServer[] = "http://127.0.0.1:8080/";
27
28 // Zero-initialized by the C runtime (BSS) - an explicit
= "" would cost the
29 // z88dk ROM a stored DATA image of the zeros
30 char query[QUERY_LEN]; //"?table=dev7";//
&player=ERICAPL2";
31 char playerName[12];
32
33 // On the ColecoVision this is a macro for the FujiNet
cartridge's reply
34 // window, not an object -- see src/misc.h.
35 #ifndef BUILD_COLECO
36 ClientState clientState;
37 #endif
38 GameState state;
39 PrefsStruct prefs;
40
41 // Common local scope temp variables
42 char tempBuffer[TEMP_BUFFER_LEN];
43 const uint8_t shipSize[5] = {5, 4, 3, 3, 2}; // Standard
ship sizes
44
45 // extern void toneFinder();
46
47 void main(void)
48 {
49     uint8_t failedApiCalls = 0;
50     char ch;
51     // Testing
52     // toneFinder();
53     // printf("Press keys\n");while(1) {while (!
kbhit());failedApiCalls = cgetc();printf("%d 0x%x\n",
failedApiCalls, failedApiCalls);} // Read Key
54
55     loadPrefs();
56     initGraphics();
57     initSound();
58
59     // soundCursor();
60     // cgetc();
61
62     // soundSelect();
63     // cgetc();
64
65     // soundJoinGame();
66     // cgetc();
67
68     // soundMyTurn();
69     // cgetc();
70
71     // soundGameDone();
72     // cgetc();
73
74     // soundTick();
75     // cgetc();
76
77
78     // soundPlaceShip();
79     // cgetc();
80
81     // soundAttack(); // jak stuknięcie, nie wybuch, za
wysoki
82     // cgetc();
83
84     // soundInvalid();
85     // cgetc();
86
87     // soundHit(); // dwa szumy, za wysokie
88     // cgetc();
89
90     showWelcomeScreen();
91     showTableSelectionScreen();
92
93     // Main event loop - process state from server and
input from keyboard/joystick
94     state.apiCallWait = 0;
95
96     while (true)
97     {
98
99         // Poll the server every so often.
100         if (!state.apiCallWait--)
101         {
102
103             // Housekeeping - allows platform specific
housekeeping, like stopping Attract/screensaver mode in Atari
housekeeping();
104
105             // Poll the server
106             switch (getStateFromServer())
107             {
108                 case STATE_UPDATE_ERROR:
109                     // ERROR - Wait a bit to avoid hammering
the server if getting bad responses
110                     // Wait max 4 seconds (since 4*60=240
fits in a single byte)
111                     if (failedApiCalls < 4)
112                     {
113                         failedApiCalls++;
114                     }
115                     state.apiCallWait = 60 * failedApiCalls;
116
117                     // After consecutive failures, let the
player know we are experiencing technical difficulties
118                     if (failedApiCalls > 1)
119                     {
120                         drawConnectionIcon(true);
121                         pause(30);
122                         drawConnectionIcon(false);
123                         pause(30);
124                         drawConnectionIcon(true);
125                     }
126                     break;
127
128                 case STATE_UPDATE_CHANGE:
129
130                     // Clear connection failure message
131                     if (failedApiCalls > 1)
132                     {
133                         drawConnectionIcon(false);
134                     }
135                     failedApiCalls = 0;
136                     processStateChange();
137
138                     // Poll again in a bit
139                     state.apiCallWait = 59;
140                     break;
141                 }
142             }
143
144             processInput();
145         }
146     }
147 }
148

```

COLECO/FUJINET.C -- THE CUSTOM NETWORK HOOK

```

1  #ifndef BUILD_COLECO
2
3  /**
4  * @brief  FujiNet network + appkey hooks
(CUSTOM_FUJINET_CALLS)
5  * @author  Thomas Cherryhomes
6  * @email   thom dot cherryhomes at gmail dot com
7  * @license  gpl v. 3, see LICENSE for details
8  */
9
10 #include <stdint.h>
11 #include <fujinet-network.h>
12 #include <fujinet-fuji.h>
13 #include "vars.h"
14
15 static uint8_t initialized = 0;
16 static uint8_t channelOpen = 0;
17
18 /*
19  The response is never copied into RAM. `buffer` is the
cartridge's reply
20  window (COLECO_REPLY_WINDOW, and see src/misc.h's
clientState) and the game
21  renders straight out of it, because the console has 1K
of RAM and the state
22  alone is 509 bytes. Two things follow:
23
24  1. The read has to be ONE transaction. network_read()'s
loop would copy its
25  second chunk to `buffer + count`, which is cartridge
ROM -- the store goes
26  nowhere and the tail is lost, silently.
network_read_nb() waits for the
27  adapter to report bytes available and then issues
exactly one read, so
28  what the cartridge paints is the whole body. The
window is 1K and the
29  largest response is 509 bytes, so it always fits.
30
31  2. Nothing may run after the read. Every mailbox
transaction repaints the
32  window, network_close() included, so closing here
would destroy the reply
33  before the caller ever looked at it. The channel is
closed at the top of
34  the NEXT call instead.
35  */
36 int16_t custom_network_call(char *url, uint8_t *buffer,
uint16_t max_len)
37 {
38     if (!initialized)
39     {
40         if (network_init() != FN_ERR_OK)
41             return -1;
42         initialized = 1;
43     }
44
45     // The url handed in is the next request's, not the
one that was opened,
46     // but every url this client builds carries the bare
"n:" prefix and so
47     // resolves to the same network unit.
48     if (channelOpen)
49     {
50         network_close(url);
51         channelOpen = 0;
52     }
53
54     if (network_open(url, OPEN_MODE_HTTP_GET,
OPEN_TRANS_NONE) != FN_ERR_OK)
55         return -1;
56     channelOpen = 1;
57
58     return network_read_nb(url, buffer, max_len);
59 }
60
61 uint16_t custom_read_appkey(uint16_t creator_id, uint8_t
app_id, uint8_t key_id, char *destination)
62 {
63     uint16_t read = 0;
64
65     fuji_set_appkey_details(creator_id, app_id, DEFAULT);
66     if (!fuji_read_appkey(key_id, &read, (uint8_t
*)destination))
67         read = 0;
68     return read;
69 }
70
71 void custom_write_appkey(uint16_t creator_id, uint8_t
app_id, uint8_t key_id, uint16_t count, char *data)
72 {
73     fuji_set_appkey_details(creator_id, app_id, DEFAULT);
74     fuji_write_appkey(key_id, count, (uint8_t *)data);
75 }
76
77 #endif /* BUILD_COLECO */
78

```

COLECO/VARS.H

```

1 #ifndef BUILD_COLECO
2
3 #ifndef KEYMAP_H
4 #define KEYMAP_H
5
6 // Screen dimensions for platform
7
8 #define WIDTH 32
9 #define HEIGHT 24
10
11 #define GAMEOVER_PROMPT_Y HEIGHT - 2
12
13 // The whole reply to the last mailbox transaction, as
14 // directly addressable
15 // cartridge ROM. fujinet-lib-experimental bus/coleco/
16 // fujinet-bus-coleco.h
17 // (FN_REPLY) is the source of truth; keep the two in
18 // step.
19 // This is where ClientState lives on this platform (see
20 // src/misc.h). The
21 // console has 1K of RAM and the game state alone is 509
22 // bytes, so the state is
23 // never copied down -- the game renders straight out of
24 // the window. Two rules
25 // follow, and neither is optional:
26 //
27 // - it is READ-ONLY. A store here goes nowhere; the
28 // cartridge cannot even
29 // see a write cycle.
30 // - it is valid only until the next fuji_* call of ANY
31 // kind. Every
32 // transaction repaints the window, appkey reads and
33 // writes included.
34 #define COLECO_REPLY_WINDOW 0xF800
35
36 // Icons - msdos tile sheet indices (src/coleco/charset.c
37 // imports all 256)
38 #define ICON_TEXT_CURSOR 0x3A
39 #define ICON_PLAYER 0x2A
40 #define ICON_MARK 0x2B
41 #define ICON_MARK_ALT 0x19
42 #define ICON_ACTIVE_PLAYER 0x05
43
44 /* Macros that evaluate the return code of
45 // readJoystick(). These are z88dk's
46 // coleco_joypad() bits (see joystick.h). The fire
47 // buttons never appear in the
48 // joystick byte -- input.c reports fire-1 as KEY_RETURN
49 // (which the shared
50 // input maps to the trigger) and fire-2 as KEY_ESCAPE,
51 // behind the same edge
52 // detector as the keypad. */
53 #define JOY_UP(v) ((v) & 0x08)
54 #define JOY_DOWN(v) ((v) & 0x04)
55 #define JOY_LEFT(v) ((v) & 0x02)
56 #define JOY_RIGHT(v) ((v) & 0x01)
57 #define JOY_BTN_1(v) ((v) & 0)
58 #define JOY_BTN_2(v) ((v) & 0)
59
60 /**
61 * Platform specific key map for common input
62 */
63
64 // There is no keyboard: a stick, two fire buttons and a
65 // twelve-key keypad.
66 // Direction comes from the stick, never from a key, so
67 // the arrow codes are
68 // deliberately unreachable placeholders -- they exist
69 // because
70 // readCommonInput() switches on all twelve of them.
71 #define KEY_LEFT_ARROW 0xF1
72 #define KEY_LEFT_ARROW_2 0xF2
73 #define KEY_LEFT_ARROW_3 0xF3
74
75 #define KEY_RIGHT_ARROW 0xF4
76 #define KEY_RIGHT_ARROW_2 0xF5
77 #define KEY_RIGHT_ARROW_3 0xF6
78
79 #define KEY_UP_ARROW 0xF7
80 #define KEY_UP_ARROW_2 0xF8
81 #define KEY_UP_ARROW_3 0xF9
82
83 #define KEY_DOWN_ARROW 0xFA
84 #define KEY_DOWN_ARROW_2 0xFB
85 #define KEY_DOWN_ARROW_3 0xFC
86
87 // Keypad # and the fire buttons; keypad * is escape.
88 #define KEY_RETURN 0x0D
89 #define KEY_ESCAPE 0x1B
90 #define KEY_ESCAPE_ALT 0x03
91 #define KEY_SPACEBAR 0x20
92 #define KEY_BACKSPACE 0x08
93
94 // What the shared screens call the escape key
95 #define ESCAPE "*"
96 #define ESC "*"
97
98 // Countdown can reach two digits
99 #define TIMER_WIDTH 2
100
101 // Name entry happens on src/coleco/osk.c's on-screen
102 // keyboard, not
103 // inputFieldCycle() -- there are no letter keys to type
104 // into it.
105 #define USE_PLATFORM_NAME_ENTRY 1
106
107 // The keypad stands in for the letter shortcuts the
108 // other ports type.
109 // src/coleco/input.c maps 1/2/3/4/5 to r/h/s/n/q; keep
110 // the two in step.
111 #define TABLE_STATUS_TEXT "1:Rfrsh 2:Help 4:Name 5:Quit"
112 #define MENU_QUIT_TEXT " 5: quit game"
113 #define MENU_HELP_TEXT " 2: how to play"
114 #define MENU_SOUND_ON_TEXT " 3: sound ON "
115 #define MENU_SOUND_OFF_TEXT " 3: sound OFF"
116 #define ROTATE_PROMPT_TEXT "press 1 to rotate"
117
118 #endif /* KEYMAP_H */
119
120 #endif /* BUILD_COLECO */
121

```

COLECO/GRAPHICS.C -- GRAPHICS II AS A CELLMAP

```

1 #ifndef BUILD_COLECO
2
3 /*
4  Graphics functionality - ColecoVision TMS9918A
5
6  GRAPHICS II driven as a cellmap: z88dk's
7  vdp_set_mode(2) lays down the
8  identity name table (0..255 per screen third) and it is
9  never touched again,
10 so every 32x24 cell owns its own 8 pattern bytes (y*256
11 + x*8) and 8 color
12 bytes (0x2000 + the same). Drawing a cell is two 8-byte
13 VRAM writes; there is
14 no glyph budget, no RAM shadow, and the msdos tile
15 vocabulary carries over
16 verbatim as indices into the tiles[] ROM array.
17
18 Ported from src/msdos/graphics.c (the look) for the
19 ColecoVision FujiNet
20 cartridge bring-up; the same layout is the template for
21 the Adam re-port.
22 */
23
24 #include <stdint.h>
25 #include <stdbool.h>
26 #include <string.h>
27 #include <video/tms99x8.h>
28 #include <interrupt.h>
29 #include "vars.h"
30 #include "../misc.h"
31
32 /**
33  * @brief character set arrays, generated by support/
34  coleco/make_charset.py
35  * tiles: 8 pattern bytes then 8 per-row fg<<4|bg color
36  bytes
37  * font: 1bpp ASCII 0x20-0x7F, colored at draw time
38  */
39 extern const unsigned char tiles[];
40 extern const unsigned char font[];
41
42 #define GLYPH(n) (font + ((unsigned int)(n) << 3))
43
44 /**
45  * @brief resolve a sheet index to its tile data. The
46  generated tiles[] only
47  * carries the referenced spans - 0x00-0x60, then the
48  0x82-0xE8 dim/animation
49  * run - so indices from 0x82 up shift down by the
50  dropped 0x61-0x81 gap.
51  * Keep in step with support/coleco/make_charset.py's
52  TILE_KEEP.
53  */
54 static const unsigned char *tilePtr(unsigned char n)
55 {
56     if (n >= 0x82)
57         n -= 0x21;
58     return tiles + ((unsigned int)n << 4);
59 }
60
61 #define TILE(n) tilePtr(n)
62
63 // Per-row color attributes (TMS fg<<4 | bg): dark blue
64 field, white text,
65 // light red alt text, white-on-red selected name, blue-
66 on-cyan idle name.
67 #define TMS_DARK_BLUE 4
68 #define TMS_CYAN 7
69 #define TMS_MED_RED 8
70 #define TMS_LIGHT_RED 9
71 #define TMS_WHITE 15
72
73 #define A_TEXT ((TMS_WHITE << 4) | TMS_DARK_BLUE)
74 #define A_ALT ((TMS_LIGHT_RED << 4) | TMS_DARK_BLUE)
75 #define A_NAME_ACTIVE ((TMS_WHITE << 4) | TMS_MED_RED)
76 #define A_NAME_IDLE ((TMS_DARK_BLUE << 4) | TMS_CYAN)
77
78 // Color table base in the z88dk mode 2 layout (pattern
79 table is at 0x0000)
80 #define CT_BASE 0x2000
81
82 // util.c
83 extern void myInt(void);
84
85 // The SN76489, write only. sound.c owns the effects;
86 this one is here so
87 // initGraphics() can silence power-on noise before the
88 first (long) network
89 // transaction - the 55AA cartridge header path skips the
90 BIOS silencing.
91 __sfr __at 0xFF psg_port_init;
92
93 /**
94  * @brief Top left of each playfield quadrant (32x24
95  layout)
96  */
97 static const unsigned char quadrant_offset[4][2] =
98 {
99     {5, 12}, // bottom left
100    {5, 1}, // Top left
101    {17, 1}, // top right
102    {17, 12} // bottom right
103 };
104
105 /**
106  * @brief offset of legends for each player
107  */
108 static const signed char legendShipOffset[5][2] =
109 {
110     {2, 0},
111     {1, 0},
112     {0, 0},
113     {0, 5},
114     {1, 6},
115 };
116
117 /**
118  * @brief Horizontal Field offset
119  */
120 static unsigned char fieldX = 0;
121
122 /**
123  * @brief Number of active players
124  */
125 static unsigned char playerCount = 0;
126
127 /**
128  * @brief plot a tile (8 pattern + 8 color bytes) at
129  column x, row y
130  */
131 static void plotTile(const unsigned char *tile, unsigned
132 char x, unsigned char y)
133 {
134     unsigned int addr;
135
136     if (x >= WIDTH || y >= HEIGHT)
137         return;
138
139     addr = ((unsigned int)y << 8) + ((unsigned int)x <<
140 3);
141     vdp_vwrite((void *)tile, addr, 8);
142     vdp_vwrite((void *)tile + 8, CT_BASE + addr, 8);
143 }
144
145 /**
146  * @brief plot two tiles merged (patterns OR'd) at column
147  x, row y.
148  * Used for the one row the top and bottom board borders
149  share on a 24-row
150  * screen (msdos has 25) - each row's color comes from
151  whichever tile has
152  * pixels there.
153  */
154 static void plotTilePair(unsigned char a, unsigned char
155 b, unsigned char x, unsigned char y)
156 {
157     unsigned char tmp[16];
158     const unsigned char *ta = TILE(a);
159     const unsigned char *tb = TILE(b);
160     unsigned char i;
161
162     for (i = 0; i < 8; i++)
163     {
164         tmp[i] = ta[i] | tb[i];
165         tmp[i + 8] = ta[i] ? ta[i + 8] : (tb[i] ? tb[i +
166 8] : ta[i + 8]);
167     }
168     plotTile(tmp, x, y);
169 }
170
171 /**
172  * @brief plot char in the given per-row color attribute
173  */
174 static void plotChar(unsigned char x, unsigned char y,
175 unsigned char attr, char c)
176 {
177     unsigned int addr;
178     unsigned char g = (unsigned char)c - 0x20;
179
180     if (x >= WIDTH || y >= HEIGHT)
181         return;
182     if (g >= 96)
183         g = 0;
184
185     addr = ((unsigned int)y << 8) + ((unsigned int)x <<
186 3);
187     vdp_vwrite((void *)GLYPH(g), addr, 8);

```

```

159     vdp_vfill(CT_BASE + addr, attr, 8);
160 }
161 /**
162  * @brief plot name text on its label background color
163  */
164 static void plotName(unsigned char x, unsigned char y,
165 unsigned char attr, const char *s)
166 {
167     char c;
168     while (c = *s++)
169     {
170         plotChar(x++, y, attr, c);
171     }
172 }
173 }
174
175 /**
176  * @brief Clear screen to the blue field
177  */
178 void resetScreen(void)
179 {
180     waitvsync();
181     vdp_vfill(0x0000, 0x00, 0x1800);
182     vdp_vfill(CT_BASE, A_TEXT, 0x1800);
183 }
184
185 /**
186  * @brief cycle to next color palette
187  */
188 unsigned char cycleNextColor()
189 {
190     return 0;
191 }
192
193 /**
194  * @brief Initialize Graphics mode
195  * @verbose GRAPHICS II as a 32x24 cellmap
196  */
197 void initGraphics()
198 {
199     psg_port_init = 0x9F; // silence all four SN76489
200 channels
201     psg_port_init = 0xBF;
202     psg_port_init = 0xDF;
203     psg_port_init = 0xFF;
204
205     vdp_set_mode(2); // clears VRAM, writes the identity
206 name table
207     vdp_vfill(CT_BASE, A_TEXT, 0x1800);
208     vdp_set_reg(7, TMS_DARK_BLUE); // border matches the
209 field
210     vdp_vpoke(0x1B00, 208); // sprite Y=208: no
211 sprites, ever
212     add_raster_int(myInt);
213 }
214
215 /**
216  * @brief Reset graphics mode - nothing to reset, the
217  * cart never exits
218  */
219 void resetGraphics(void)
220 {
221 }
222
223 /**
224  * @brief Store screen buffer - not supported (no RAM for
225  * a shadow);
226  * the shared core redraws instead
227  */
228 bool saveScreenBuffer()
229 {
230     return false;
231 }
232
233 void restoreScreenBuffer()
234 {
235 }
236
237 /**
238  * @brief Text output
239  */
240 void drawText(unsigned char x, unsigned char y, const
241 char *s)
242 {
243     char c;
244     while (c = *s++)
245     {
246         if (x >= WIDTH)
247         {
248             x = 0;
249             y++;
250         }
251         plotChar(x++, y, A_TEXT, c);
252     }
253 }

```

```

249
250 void drawTextAlt(unsigned char x, unsigned char y, const
251 char *s)
252 {
253     char c;
254     while (c = *s++)
255     {
256         if (c >= 'A' && c <= 'Z')
257         {
258             plotChar(x++, y, A_ALT, c);
259         }
260         else
261         {
262             plotChar(x++, y, A_TEXT, c);
263         }
264     }
265 }
266
267 /**
268  * @brief draw icon (tile) from the msdos sheet
269  */
270 void drawIcon(unsigned char x, unsigned char y, unsigned
271 char icon)
272 {
273     plotTile(TILE(icon), x, y);
274 }
275
276 void drawBlank(unsigned char x, unsigned char y)
277 {
278     plotTile(TILE(0x00), x, y);
279 }
280
281 void drawSpace(unsigned char x, unsigned char y, unsigned
282 char w)
283 {
284     while (w--)
285         drawBlank(x++, y);
286 }
287
288 void drawClock(void)
289 {
290     drawIcon(WIDTH - 1, HEIGHT - 1, 0x1D);
291 }
292
293 void drawConnectionIcon(bool show)
294 {
295     if (show)
296     {
297         drawIcon(0, HEIGHT - 1, 0x1E);
298         drawIcon(1, HEIGHT - 1, 0x1F);
299     }
300     else
301     {
302         drawIcon(0, HEIGHT - 1, 0x00);
303         drawIcon(1, HEIGHT - 1, 0x00);
304     }
305 }
306
307 /**
308  * @brief true when this player's board shares its middle
309  * border row (row 11)
310  * with the board above/below it, which happens whenever
311  * that counterpart is
312  * actually drawn
313  */
314 static bool sharedBorderRow(unsigned char player)
315 {
316     if (player == 0)
317         return playerCount >= 2;
318     if (player == 2)
319         return playerCount == 4;
320     return true; // 1 always has 0 below; 3 only exists
321 in 4-player
322 }
323
324 /**
325  * @brief Draw Player Name plate and board chrome
326  */
327 void drawPlayerName(unsigned char player, const char
328 *name, bool active)
329 {
330     uint8_t x = quadrant_offset[player][0] - 1;
331     uint8_t y = quadrant_offset[player][1] - 1;
332     uint8_t add = active ? 0x00 : 0x80;
333     uint8_t i = 0;
334     bool shared = sharedBorderRow(player);
335
336     x += fieldX;
337
338     if (player == 0 || player == 3)
339     {
340         // Bottom player boards
341
342         // Thin horizontal border (row 11, shared with
343         the top board's lower
344         // border when both are on screen - merge the two
345         tile sets)
346         if (shared)

```

```

338     {
339         plotTilePair(0x08 + add, 0x0A + add, x, y);
340         for (i = 1; i < 11; i++)
341         {
342             plotTilePair(0x27 + add, 0x29 + add, x +
i, y);
343         }
344         plotTilePair(0x09 + add, 0x0B + add, x + 11,
y);
345     }
346     else
347     {
348         drawIcon(x, y, 0x08 + add);
349         for (i = 1; i < 11; i++)
350         {
351             drawIcon(x + i, y, 0x27 + add);
352         }
353         drawIcon(x + 11, y, 0x09 + add);
354     }
355
356     // Name label
357     drawIcon(x, y + 11, 0x5E + add);
358     for (i = 1; i < 11; i++)
359     {
360         drawIcon(x + i, y + 11, 0x60 + add);
361     }
362     drawIcon(x + 11, y + 11, 0x5F + add);
363     plotName(x + 2, y + 11, active ? A_NAME_ACTIVE :
A_NAME_IDLE, name);
364
365     // Active indicator
366     if (active)
367         drawIcon(x + 1, y + 11, 0x5B);
368
369     // Bottom border below name label
370     drawIcon(x, y + 12, 0x20 + add);
371     for (i = 1; i < 11; i++)
372     {
373         drawIcon(x + i, y + 12, 0x28 + add);
374     }
375     drawIcon(x + 11, y + 12, 0x21 + add);
376 }
377 else
378 {
379     // Top player boards
380
381     // Name label
382     drawIcon(x, y, 0x5C + add);
383     for (i = 1; i < 11; i++)
384     {
385         drawIcon(x + i, y, 0x60 + add);
386     }
387     drawIcon(x + 11, y, 0x5D + add);
388     plotName(x + 2, y, active ? A_NAME_ACTIVE :
A_NAME_IDLE, name);
389
390     // Active indicator
391     if (active)
392         drawIcon(x + 1, y, 0x5B);
393
394     // Thin Horizontal Border (row 11, see above)
395     if (shared)
396     {
397         plotTilePair(0x0A + add, 0x08 + add, x, y +
11);
398         for (i = 1; i < 11; i++)
399         {
400             plotTilePair(0x29 + add, 0x27 + add, x +
i, y + 11);
401         }
402         plotTilePair(0x0B + add, 0x09 + add, x + 11,
y + 11);
403     }
404     else
405     {
406         drawIcon(x, y + 11, 0x0A + add);
407         for (i = 1; i < 11; i++)
408         {
409             drawIcon(x + i, y + 11, 0x29 + add);
410         }
411         drawIcon(x + 11, y + 11, 0x0B + add);
412     }
413 }
414
415 // Draw left/right borders and drawers
416 if (player > 1 || playerCount == 2 && player > 0)
417 {
418     // Right drawer
419     // top
420     drawIcon(x + 11, y + 1, 0x25 + add);
421     drawIcon(x + 12, y + 1, 0x31 + add);
422     drawIcon(x + 13, y + 1, 0x31 + add);
423     drawIcon(x + 14, y + 1, 0x31 + add);
424     drawIcon(x + 15, y + 1, 0x2D + add);
425     drawIcon(x, y + 1, 0x22 + add);
426
427     // Edges
428     for (i = 0; i < 8; i++)
429     {
430         drawIcon(x + 11, y + 2 + i, 0x03 + add);
431         drawIcon(x + 15, y + 2 + i, 0x02 + add);
432         drawIcon(x, y + 2 + i, 0x22 + add);
433     }
434
435     // bottom
436     drawIcon(x, y + 10, 0x22 + add);
437     drawIcon(x + 11, y + 10, 0x25 + add);
438     drawIcon(x + 12, y + 10, 0x31 + add);
439     drawIcon(x + 13, y + 10, 0x31 + add);
440     drawIcon(x + 14, y + 10, 0x31 + add);
441     drawIcon(x + 15, y + 10, 0x2F + add);
442 }
443 else
444 {
445     // Left drawer
446     drawIcon(x - 4, y + 1, 0x2C + add);
447     drawIcon(x - 3, y + 1, 0x31 + add);
448     drawIcon(x - 2, y + 1, 0x31 + add);
449     drawIcon(x - 1, y + 1, 0x31 + add);
450     drawIcon(x, y + 1, 0x24 + add);
451     drawIcon(x + 11, y + 1, 0x23 + add);
452
453     // Edges
454     for (i = 0; i < 8; i++)
455     {
456         drawIcon(x - 4, y + 2 + i, 0x02 + add);
457         drawIcon(x, y + 2 + i, 0x02 + add);
458         drawIcon(x + 11, y + 2 + i, 0x23 + add);
459     }
460
461     drawIcon(x - 4, y + 10, 0x2E + add);
462     drawIcon(x - 3, y + 10, 0x31 + add);
463     drawIcon(x - 2, y + 10, 0x31 + add);
464     drawIcon(x - 1, y + 10, 0x31 + add);
465     drawIcon(x, y + 10, 0x24 + add);
466     drawIcon(x + 11, y + 10, 0x23 + add);
467 }
468 }
469
470 /**
471  * @brief Draw the board
472  */
473 void drawBoard(unsigned char currentPlayerCount)
474 {
475     unsigned char i;
476     playerCount = currentPlayerCount;
477
478     fieldX = playerCount > 2 ? 0 : 6;
479
480     for (i = 0; i < playerCount; i++)
481     {
482         drawPlayerName(i, "", false);
483     }
484 }
485
486 /**
487  * @brief draw a horizontal line of w characters at x,y
488  */
489 void drawLine(unsigned char x, unsigned char y, unsigned
char w)
490 {
491     while (w--)
492         plotTile(TILE(0x3F), x++, y);
493 }
494
495 /**
496  * @brief draw ship, shared between a few routines
497  */
498 static void drawShipInternal(unsigned char x, unsigned
char y, unsigned char size, unsigned char delta)
499 {
500     uint8_t c = delta ? 0x37 : 0x32;
501
502     if (delta)
503     {
504         // Vertical
505         drawIcon(x, y++, c--); // top
506
507         while (size > 2) // middle
508         {
509             drawIcon(x, y++, c);
510             size--;
511         }
512
513         c--; // bottom
514         drawIcon(x, y++, c);
515     }
516     else
517     {
518         // Horizontal
519         drawIcon(x++, y, c++); // Left
520
521         while (size > 2) // middle
522         {
523             drawIcon(x++, y, c);
524             size--;

```

```

525     }
526
527     c++; // Right
528     drawIcon(x++, y, c);
529 }
530 }
531
532 /**
533  * @brief Draw ship at position on sea
534  */
535 void drawShip(unsigned char quadrant, unsigned char size,
536 unsigned char pos, bool hide)
537 {
538     uint8_t delta = 0;
539     uint8_t x = 0, y = 0, i = 0;
540
541     if (pos > 99)
542     {
543         delta = 1;
544         pos -= 100;
545     }
546
547     x = pos % 10;
548     y = pos / 10;
549
550     x += quadrant_offset[quadrant][0];
551     y += quadrant_offset[quadrant][1];
552
553     if (hide)
554     {
555         if (!delta)
556         {
557             for (i = 0; i < size; i++)
558             {
559                 drawIcon(x++, y, 0x38);
560             }
561         }
562         else
563         {
564             for (i = 0; i < size; i++)
565             {
566                 drawIcon(x, y++, 0x38);
567             }
568         }
569     }
570     else
571     {
572         drawShipInternal(x, y, size, delta);
573     }
574 }
575
576 /**
577  * @brief Draw a ship in the drawer
578  */
579 void drawLegendShip(uint8_t player, uint8_t index,
580 uint8_t size, uint8_t status)
581 {
582     uint8_t i = 0;
583     uint8_t x = quadrant_offset[player][0] + fieldX +
584 legendShipOffset[index][0];
585     uint8_t y = quadrant_offset[player][1] +
586 legendShipOffset[index][1];
587
588     if (player > 1 || (player > 0 && fieldX > 0))
589     {
590         y++;
591         x += 11;
592     }
593     else
594     {
595         y++;
596         x -= 4;
597     }
598
599     if (status)
600     {
601         drawShipInternal(x, y, size, 1); // draw a nice
602         vertical ship
603     }
604     else
605     {
606         // Draw the red splats, instead
607         for (i = 0; i < size; i++)
608             drawIcon(x, y + i, 0x1C);
609     }
610 }
611
612 /**
613  * @brief Draw game field for given quadrant
614  */
615 void drawGamefield(uint8_t quadrant, uint8_t *field)
616 {
617     uint8_t ix = 0, iy = 0;
618     uint8_t x = quadrant_offset[quadrant][0] + fieldX;
619     uint8_t y = quadrant_offset[quadrant][1];
620
621     for (iy = 0; iy < 10; iy++)
622     {
623         for (ix = 0; ix < 10; ix++)
624         {
625             if (*field)
626             {
627                 drawIcon(x + ix, y + iy, *field == 1 ?
628 0x39 : 0xE1);
629             }
630             field++;
631         }
632     }
633 }
634
635 void drawGamefieldUpdate(uint8_t quadrant, uint8_t
636 *gamefield, uint8_t attackPos, uint8_t anim)
637 {
638     uint8_t x = quadrant_offset[quadrant][0] + fieldX +
639 (attackPos % 10);
640     uint8_t y = quadrant_offset[quadrant][1] +
641 (attackPos / 10);
642     uint8_t c = gamefield[attackPos];
643
644     // Animate attack
645     if (anim > 9)
646     {
647         drawIcon(x, y, 217 + anim);
648         return;
649     }
650
651     if (c == FIELD_ATTACK)
652     {
653         drawIcon(x, y, anim ? 0x1B : 0x39);
654     }
655     else if (c == FIELD_MISS)
656     {
657         drawIcon(x, y, 0xE1);
658     }
659 }
660
661 /**
662  * @brief Draw game field cursor
663  */
664 void drawGamefieldCursor(uint8_t quadrant, uint8_t x,
665 uint8_t y, uint8_t *gamefield, uint8_t blink)
666 {
667     unsigned char ex = quadrant_offset[quadrant][0] +
668 fieldX + x;
669     unsigned char ey = quadrant_offset[quadrant][1] + y;
670     unsigned char pos = (y * 10) + x;
671     unsigned char c;
672
673     switch (gamefield[pos])
674     {
675     case FIELD_ATTACK:
676         c = 0x43;
677         break;
678     case FIELD_MISS:
679         c = 0x46;
680         break;
681     default:
682         c = 0x40;
683         break;
684     }
685
686     drawIcon(ex, ey, c + blink);
687 }
688
689 /**
690  * @brief Draw end game message
691  */
692 void drawEndgameMessage(const char *message)
693 {
694     uint8_t i, x, ix;
695     i = (uint8_t)strlen(message);
696     x = WIDTH / 2 - i / 2;
697
698     for (ix = 0; ix < WIDTH; ix++)
699         drawIcon(ix, HEIGHT - 2, 0xE2);
700
701     drawSpace(0, HEIGHT - 1, WIDTH);
702     drawText(x, HEIGHT - 1, message);
703 }
704
705 /**
706  * @brief Draw a box
707  */
708 void drawBox(unsigned char x, unsigned char y, unsigned
709 char w, unsigned char h)
710 {
711     drawIcon(x, y, 0x3B);
712     drawIcon(x + w + 1, y, 0x3C);
713     drawIcon(x, y + h + 1, 0x3D);
714     drawIcon(x + w + 1, y + h + 1, 0x3E);
715 }
716
717 #endif /* BUILD_COLECO */

```

COLECO/INPUT.C

```

1  #ifndef BUILD_COLECO
2
3  /**
4   * @brief ColecoVision input routines
5   * @author Thomas Cherryhomes
6   * @license gpl v.3
7   */
8
9  #include <games.h>
10 #include "vars.h"
11 #include "joystick.h"
12 #include "../platform-specific/input.h"
13 #include "../platform-specific/graphics.h"
14
15 /*
16  A ColecoVision controller is a four-way stick, two fire
17  buttons and a
18  twelve-key keypad. z88dk's joystick() is
19  coleco_joypad(): device 3 returns
20  the player-1 stick and fire buttons in the low byte and
21  the keypad as an
22  ASCII character in the high byte ('0'-'9', '*', '#').
23
24  Only the stick travels in readJoystick(). Both fire
25  buttons and the keypad
26  are synthesized as key codes behind kbhit()/cgetc() -
27  the surface the
28  keyboard platforms present - with one edge detector so
29  a held key reports
30  exactly once. Fire-1 is KEY_RETURN, which the shared
31  readCommonInput()
32  already maps to input.trigger, and which also satisfies
33  the screens that
34  block on cgetc() ("press any key"); fire-2 is
35  KEY_ESCAPE (the in-game menu).
36  clearCommonInput()'s drain loop terminates because a
37  consumed edge does not
38  re-arm until the key is released.
39  */
40
41 #define PAD_KEYPAD 3 /* joystick() device: stick +
42 keypad, player 1 */
43
44 /* The fire buttons, as values the keypad never returns.
45 */
46 #define EV_FIRE1 0x01
47 #define EV_FIRE2 0x02
48
49 static unsigned char lastEvent;
50 static unsigned char pendingKey;
51
52 unsigned char readJoystick()
53 {
54     return (unsigned char)joystick(PAD_KEYPAD) & 0x0F;
55 }
56
57 static unsigned char mapEvent(unsigned char event)
58 {
59     switch (event)
60     {
61     case EV_FIRE1:
62         return KEY_RETURN;
63     case EV_FIRE2:
64         return KEY_ESCAPE;
65     }
66
67     /* The keypad stands in for the letter shortcuts the
68     other ports type.
69     vars.h's *_TEXT labels tell the player these; keep
70     the two in step. */
71     case '1':
72         return 'r'; /* refresh, and rotate during
73 placement */
74     case '2':
75         return 'h'; /* how to play */
76     case '3':
77         return 's'; /* sound */
78     case '4':
79         return 'n'; /* name */
80     case '5':
81         return 'q'; /* quit */
82
83     case '*':
84         return KEY_ESCAPE;
85     case '#':
86         return KEY_RETURN;
87     }
88     return 0;
89 }
90
91 unsigned char kbhit(void)
92 {
93     unsigned int pad;
94     unsigned char event;
95
96     if (pendingKey)
97         return 1;
98
99     pad = joystick(PAD_KEYPAD);
100    /* The keypad wins over the buttons: it is the
101    deliberate press. */
102    event = (unsigned char)(pad >> 8);
103    if (!event)
104    {
105        if (pad & JOY_BTN_2_MASK)
106            event = EV_FIRE2;
107        else if (pad & JOY_BTN_1_MASK)
108            event = EV_FIRE1;
109    }
110
111    if (event == lastEvent)
112        return 0;
113    lastEvent = event;
114    if (!event)
115        return 0;
116
117    pendingKey = mapEvent(event);
118    return pendingKey != 0;
119 }
120
121 unsigned char cgetc(void)
122 {
123     unsigned char k;
124
125     while (!kbhit())
126         waitvsync();
127
128     k = pendingKey;
129     pendingKey = 0;
130     return k;
131 }
132
133 #endif /* BUILD_COLECO */

```

COLECO/OSK.C

```

1 #ifndef BUILD_COLECO
2
3 /**
4  * @brief   ColecoVision on-screen keyboard
5  * @author   Thomas Cherryhomes
6  * @email    thom dot cherryhomes at gmail dot com
7  * @license  gpl v. 3, see LICENSE for details
8  */
9
10 #include <games.h>
11 #include <string.h>
12 #include "vars.h"
13 #include "joystick.h"
14 #include "../platform-specific/graphics.h"
15 #include "../platform-specific/sound.h"
16
17 /*
18  A ColecoVision controller has twelve keys and a stick,
19  so any text the
20  player has to type gets typed on screen. This is the
21  grid the 5 Card Stud
22  ColecoVision client, the Intellivision client's
23  grid_entry and the
24  ColecoVision CONFIG client's fujiedit.c all use, and
25  they look alike for the
26  same reason: the cursor position IS the character, so
27  there is no shift key
28  and no paging.
29
30  Two rules are inherited deliberately:
31
32  - cancel is the OK cell's absence, not the backspace
33  key. Backspace must
34  never discard the edit; there is no cancel here at
35  all, because the
36  caller only reaches this screen when a name is
37  required.
38  - the grid never runs a mailbox transaction. The
39  caller may be holding
40  data in the cartridge's reply window, and any
41  transaction would repaint
42  it.
43
44  Entry is uppercase-only (the server lowercases names
45  anyway - see
46  welcomeActionVerifyPlayerName()).
47 */
48 #define OSK_COLS 10
49 #define OSK_ROWS 4
50 #define OSK_PITCH 3 /* marker column + two label columns
51 */
52 #define OSK_X 1
53 #define OSK_Y 19 /* below the name box
54 showPlayerNameScreen draws at 16-18 */
55
56 /* The last row is one short: its tenth cell does not
57 exist. */
58 #define OSK_LAST_ROW_COLS 9
59
60 #define CELL_SPACE (OSK_COLS * 3 + 6) /* index 36 */
61 #define CELL_DEL (CELL_SPACE + 1)
62 #define CELL_OK (CELL_SPACE + 2)
63
64 static const char cells[] =
65     "ABCDEFGHJI"
66     "KLMNOPQRST"
67     "UVWXYZ0123"
68     "456789";
69
70 /* The fire button, as a value the keypad never returns.
71 */
72 #define EV_FIRE 1
73
74 #define REPEAT_FIRST 18 /* vblanks before a held stick
75 repeats */
76 #define REPEAT_NEXT 5
77
78 static unsigned char curX, curY;
79
80 static unsigned char cellCols(unsigned char row)
81 {
82     return row == OSK_ROWS - 1 ? OSK_LAST_ROW_COLS :
83     OSK_COLS;
84 }
85
86 static void drawCell(unsigned char col, unsigned char
87 row, unsigned char marked)
88 {
89     unsigned char idx = row * OSK_COLS + col;
90     unsigned char x = OSK_X + col * OSK_PITCH;
91     char glyph[3];
92
93     drawText(x, OSK_Y + row, marked ? ">" : " ");
94
95     if (idx == CELL_SPACE)
96         drawText(x + 1, OSK_Y + row, "SP");
97     else if (idx == CELL_DEL)
98         drawText(x + 1, OSK_Y + row, "<-");
99     else if (idx == CELL_OK)
100         drawText(x + 1, OSK_Y + row, "OK");
101     else
102     {
103         glyph[0] = cells[idx];
104         glyph[1] = ' ';
105         glyph[2] = 0;
106         drawText(x + 1, OSK_Y + row, glyph);
107     }
108 }
109
110 static void drawGrid(void)
111 {
112     unsigned char row, col;
113
114     for (row = 0; row < OSK_ROWS; row++)
115         for (col = 0; col < cellCols(row); col++)
116             drawCell(col, row, row == curY && col ==
117 curX);
118 }
119
120 static void clearGrid(void)
121 {
122     unsigned char row;
123
124     for (row = 0; row < OSK_ROWS; row++)
125         drawSpace(0, OSK_Y + row, WIDTH);
126 }
127
128 /* Repaint the field and its trailing cursor chip, the
129 same shape
130 inputFieldCycle() draws on the platforms that have a
131 keyboard. */
132 static void drawField(unsigned char x, unsigned char y,
133 unsigned char max,
134 const char *buffer)
135 {
136     unsigned char len = (unsigned char)strlen(buffer);
137
138     drawSpace(x, y, max);
139     drawText(x, y, buffer);
140     if (len < max)
141         drawIcon(x + len, y, ICON_TEXT_CURSOR);
142 }
143
144 static void appendChar(char *buffer, unsigned char *len,
145 unsigned char max,
146 char ch)
147 {
148     if (*len >= max)
149     {
150         soundInvalid();
151         return;
152     }
153     buffer[*len++] = ch;
154     buffer[*len] = 0;
155     soundCursor();
156 }
157
158 void platformNameEntry(unsigned char x, unsigned char y,
159 unsigned char max,
160 char *buffer)
161 {
162     unsigned char len = (unsigned char)strlen(buffer);
163     unsigned char hold = 0, lastDir = 0, lastEvent = 0;
164     unsigned int pad;
165     unsigned char dir, move, event, idx;
166
167     curX = 0;
168     curY = 0;
169     drawGrid();
170     drawField(x, y, max, buffer);
171
172     for (;;)
173     {
174         waitvsync();
175
176         pad = joystick(3);
177         dir = (unsigned char)pad & 0x0F;
178         event = (unsigned char)(pad >> 8);
179         if (!event && (pad & (JOY_BTN_1_MASK |
180 JOY_BTN_2_MASK)))
181             event = EV_FIRE;
182
183         /* Auto-repeat is counted in vblanks. This loop
184 runs once per frame,
185 so 'hold' is already a frame count. */
186         move = 0;
187         if (dir != lastDir)

```

```

163     {
164         lastDir = dir;
165         hold = REPEAT_FIRST;
166         move = dir;
167     }
168     else if (dir && --hold == 0)
169     {
170         hold = REPEAT_NEXT;
171         move = dir;
172     }
173
174     if (move)
175     {
176         drawCell(curX, curY, 0);
177
178         if ((move & JOY_LEFT_MASK) && curX)
179             curX--;
180         else if ((move & JOY_RIGHT_MASK) && curX + 1 < cellCols(curY))
181             curX++;
182         else if ((move & JOY_UP_MASK) && curY)
183             curY--;
184         else if ((move & JOY_DOWN_MASK) && curY + 1 < OSK_ROWS)
185             curY++;
186
187         if (curX >= cellCols(curY))
188             curX = cellCols(curY) - 1;
189
190         drawCell(curX, curY, 1);
191         soundCursor();
192     }
193
194     if (event == lastEvent)
195         continue;
196     lastEvent = event;
197     if (!event)
198         continue;
199
200     /* The keypad is a shortcut over the grid, never
a way into it: a
201     digit types itself, * backspaces and #
accepts, and the fire button
202     takes whatever the cursor is sitting on. */
203     if (event >= '0' && event <= '9')
204     {
205         appendChar(buffer, &len, max, (char)event);
206         drawField(x, y, max, buffer);
207         continue;
208     }
209
210     idx = curY * OSK_COLS + curX;
211     if (event == '*')
212         idx = CELL_DEL;
213     else if (event == '#')
214         idx = CELL_OK;
215     else if (event != EV_FIRE)
216         continue;
217
218     if (idx == CELL_OK)
219     {
220         if (!len)
221         {
222             soundInvalid();
223             continue;
224         }
225         break;
226     }
227
228     if (idx == CELL_DEL)
229     {
230         if (!len)
231         {
232             soundInvalid();
233             continue;
234         }
235         buffer[--len] = 0;
236         soundCursor();
237     }
238     else
239         appendChar(buffer, &len, max, idx ==
CELL_SPACE ? ' ' : cells[idx]);
240
241     drawField(x, y, max, buffer);
242 }
243
244 clearGrid();
245 }
246
247 #endif /* BUILD_COLECO */
248

```

COLECO/SOUND.C

```
1 #ifndef BUILD_COLECO
```

```
2
```

```

3 /**
4  * @brief ColecoVision Sound Routines (SN76489)
5  * @author Thomas Cherryhomes
6  * @email thom dot cherryhomes at gmail dot com
7  * @license gpl v. 3, see LICENSE for details
8  *
9  * The effect tables are the msdos port's PC-speaker
frequencies. The SN76489's
10 * tone floor is ~109Hz (10-bit divider from the 3.58MHz
color clock), so the
11 * sub-bass explosion effects (attack/hit/sink at
30-120Hz) play on the noise
12 * channel instead of clamping into thin beeps.
13 */
14
15 #include <stdint.h>
16 #include "../misc.h"
17
18 // The SN76489 sits at I/O port 0xFF (write only).
19 __sfr __at 0xFF psg_port;
20
21 static void sn_out(uint8_t b)
22 {
23     psg_port = b;
24 }
25
26 /**
27  * @brief Beep channel 0
28  * @param hz Frequency in Hz
29  * @param gate Tone-on time in vertical blanks
30  * @param postGate Delay after tone off in vertical
blanks
31 */
32 static void beep(unsigned int hz, uint8_t gate, uint8_t
postGate)
33 {
34     uint16_t n;
35
36     if (prefs.disableSound)
37         return;
38
39     n = 3579545UL / (32UL * hz);
40     if (n > 1023)
41         n = 1023;
42
43     sn_out(0x80 | (n & 0x0F)); // channel 0 tone, low 4
bits
44     sn_out((n >> 4) & 0x3F); // high 6 bits
45     sn_out(0x90 | 0x02); // channel 0 attenuation
-4dB
46
47     while (gate--)
48         waitvsync();
49
50     sn_out(0x9F); // channel 0 off
51
52     while (postGate--)
53         waitvsync();
54 }
55
56 /**
57  * @brief Noise burst for the explosion effects
58  * @param ctrl noise control bits (0x04 = white; rate
0=high..2=low pitch)
59  * @param gate Noise-on time in vertical blanks
60 */
61 static void noise(uint8_t ctrl, uint8_t gate)
62 {
63     if (prefs.disableSound)
64         return;
65
66     sn_out(0xE0 | ctrl);
67     sn_out(0xF0 | 0x02); // noise attenuation -4dB
68
69     while (gate--)
70         waitvsync();
71
72     sn_out(0xFF); // noise off
73 }
74
75 void initSound()
76 {
77     // Silence all four channels - the 55AA cartridge
header path skips the
78     // BIOS power-on silencing
79     sn_out(0x9F);
80     sn_out(0xBF);
81     sn_out(0xDF);
82     sn_out(0xFF);
83 }
84
85 void soundJoinGame()
86 {
87     beep(430, 5, 8);
88     beep(340, 5, 0);
89     beep(500, 5, 0);
90 }
91
92 void soundMyTurn()
93 {
94     beep(430, 4, 2);
95     beep(430, 4, 2);
96 }
97
98 void soundGameDone()
99 {
100     beep(311, 10, 0);
101     beep(330, 20, 0);
102     beep(392, 10, 0);
103     beep(415, 20, 0);
104 }
105
106 void soundCursor()
107 {
108     beep(300, 1, 0);
109 }
110
111 void soundPlaceShip()
112 {
113     beep(300, 3, 1);
114     beep(350, 3, 0);
115 }
116
117 void soundTick()
118 {
119     beep(110, 1, 0);
120 }
121
122 void soundSelect()
123 {
124     beep(350, 2, 1);
125     beep(250, 2, 0);
126     beep(150, 2, 0);
127 }
128
129 void soundMiss()
130 {
131     beep(150, 1, 0);
132     beep(170, 1, 0);
133 }
134
135 void soundInvalid()
136 {
137     beep(150, 2, 2);
138     beep(150, 2, 0);
139 }
140
141 void soundAttack()
142 {
143     // Descending rumble: white noise, high to low shift
rate
144     noise(0x04, 4);
145     noise(0x05, 4);
146     noise(0x06, 4);
147 }
148
149 void soundHit()
150 {
151     noise(0x04, 2);
152     noise(0x05, 3);
153     noise(0x06, 5);
154 }
155
156 void soundSink()
157 {
158     noise(0x04, 4);
159     noise(0x05, 6);
160     noise(0x06, 10);
161 }
162
163 // Not applicable to coleco
164 void soundStop() {}
165 void disableKeySounds() {}
166 void enableKeySounds() {}
167
168 #endif /* BUILD_COLECO */
169

```

COLECO/UTIL.C

```

1 #ifndef BUILD_COLECO
2
3 /**
4  * @brief Utility Functions - ColecoVision
5  * @author Thomas Cherryhomes
6  * @email thom dot cherryhomes at gmail dot com
7  * @license gpl v. 3, see LICENSE for details
8  */
9
10 #include <stdbool.h>
11 #include <stdint.h>
12 #include <string.h>
13 #include <stdlib.h>
14 #include <fujinet-fuji.h>
15 #include <fujinet-coleco.h>
16 #include "vars.h"
17 #include "../platform-specific/graphics.h"
18 #include "../platform-specific/util.h"
19
20 // The firmware's WRITE_HOST_SLOTS expects all 8 slots.
21 #define FUJI_HOST_SLOT_COUNT 8
22
23 #define LOBBY_DEVICE_SLOT 0
24 #define LOBBY_MODE_READ 1
25
26 volatile uint16_t currentTime;
27 static volatile bool vsync;
28 static bool seeded;
29
30 // Installed as the VDP interrupt hook by initGraphics()
31 via add_raster_int().
32 // On this machine that hook is the vblank NMI: it cannot
33 be masked, DI will
34 // not stop it, and it lands in the middle of mailbox
35 transactions by design.
36 // That is harmless only for as long as it stays away
37 from $F800 and up -- a
38 // handler that reads the mailbox pages corrupts whatever
39 transaction it
40 // interrupted. Bumping two counters is safe; do not grow
41 this.
42 void myInt(void)
43 {
44     currentTime++;
45     vsync = true;
46 }
47
48 void waitvsync(void)
49 {
50     while (!vsync)
51         ;
52     vsync = false;
53 }
54
55 void resetTimer(void)
56 {
57     currentTime = 0;
58 }
59
60 uint16_t getTime(void)
61 {
62     return currentTime;
63 }
64
65 uint8_t getJiffiesPerSecond()
66 {
67     return 60;
68 }
69
70 uint8_t getRandomNumber(uint8_t maxExclusive)
71 {
72     if (!seeded)
73     {
74         // Seeded from the vblank count at the first
75         // request, which arrives
76         // only after some human-paced input
77         seeded = true;
78         srand(currentTime);
79     }
80     return (uint8_t)(rand() % maxExclusive);
81 }
82
83 void housekeeping()
84 {
85 }
86
87 static void quitStatus(const char *s)
88 {
89     drawSpace(0, HEIGHT - 1, WIDTH);
90     drawText((WIDTH - (unsigned char)strlen(s)) / 2,
91             HEIGHT - 1, s);
92 }
93
94 static bool sameHost(const char *a, const char *b)
95 {
96     while (*a && *b)
97     {
98         if ((*a | 0x20) != (*b | 0x20))
99             return false;
100         a++;
101         b++;
102     }
103     return *a == *b;
104 }
105
106 /*
107  SET_DEVICE_FULLPATH is a fixed 256-byte payload -- a
108  short one is rejected on
109  the ESP32 side -- so the path is padded here rather
110  than at the call site.
111  In ROM, so the padding costs no RAM.
112 */
113 static const char lobbyPath[MAX_FILENAME_LEN] = "colego/
114 lobby.rom";
115 static const char lobbyHost[] = "ec.tnfs.io";
116
117 /*
118  The 8x32 host slot table is read IN PLACE out of the
119  cartridge's reply
120  window, the same way the game state is (see src/
121  misc.h). READ_HOST_SLOTS is a
122  single transaction and 256 bytes fit inside the 1K
123  window, so the whole table
124  is addressable ROM by the time the call returns --
125  which is the only way it
126  is affordable at all on a 1K machine.
127 */
128 static uint8_t findLobbyHost(void)
129 {
130     HostSlot *slots = (HostSlot *)COLECO_REPLY_WINDOW;
131     uint8_t i;
132
133     if (!fuji_get_host_slots(slots,
134         FUJI_HOST_SLOT_COUNT))
135         return FUJI_HOST_SLOT_COUNT;
136
137     for (i = 0; i < FUJI_HOST_SLOT_COUNT; i++)
138         if (sameHost(lobbyHost, (const char *)slots[i]))
139             return i;
140
141     return FUJI_HOST_SLOT_COUNT;
142 }
143
144 /*
145  The slot is used as found; it is never created.
146  WRITE_HOST_SLOTS is
147  all-or-nothing over the whole 8x32 table, and the seven
148  slots that would have
149  to be preserved only exist in the reply window, which
150  cannot be patched -- so
151  creating one means staging 256 bytes somewhere, and
152  there is nowhere. A
153  player who reached this client through the FujiNet
154  Lobby already has the
155  host; anyone else is told to add it in CONFIG.
156 */
157 static bool mountLobby(uint8_t slot)
158 {
159     if (!fuji_mount_host_slot(slot))
160         return false;
161     if (!fuji_set_device_filename(LOBBY_MODE_READ, slot,
162         LOBBY_DEVICE_SLOT,
163         (char *)lobbyPath))
164         return false;
165
166     // Only starts the transfer. The image is pushed to
167     // the cartridge
168     // asynchronously, after this has already been
169     // answered.
170     return fuji_mount_disk_image(LOBBY_DEVICE_SLOT,
171         LOBBY_MODE_READ) != 0;
172 }
173
174 void quit(void)
175 {
176     uint8_t bootState, slot, pct = 0xFF;
177     char pctText[5];
178
179     quitStatus("LOADING LOBBY...");
180
181     if (!fuji_coleco_present())
182     {
183         quitStatus("NO FUJINET CARTRIDGE");
184         return;
185     }
186
187     slot = findLobbyHost();
188     if (slot == FUJI_HOST_SLOT_COUNT)

```

```
164     {
165         quitStatus("ADD EC.TNFS.IO IN CONFIG");
166         return;
167     }
168
169     if (!mountLobby(slot))
170     {
171         quitStatus("LOBBY NOT AVAILABLE");
172         return;
173     }
174
175     // Watch the cartridge's own progress counter rather
176     // than guessing a delay.
177     for (;;)
178     {
179         bootState = fuji_coleco_boot_state();
180
181         if (bootState == FUJI_COLECO_BOOT_READY)
182             break;
183         if (bootState == FUJI_COLECO_BOOT_FAILED)
184         {
185             quitStatus("LOBBY LOAD FAILED");
186             return;
187         }
188
189         if (fuji_coleco_boot_percent() != pct)
190         {
191             pct = fuji_coleco_boot_percent();
192             itoa(pct, pctText, 10);
193             strcat(pctText, "%");
194             drawText(18, HEIGHT - 1, pctText);
195         }
196     }
197
198     // Stop the firmware serving CONFIG at boot, then
199     // hand the console over.
200     // The swap does not return: every byte of $8000-
201     // $FFFF changes underneath.
202     fuji_set_boot_config(0);
203     fuji_coleco_boot_swap();
204 }
205
206 #endif /* BUILD_COLECO */
```

APPENDIX D

Troubleshooting

In the spirit of the Owner’s Manual’s troubleshooting checklist — the symptoms a FujiNet program actually shows, and what each one means.

SYMPTOM	REMEDY
The client polls forever; <code>FN_ACKSEQ</code> never changes.	The register write did not happen. In C, a mailbox “write” done as <code>(void)FN_REGSEL[reg]</code> is discarded by sccz80 — store through a <code>volatile</code> sink (<code>FN_TOUCH</code>). Check with <code>zcc +coleco -O2 -a</code> : two loads, not one.
A directory or reply reads back all zeros through the right expression.	You indexed the reply window as <code>FN_REPLY[i]</code> . sccz80 mis-indexes the macro’s cast-constant; read through a <code>volatile unsigned char *</code> local instead.
<code>FN_COMMIT</code> returns carry / C returns <code>FN_EWAIT</code> (<code>\$FF</code>).	The cartridge never answered: no FujiNet cart, or the RP2040 never enumerated its ESP32. Check presence (<code>F</code> , <code>N</code> at <code>\$FC09</code> / <code>\$FC0A</code>) first.
<code>FN_ERR</code> is nonzero after a commit.	The cartridge answered but the transport failed: 1 no link, 2 timeout, 3 bad frame, 4 too big. Different from a NAK — this is below the FujiNet, not the FujiNet refusing.
<code>FN_REPLYCMD</code> is <code>\$15</code> (NAK).	Everything carried; the FujiNet refused the command — wrong parameters, a short fixed-size payload, an unmounted host, a missing file, or an opcode this platform does not dispatch.
A short reply shows stale trailing bytes.	The reply window is never cleared. Capture <code>FN_RXLEN</code> the instant a transaction acknowledges and believe only that many bytes.
Every <code>SET_DIRECTORY_POSITION</code> NAKs for the rest of the session.	You read past the end of a directory on an SD host, which hands back <code>..</code> forever instead of <code>\$7F \$7F</code> . Stop on <code>\$7F \$7F</code> , <code>.</code> and <code>..</code> alike, and never read past the end.
The screen looks right but the program is dead / behaves randomly.	Likely the vblank NMI touched the mailbox pages. The NMI cannot be masked; its handler must never read <code>\$F800</code> or above. Keep it to bumping a counter and reading the VDP status port.
The sound chip buzzes from power-on.	A <code>\$55AA</code> client skips the BIOS title screen that would silence the PSG. Call <code>snd_init()</code> first thing.
The build warns about the code or data fence, or <code>checkrom.py</code> fails.	Something reached the mailbox pages (<code>\$F800</code> +). Move it below <code>\$F800</code> ; those pages must hold nothing but the <code>FUJI</code> claim.

Every address and listing in this book is transcribed from the live FujiNet sources and verified against the firmware and a running adapter.

When in doubt, the sources win. — The FujiNet Project · fujinet.online